Merge Depth, Input Steering, and the Coding Core of Matrix Transposition on Tapes

Hans Brende Claude Opus 5.5 (Anthropic)

October 2026, revised November 2026

Abstract

Can a deterministic multitape Turing machine transpose a $d \times d$ matrix with $b(d) = \max(1, \lceil \log_2 d \rceil)$-bit entries in time linear in the input length $L = d^2 b(d)$? Assuming an affirmative answer to Question 9.1 of *Directed Recovery Structures in Tape Computations*, which asks whether storage codes on paired-distance separated relations of bounded degree have rate at most one half, we locate where such a machine could still exist. Question 9.1 has since been answered negatively, in *Storage Codes above Half Rate under Paired-Distance Separation*; so the localization is vacuous as stated, but it holds under the native coding hypotheses that the proofs use, half-rate bounds for the relations that the machines themselves produce, and these remain open. For oblivious machines the depth of the schedule no longer matters: a rank decomposition (hop charging) excludes every transposer whose code merges deep data lineages only boundedly often along its paths, on every schedule, however deeply nested and however many cells it revisits; for linear codes the linear crux of the series suffices. What remains for oblivious machines is depth reduction of the code's merge structure. For adaptive machines we prove a form of an earlier low-adaptivity bound that is robust against padding of bounded entry: if the work heads enter every cell boundedly often, the coarse skeleton of the run, its head positions and output count rounded to multiples of a fixed width, must carry about $L/2$ bits. Hence no linear-time transposer has bounded entry on every tape, and one whose work tapes have bounded entry must revisit input cells unboundedly often and carry about half of the input in the coarse motion of its input head, a condition that padding meets at no cost. Unconditionally, the source-cover exclusions for oblivious geometries extend to machines of sufficiently low adaptivity, and every linear-time transposer, adaptive or not, yields an external-memory transposition with linearly many I/Os that is oblivious on $2^{L-O(d)}$ inputs; an external-memory lower bound without the indivisibility assumption would therefore refute linear-time transposition. We also prove a deep-nesting conjecture of the series for pairs of block labels at one shift, conditional on a conjecture that straddle counts concentrate as the shift varies, with partial results on that conjecture, and we rule out one natural building block for constructions: Tanner codes assembled from projective planes, with every coordinate recovered from one line, violate the paired-distance condition. Every numbered result is formalized in Lean 4 on the standard axioms; the linear crux, the straddle conjecture and an affirmative answer to Question 9.1, which is now known to be false, enter the formal statements as explicit hypotheses. Whether a linear-time transposer exists remains open.

Contributions. Hans Brende chose the problem and directed the research. Claude Opus 5.5, a large language model developed by Anthropic, developed the mathematics, wrote the Lean formalization, and wrote this paper under Hans Brende's direction. The paper builds on *Directed Recovery Structures in Tape Computations* [10] and *Source Covers and Barriers to Tape Transposition* [11], by Hans Brende and GPT-6 Astra (OpenAI), and on *Directed Reciprocity and Deep-Nesting Bounds for Matrix Transposition on Tapes* [9], by Hans Brende and Gemini 4 Argon (Google DeepMind).

Contents

1 Introduction **3**

2 Preliminaries **6**

3 Merge depth replaces schedule depth **8**
3.1 Deep cells and the cold genie . . . 9
3.2 Refinements of the dependency DAG . . . 10
3.3 Hop charging along transport trees . . . 12
3.4 Hop charging along lineages . . . 13
3.5 Exclusion of bounded merge depth . . . 17
3.6 Routing through deep hops, and the number of deep cells . . . 21
3.7 The merge DAG and code-level depth reduction . . . 22

4 Skeleton entropy and input steering **24**
4.1 Skeletons . . . 25
4.2 The block machine . . . 26
4.3 One recovery relation per skeleton class . . . 29
4.4 Paired distances on a skeleton class . . . 31
4.5 Theorem SE . . . 35
4.6 Bounded entry and input steering . . . 37

5 Localization **40**

6 Unconditional reductions **42**
6.1 Low adaptivity . . . 42
6.2 Block scale: an oblivious external-memory image . . . 45

7 Evidence at deep nesting: block pairs **49**
7.1 Setting . . . 49
7.2 Straddle counts and Conjecture S . . . 51
7.3 Half-blindness and witnesses . . . 53
7.4 Block pairs at one odd shift . . . 56
7.5 Partial results on Conjecture S . . . 57

8 What a linear-time transposer would need **61**

9 Open problems **65**

A Index of Lean declarations **66**

1 Introduction

The question. Let $b(d) = \max(1, \lceil \log_2 d \rceil)$ and $L = d^2 b(d)$. Statement (T) asserts that one deterministic multitape Turing machine transposes every $d \times d$ matrix with $b(d)$-bit entries, given and returned in row-major order, within $O(L)$ steps. The obvious recursive algorithm takes $O(L \log d)$ steps. By a reduction of Harvey and van der Hoeven, discussed in the first paper of the series [6], a linear-time multiplication machine would give a linear-time transposer, so a proof of $\neg$(T) would give a superlinear lower bound for integer multiplication on multitape Turing machines. Whether (T) holds is open, and this paper does not decide it. It is the eleventh paper of a series that collects the necessary conditions a linear-time transposer must satisfy and the constructions they leave open.

Where the series stands. The fourth paper reduced the impossibility of oblivious linear-time transposers to two statements [4]: *depth reduction*, that the dependency DAG of every linear-time schedule can be cut to bounded depth by a cheap genie, and a coding bound on the remaining bounded-depth cones, the *crux*. On the depth side, the eighth paper's Theorem F cuts the walk DAG of the work heads to bounded depth whenever the heads have bounded owner-load, in particular whenever they enter every cell boundedly often [3, Theorem 3.6 and Corollary 3.2]; whether such cuts exist for all pairs of walks is the open statement TCP(2), and the deep-nesting pairs are its sharpest test case [3, 9]. On the coding side, the fifth paper showed that the cone relations of every linear-time schedule are paired-distance separated after a paid trimming, and its Question 9.1 asks whether every storage code on such relations of bounded degree has rate at most one half [10]. The answer is affirmative on several classes of relations [10, 7, 9]; we write [Q9.1] for an affirmative answer in general. Together with Theorem F, [Q9.1] excludes oblivious transposers whose work heads enter every cell boundedly often (the eighth paper's Corollary F′ [3, Corollary 6.3]). Independently of coding hypotheses, the ninth paper excludes oblivious geometries with bounded span overlap and few large return families [11], and the frozen gap-area certificate cannot exclude the remaining two-exception inverse-square case at moduli comparable to the protected rank [5]. For adaptive machines, the eighth paper's Corollary F″ extends Corollary F′, given [Q9.1], to bounded-entry machines whose traces carry at most ηL bits for a fixed $\eta < \frac{1}{2}$ [3, Corollary 6.4]; padding the motion of the input head defeats that hypothesis without changing the computation [3, Section 6.2].

Update (November 2026). The thirteenth paper of the series has answered Question 9.1 negatively [8, Theorem A and Corollary A′]: there are recovery relations of bounded, though astronomically large, incidence degree on which every paired distance tends to infinity and which carry storage codes of rate above 0.5021, and binary ones of rate above 0.5013. So [Q9.1] is false, and so are its Lean forms `MergeDepth.Q91` and `MergeDepth.Q91Binary`. The results below that assume it, Lemma 3.15(b), Corollary HC*, Lemma 4.12, Theorem SE, Corollaries IS and RV, Theorem L and Remark 3.23(c), remain true as implications, and we leave them and their Lean forms unchanged, but as stated they are vacuous. Their proofs use [Q9.1] only through half-rate bounds for relations inside balls of bounded radius and for the relations of skeleton classes, the native hypotheses of Section 2. These are open: the thirteenth paper's relations are not known to arise from tape machines, and it constructs no transposer. The localization therefore holds under the native hypotheses; for Corollary HC* the all-code crux for ball systems serves as well, and for Corollary RV a deficit bound (Remark 4.19, a proof sketch only). The other results, among them Theorems HC and HC′, Corollary HC given the linear crux, Theorems LA and EM, Corollary EM, Theorem BP given Conjecture S and Lemma PG, are not affected.

This paper. Given [Q9.1], we show that every linear-time transposer revisits some tape unboundedly often, that an oblivious one must in addition merge deep lineages unboundedly often in its

code, and that one whose work tapes have bounded entry must steer its input head by the data. Unconditionally, at block scale every linear-time transposer, adaptive or not, runs an oblivious external-memory transposition with linearly many I/Os on a large set of inputs, so that one coding problem underlies all cases.

1. *Merge depth replaces schedule depth* (Section 3). The eighth paper's Corollary F cuts the whole schedule to bounded depth with Theorem F, which needs bounded owner-load; schedules that revisit cells unboundedly often, such as the deep-nesting pairs, may have no such cut. Hop charging handles them. Theorem F cuts only the hops at boundedly entered cells (the cold genie, Lemma 3.1); the hops at the other cells, the deep hops, are charged instead, one revealed coordinate for every τ deep hops along a lineage of the code. If the code merges deep lineages at most s times along every path from most outputs, then, on *every* schedule, however deeply nested and however many cells it revisits, the transposition matrix splits into a part of small rank and a part supported on balls of a radius that does not depend on the schedule (Theorems HC and HC$'$, for linear codes), and for all codes the side information inside such balls has a correspondingly short index code (Theorem HC$'$(b)). So the linear crux excludes such transposers with linear codes (Corollary HC), and [Q9.1] excludes them for all codes (Corollary HC*). This answers, under these hypotheses and for codes of bounded merge depth, the eighth paper's question whether a scratch of revisited cells can help [3, Problem 10.5]. Routing through deep hops is limited by their number (Proposition 3.17), the number of deep cells adds nothing (Proposition 3.18), and a cheap cut of the code's merge structure would suffice (Proposition 3.21). What remains for oblivious machines is depth reduction of that structure (Conjecture HV*, Conjecture 3.22).

2. *Skeleton entropy and input steering* (Section 4). Theorem SE is the form of Corollary F$''$ that is robust against padding of bounded entry, as the eighth paper asks: it bounds the entropy of the coarse skeleton at a fixed scale w, the head positions and the output count rounded down to multiples of w, instead of the entropy of the trace, and it contains Corollary F$''$. Its proof turns the machine into a *block machine* that is oblivious on each skeleton class, so that Theorem F, the cone lemma and the paired-distance argument of the fifth paper apply class by class. Heads of bounded entry carry almost no coarse entropy at large scales, and two corollaries follow, given [Q9.1]. No linear-time transposer, adaptive or not, has bounded entry on every tape, the input tape included (Corollary RV). A transposer whose work tapes have bounded entry must therefore revisit input cells unboundedly often, and its input head must carry about $L/2$ bits of the data in its coarse motion at every fixed scale (Corollary IS, which contains Corollary F$'$ and extends it to adaptive machines). Padding of bounded entry before the run does not help a machine meet this condition, but padding that revisits input cells unboundedly often meets it at no cost (Remark 4.18): it is a necessary condition, not an obstruction.

3. *Localization* (Section 5). Theorem L combines these. Given [Q9.1], every linear-time transposer revisits some tape unboundedly often; if it is oblivious, it revisits a work tape and its code merges deep lineages unboundedly often, for every pruning, cold genie and lineage assignment; and if its work tapes have bounded entry, it revisits input cells and carries about half of the input in the coarse motion of its input head. Three kinds of machines remain, and the open problems concern them.

4. *Unconditional reductions* (Section 6). A machine is excluded if, for a small enough $\eta > 0$, it has a trace of probability at least $2^{-\eta L}$ whose return families fall under the source-cover exclusions

(Theorem LA); the passage from a machine to the ninth paper's admissible geometries is formal. At block scale every linear-time transposer, adaptive or not, yields an external-memory transposition with $O(d)$ I/Os, linear in the number of blocks, that is oblivious on a set of $2^{L-O(d)}$ inputs (Theorem EM). External-memory lower bounds of this kind are known only for algorithms that move records as indivisible atoms [1], or conditionally on a network-coding conjecture [14] that is false in general [2]; an indivisibility-free bound for this oblivious, large-set form of transposition would refute (T) for every number of tapes (Corollary 6.5).

5. *Deep nesting* (Section 7). The eighth paper's Conjecture A″ would make the deep-nesting pairs depth-robust and so refute TCP(2). Theorem BP proves it for pairs of block labels of bounded length at one odd shift, with no $\log q$ term and no cost hypothesis, conditional on Conjecture S (Conjecture 7.3): the straddle counts of most coarse classes concentrate as the shift varies. We give partial results on Conjecture S: an exact Fourier form of the normalized variances, a family of classes on which they stay large, and bounds that reduce the conjecture to the average of their non-local part. Theorem BP is evidence that the deep-nesting pairs are depth-robust; it does not decide TCP(2), since general labels remain.

6. *What a construction would need* (Section 8). By the first paper, every linear-time transposer carries, on the block network of its run, a causal code that solves complete exchange on a large set of inputs and beats multicommodity flow by a factor $\Omega(\log d)$ [6, Theorems 8.8 and 9.2]. Section 8 collects what the series now requires of such a machine and states the two forms that remain: an oblivious machine whose code merges deep lineages along patterns without cheap cuts, or an adaptive machine whose revisits the present methods do not constrain further. Lemma PG rules out one natural building block: Tanner codes assembled from projective planes, with every coordinate recovered from one line of a plane, violate the paired-distance condition for every order $q \geq 7$.

How to read this paper: the formal proofs. Every numbered theorem, lemma, corollary and proposition is formalized in Lean 4 with Mathlib, and every declaration that the paper cites depends at most on the axioms `propext`, `Classical.choice` and `Quot.sound`. Each numbered result carries a status line that names its declarations and their file, and numbered definitions carry one when they have a Lean counterpart. The proof of a numbered result is an outline of the formal proof; the Lean statement is authoritative, and the outline says where it is stronger than the printed statement, for instance in its parameter ranges or because it uses only part of the hypotheses. Results that assume an affirmative answer to Question 9.1, the linear crux or Conjecture 7.3 say so in their statements, and their Lean versions take these as explicit hypotheses: `MergeDepth.Q91`, or its binary form `MergeDepth.Q91Binary`, `MergeDepth.LinCrux` and `MergeDepth.ConjectureS`. The formal proofs build on the Lean libraries of the eighth, fifth, ninth and tenth papers, in which the results they use, Theorem F among them, are Lean-verified; what they need from results that earlier papers formalize only in part or not at all, such as the cone lemma, the paired-distance theorem and the uniformity argument of the fifth paper's Proposition 9.2, is reproved in Lean in the forms used here. Results of earlier papers are cited by their numbers at a pinned revision. Claims that have only a proof sketch appear in remarks marked "Proof sketch only", numerical observations in remarks marked as numerical, and heuristics are labelled as such; none of them is used in a proof. Appendix A lists the declarations, and the paper's README explains how to build the library and check its axioms.

Related work. The machine model, the dependency DAGs, genies, the cone lemma and the crux are those of the fourth and eighth papers [4, 3]; the paired-distance theorem and Question 9.1 are

from the fifth [10], and the negative answer to that question from the thirteenth [8]; the source-cover exclusions are from the ninth [11]; the deep-nesting results build on the tenth [9]. Theorem EM is a standard simulation of tapes in the external-memory model of Aggarwal and Vitter [1], followed by the popular-trace argument that also underlies the non-oblivious case of [14]; its point here is the conclusion for transposition with internal memory of a constant number of blocks. The network-coding conjecture behind the conditional bounds of [14] is false: Braverman and He showed that for multiple unicast in undirected networks, coding can beat multicommodity flow by a factor that grows with the size of the network [2].

Organization. Section 2 recalls the notation. Sections 3 and 4 prove the hop-charging and skeleton-entropy theorems, and Section 5 combines them. Section 6 proves the unconditional reductions, and Section 7 the block-pair theorem and the partial results on Conjecture S. Section 8 collects what a construction would need and proves Lemma PG, and Section 9 lists the open problems. Appendix A indexes the Lean declarations.

2 Preliminaries

We use the definitions of the eighth paper of the series [3, Section 2] and recall only what the statements below need.

Machines and (T). A machine is a deterministic Turing machine with a finite state set, a read-only two-way input tape, $k \geq 1$ two-way work tapes over a finite alphabet, and a write-only one-way output tape that receives at most one bit per step; all heads start at cell 0 and move by -1, 0 or $+1$ [3, Section 2.1]. For $d \geq 1$ let $b(d) = \max(1, \lceil \log_2 d \rceil)$ and $L = d^2 b(d)$. The input is a $d \times d$ matrix with $b(d)$-bit entries in row-major order, and the machine must write its transpose: on input w, the output bit at position ℓ is $w_{\mathrm{tpos}(\ell)}$, where $\mathrm{tpos}((di+j)b+s) = (dj+i)b+s$ for $0 \leq i, j < d$ and $0 \leq s < b = b(d)$. Statement (T) says that some machine does this within $c(L+1)$ steps, for a constant c and every d; its formal version, `Transpose.LinearTimeTransposable`, is fixed in a frozen file of the first paper's library [6]. A machine is *oblivious* at a length if its head positions, append times and halting time do not depend on the input of that length. The *trace* of a run is the sequence of all head moves together with the append times and the halting time; for a uniformly random input X, $\tau(X)$ is a random trace. The *value* v_t of step t consists of the state after the step, the symbols written and the output bit or its absence; $\beta(M)$ bits suffice to encode it, where $\beta(M)$ is the value width of M [4, Section 4.1].

Walks, hops and entries. A *walk* of length T is a map $x\colon [0,T) \to \mathbb{Z}$ with $|x(t+1) - x(t)| \leq 1$. For a head h and a time u, $\mathrm{prev}_h(u)$ is the last earlier time at which head h was at the cell $x_h(u)$; the pair $(\mathrm{prev}_h(u), u)$ is a *hop*, *long* if $u - \mathrm{prev}_h(u) \geq 2$. An *entry* of a cell c by a walk x is a time t with $x(t) = c$ and either $t = 0$ or $x(t-1) \neq c$ [3, Definition 2.1]. During a long hop (p, u) at a cell c the head stays on one side of c, and the *owner* of the hop is the first time in (p, u) at which the head is farthest from c. A set of long hops has *owner-load at most* ℓ if at every time t, for every head and side, its hops of that head and side whose open intervals contain t have at most ℓ distinct owners [3, Definition 2.2]. If every cell is entered at most V times, the long hops have owner-load at most $2V - 1$ [3, Corollary 3.2]. The *walk DAG* of walks $x_1, \ldots, x_k$ has vertex set $[0,T)$, *state edges* $t \to t-1$ and *hop edges* $u \to p$ for the long hops (p, u); a *cut* is a set of edges, of *depth at most* D if every path avoiding it has at most D hop edges [3, Section 2.3].

Theorem F. For walks $x_1, \dots, x_k$ of length T, coupled arbitrarily, a set of their long hops of owner-load at most ℓ and $\varepsilon \in (0,1]$, the corresponding sub-DAG of the walk DAG has a cut of at most εT edges and depth less than a number $D_{\mathrm{F}}(k,\ell,\varepsilon) = 2^{O(k^3\ell^3/\varepsilon)}$ that does not depend on T, on the coupling or on the scales of the walks [3, Theorem 3.6]. Theorem F is Lean-verified there; it is the only depth-reduction input of this paper.

Dependency DAGs, genies and cones. For an oblivious machine M at length L, or for one trace τ of any machine, the *dependency DAG* H_M or H_τ has vertex set $[0,T)$, state edges $t \to t-1$ and cell edges $t \to \mathrm{prev}_j(t)$ along the work-head walks; the input head adds no edges [3, Section 2.4]. A *genie* is a set $\Gamma \subseteq [0,T)$ of revealed steps, of cost $\beta|\Gamma|$. A path $t_0 \to \cdots \to t_n$ is Γ-*free* if $t_1, \dots, t_n \notin \Gamma$, and the genie has *full depth at most* D if every Γ-free path has at most D edges. A cut of the walk DAG of small size and bounded depth yields a genie of small cost and bounded full depth [3, Lemma 2.6]. By the cone lemma of the fourth paper [4, Lemma 4.4 and Corollary 4.5], output bit ℓ is then a function of the revealed word and of the input bits at a set $I_\Gamma(\ell)$ of at most $(k+1)^{D+1}$ positions.

Storage codes and Question 9.1. A *recovery relation* on a finite coordinate set V assigns to every $q \in V$ a set $N(q) \subseteq V$. A *storage code* for it over a finite alphabet $\mathcal{A}$ is a set $\mathcal{C} \subseteq \mathcal{A}^V$ in which the value at every coordinate q is determined by the values on $N(q)$, and its *rate* is $\log_{|\mathcal{A}|}|\mathcal{C}|/|V|$ [10, Definition 2.1]. The *incidence graph* $E(N)$ has vertices L_p and R_p for $p \in V$ and an edge L_pR_q whenever $p \in N(q)$; a sequence of relations satisfies the *paired-distance condition* if for every fixed r only $o(|V|)$ coordinates p have L_p and R_p at distance at most r in $E(N)$ [10, Definition 2.3]. If M is oblivious and transposes, every fibre of the revealed word of a genie is a binary storage code for the *cone relation* $N_\Gamma(q) = I_\Gamma(\mathrm{tpos}(q))$ [4, Remark 4.9(b)]. *Directed Recovery Structures in Tape Computations* shows that the cone relations of every linear-time schedule, and of every single frozen trace, satisfy the paired-distance condition after a paid deletion of few coordinates [10, Theorem 3.3, Corollary 3.8 and Remark 3.9], and asks whether every storage code on relations of bounded incidence degree that satisfy the paired-distance condition has rate at most $\frac{1}{2} + o(1)$, for every alphabet [10, Question 9.1]. We write *[Q9.1]* for an affirmative answer to that question for every degree bound; in Lean it is `MergeDepth.Q91`, and its binary form `MergeDepth.Q91Binary`, which follows from it (`MergeDepth.Q91.binary`), is all that the formal proofs below use. By [10, Proposition 9.2], [Q9.1] bounds by $\frac{1}{2} + o(1)$ the rate of every storage code on the cone relations of linear-time schedules with genies of bounded full depth, uniformly; the eighth paper's Corollaries F′ and F″ use it in this form [3, Corollaries 6.3 and 6.4]. Here [Q9.1] enters in the same way, through analogues of that proposition for relations inside balls of bounded radius (Lemma 3.15) and for the relations of skeleton classes (Lemma 4.12).

Update (November 2026). Question 9.1 has a negative answer. The thirteenth paper of the series constructs recovery relations of incidence degree at most a bound $\Delta < 127^{8\,791\,448}$, on which every paired distance tends to infinity, with $\mathbb{F}_{127^2}$-linear storage codes of rate at least $8100/16130 > 0.5021$, and binary storage codes, not $\mathbb{F}_2$-linear, of rate above 0.5013 at incidence degree at most 14Δ [8, Theorem A and Corollary A′]. So [Q9.1] is false, and so are `MergeDepth.Q91` and `MergeDepth.Q91Binary`; the thirteenth paper's Lean library derives their negations from explicit hypotheses that it proves in writing but does not formalize. The native statements through which [Q9.1] enters here are the conclusions of Lemma 3.15(b) and Lemma 4.12: storage codes on relations inside balls of bounded radius, and binary storage codes on the relations of skeleton classes, have rate at most $\frac{1}{2} + o(1)$, uniformly, as for the cone relations in the conclusion of the fifth paper's Proposition 9.2. The thirteenth paper does not decide them, since its relations are not known to

arise from tape machines, and they remain open. The other results that assume [Q9.1] use it only through these two conclusions, so they hold with them as hypotheses in place of [Q9.1]; as stated, all of them are vacuous.

The linear crux. For machines whose step values are vectors over a field $\mathbb{F}$ and whose updates are $\mathbb{F}$-linear, the eighth paper states the *linear crux* $\mathrm{LinCrux}_{\mathbb{F}}$ as relative rigidity of the transposition matrix with respect to the ball pattern of the schedule [3, Definition 7.4 and Theorem 7.5]; we write *[LinCrux]* for it, and in Lean it is `MergeDepth.LinCrux`. Corollary HC (Corollary 3.13) states the form used.

Admissible geometries. The source-cover paper [11, Definition 2.2] models one fixed computation geometry, a set of inputs on which it is correct, records of bounded capacity determined by the actual parents, a span partition of its return families, a core and a bounded number of exceptional families, as an *admissible geometry*; Section 6 uses its exclusions of geometries with few large return families [11, Corollaries 4.3 and 4.5].

Dependencies. Results that assume [Q9.1], [LinCrux] or Conjecture S (Conjecture 7.3) say so in their statements, and their Lean versions take them as hypotheses. Theorem F and the other results of earlier papers that the formal proofs use are Lean-verified in the libraries of those papers. Of the results recalled above, the fourth paper's cone lemma and the fifth paper's Proposition 9.2 are proved only in writing there, and the fifth paper's paired-distance theorem is formalized there only in part; the formal proofs below reprove them in the forms they need (Lemmas 3.3, 3.15, 4.7 and 4.12 and Proposition 4.11).

3 Merge depth replaces schedule depth

Corollary F of the eighth paper excludes, given a crux, every oblivious linear-time transposer whose work heads enter every cell boundedly often [3, Corollary 6.1]; its proof cuts the whole schedule to bounded depth with Theorem F. A schedule that revisits a few cells unboundedly often need not have such a cut: the deep-nesting pairs are depth-robust if the eighth paper's Conjecture A′ holds [3, Corollary 5.26]. The eighth paper therefore asks whether a transposer can use such a scratch to supply the information that the crux denies to the shallow part of its schedule [3, Problem 10.5].

Given a crux, this section answers that question in the negative for codes that merge deep data only boundedly often. Theorem F still cuts the hops at boundedly entered cells; the hops at the other cells are charged instead of cut. Along every lineage of the code, a chain of nodes each of which continues one earlier value, a checkpoint is revealed every τ deep hops, at a total cost of $O(L/\tau)$. If the code merges deep lineages at most s times along every path, the transposition matrix then splits into a part of small rank and a part supported on balls whose radius does not depend on the schedule, and for codes that need not be linear the revealed values form a short index code for such balls (Theorems HC and HC′). So the linear crux at that radius excludes such linear transposers on every schedule, whatever the number of revisited cells (Corollary HC), and an affirmative answer to Question 9.1 of the fifth paper excludes them for all codes (Corollary HC*). What remains concerns the code, not the schedule: routing through deep hops is limited by their number, the number of deep cells adds nothing, and a cheap depth reduction of the code's merge structure would suffice (Propositions 3.17, 3.18 and 3.21, Conjecture 3.22). Every numbered result of this section is formalized in Lean, with the linear crux or [Q9.1] as a hypothesis where the statement assumes it; the outlines say where a formal statement is more general than the written one.

3.1 Deep cells and the cold genie

Throughout this section M is a machine with k work tapes that is oblivious at a length $L = d^2 b(d)$ with halting time $T \le C(L+1)$, and $\beta \ge \beta(M)$; here $C \ge 1$ and β are integers, which loses nothing, since a real bound C may be replaced by $\lceil C \rceil$ (Corollaries HC and HC* are stated for real C), and $\beta(M) \ge 2$. We use the dependency DAG H_M, the work-head walks $x_1, \dots, x_k$ and the input-head walk x_0 as in the eighth paper [3, Sections 2.2 and 2.4]; output bit ℓ is appended at step t_ℓ, and $\mathrm{out}(t)$ is the set of out-neighbours of t in H_M. A path $t_0 \to \cdots \to t_n$ is Γ-*free* if $t_1, \dots, t_n \notin \Gamma$.

Fix an integer $V \ge 1$. As in [3, Corollary 3.8], let $\mathcal{R}$ be the set of pairs (h, c) such that the walk x_h enters the cell c more than V times on $[0, T)$, the *deep cells*. A *deep hop* is a long hop of a head h at a cell c with $(h, c) \in \mathcal{R}$, and $N_{\mathcal{R}}$ is the number of deep hops, counted per head. As for the walk DAG [3, Section 2.3], we identify parallel edges of H_M, so an edge is determined by its endpoints; we call it *deep* if its endpoints form a deep hop of at least one head. So an edge of length at least two that is not deep is a long hop, for each head that makes it, at a pair outside $\mathcal{R}$. A long hop (p, u) of a head h ends at an entry u of its cell, since $x_h(u-1) \ne x_h(u)$, and distinct hops of h end at distinct times (`MergeDepth.card_allHops_le`); hence

$$\#\{\text{deep edges of } H_M\} \;\le\; N_{\mathcal{R}} \;\le\; kT \;\le\; kC(L+1). \tag{1}$$

Lemma 3.1 (The cold genie)**.** Lean: `MergeDepth.cold_genie_exists` [`MergeDepth/HopMachine.lean`]

Let $\varepsilon > 0$, $D_1 = D_{\mathrm{F}}(k, 2V-1, \varepsilon/(4\beta C))$, $P = \lceil 4\beta C/\varepsilon \rceil$ and $D = D_1 P$. There is a set $\Gamma_{\mathrm{F}} \subseteq [0, T)$ with $\beta|\Gamma_{\mathrm{F}}| \le \varepsilon L$ such that every Γ_{F}-free path of H_M that contains no deep edge has at most $D-1$ edges.

Outline of the formal proof. Let H' be the set of long hops of the work heads at pairs outside $\mathcal{R}$. Each of them lies at a cell that its head enters at most V times, so H' has owner-load at most $2V-1$ [3, Corollary 3.2], and Theorem F [3, Theorem 3.6], both Lean-verified there, applied at $\varepsilon' = \varepsilon/(4\beta C)$, gives a cut F of the corresponding sub-DAG $G(H')$ of the walk DAG with at most $\varepsilon' T$ edges and depth less than D_1. Let Γ_0 be the set of steps at which an edge of F ends, and $\Gamma_{\mathrm{F}} = \Gamma_0 \cup \{t < T : t \equiv P-1 \pmod P\}$. Then $\beta|\Gamma_{\mathrm{F}}| \le \beta\varepsilon' T + \beta T/P \le \varepsilon T/(2C) \le \varepsilon(L+1)/2 \le \varepsilon L$.

Let $\pi = (t_0 \to \cdots \to t_n)$ be a Γ_{F}-free path of H_M without deep edges. An edge of π of length at least two is a long hop of some head, and since the edge is not deep, that hop lies in H'. So every edge of π is an edge of $G(H')$, and none lies in F, because it ends outside Γ_0. Hence π has at most $D_1 - 1$ edges of length at least two. They split π into at most D_1 runs of edges of length one; the vertices of a run after its first are consecutive integers, none congruent to $P-1$ modulo P, so a run has at most $P-1$ edges. Thus $n \le D_1 - 1 + D_1(P-1) = D - 1$.

Formally, the work heads are walks $\mathbb{N} \to \mathbb{Z}$, and the edges of H_M are their dependency steps (`MergeDepth.DepStep`): the state edges below T and the long hops of all heads. A step is deep if some head makes it as a hop at a pair of $\mathcal{R}$ (`MergeDepth.DeepStep`). The cut is the eighth paper's `BoundedNesting.theoremF_owner` on its `BoundedNesting.lightHops`, and `MergeDepth.cold_genie` proves the path bound for every $\varepsilon' > 0$ and $P \ge 1$, with $P|\Gamma_{\mathrm{F}}| \le P\varepsilon' T + T$. □

A set $\Gamma_{\mathrm{F}} \subseteq [0, T)$ with $\beta|\Gamma_{\mathrm{F}}| \le \varepsilon L$ such that every Γ_{F}-free path of H_M without deep edges has at most $D-1$ edges is a *cold genie at (V, ε) with depth D* (`MergeDepth.IsColdGenie`). When no depth is named, D is the number $D_1 P$ of Lemma 3.1; it is defined for every real $C > 0$ and depends only on k, V, β, C and ε, and the lemma shows that cold genies at (V, ε) exist.

Lemma 3.1 is the conversion of [3, Lemma 2.6], with the long-edge conversion [4, Lemma 4.21], restricted to paths without deep edges. It holds for every schedule, because Theorem F is applied

only to the hops at boundedly entered cells. If every work-tape cell is entered at most V times, there are no deep edges, and Γ_{F} is a genie of bounded full depth as in Corollary F.

3.2 Refinements of the dependency DAG

The genies below reveal single coordinates or single fields of a step. We state the combinatorics once, for DAGs that refine H_M.

Definition 3.2 (Refinements)**.** A *refinement* of H_M of *width* m is a finite DAG G with a map $\mathrm{st}\colon V(G) \to [0,T)$ such that at most m nodes lie at each step and every edge $u \to u'$ of G *projects* to an edge $\mathrm{st}(u) \to \mathrm{st}(u')$ of H_M, together with a set of *input-marked* nodes u, all with $x_0(\mathrm{st}(u)) \in [0,L)$, and, for every output position ℓ, a set of *output nodes* at step t_ℓ. An edge of G is *deep* if its projection is deep. For $\Gamma \subseteq [0,T)$ and a set Q of nodes, a node is *revealed* if it lies at a step of Γ or in Q; a path $u_0 \to \cdots \to u_n$ of G is (Γ,Q)*-free* if $u_1,\dots,u_n$ are not revealed, and Γ*-free* if it is $(\Gamma,\emptyset)$-free. The set $I_{\Gamma,Q}(u)$ consists of the positions $x_0(\mathrm{st}(z))$ of the input-marked nodes z that end (Γ,Q)-free paths starting at u.

A path of G projects to a path of H_M with the same number of edges, and a (Γ,Q)-free path projects to a Γ-free one. Hence, if every (Γ,Q)-free path from an output node u of ℓ that ends at an input-marked node has at most r edges, then $I_{\Gamma,Q}(u) \subseteq B^r(\ell)$, the set of positions read at steps reachable from t_ℓ by paths of at most r edges [3, Definition 7.2].

The coordinate DAG. Suppose that M has an $\mathbb{F}$-linear data path at length L with a representation of width m [3, Definition 7.1]: writing $v_t \in \mathbb{F}^m$ for the encoded value of step t on input $w \in \{0,1\}^L$, we have $v_t = c_t + \sum_{t' \in \mathrm{out}(t)} A_{t,t'} v_{t'} + a_t w_{x_0(t)}$, where the last term is present only if $x_0(t) \in [0,L)$, and output bit ℓ is $\varphi_\ell(v_{t_\ell})$ for an affine functional φ_ℓ with linear part $\varphi_\ell^{\mathrm{lin}}$. The *coordinate DAG* has the nodes (t,i) with $t < T$ and $i \in [m]$, where $\mathrm{st}(t,i) = t$; an edge $(t,i) \to (t',i')$ whenever $t' \in \mathrm{out}(t)$ and $(A_{t,t'})_{ii'} \neq 0$; input marks at the nodes (t,i) with $x_0(t) \in [0,L)$ and $(a_t)_i \neq 0$; and the output nodes (t_ℓ,i) with $(\varphi_\ell^{\mathrm{lin}})_i \neq 0$. It is a refinement of width m.

The field DAG. The *fields* of step t are the state σ_t after the step, the symbol $\kappa_{t,j}$ written on work tape j for $1 \le j \le k$, and the output field o_t, the appended bit or its absence; each takes at most 2^β values. The *predecessors* of a field of step t are σ_{t-1} if $t \ge 1$, and $\kappa_{\mathrm{prev}_j(t),j}$ for every j for which $\mathrm{prev}_j(t)$ is defined. By the local recursion of the fourth paper [4, Lemma 4.2 and its proof], every field of step t is, on $\{0,1\}^L$, a function of its predecessors and, if $x_0(t) \in [0,L)$, of the input bit $w_{x_0(t)}$. A *pruning* chooses for every field f a set $P(f)$ of its predecessors and whether f *keeps* its input bit, such that f is still a function, on $\{0,1\}^L$, of the fields in $P(f)$ and of the kept input bit; keeping everything is a pruning. The *field DAG* of a pruning has the edges $f \to g$ for $g \in P(f)$, input marks at the fields of steps with $x_0(t) \in [0,L)$ that keep their input bit, and the output node o_{t_ℓ} for ℓ. It is a refinement of width $k+2$. A deep edge of it leads from a field of step t to a field of a step $t' \le t-2$, so its head is a symbol field $\kappa_{t',j}$.

Lemma 3.3 (Expansion)**.** `Lean`: `MergeDepth.expansion_cone`, `MergeDepth.coneI`, `MergeDepth.C_eq_zero_of_not_mem_coneI` [`MergeDepth/ExpansionCone.lean`]

Let $\Gamma \subseteq [0,T)$, let Q be a set of nodes of the coordinate DAG or of a field DAG, and let X be a set of output positions. Let P_{tpos} be the $L \times L$ matrix with rows $e_{\mathrm{tpos}(\ell)}$, as in [3, Section 7.2].

(a) Coordinates. *For $w \in \mathbb{F}^L$ define $V_t(w) \in \mathbb{F}^m$ by the recursion of the data path, and for a node $u = (t,i)$ let V_u^{lin} be the linear part of $w \mapsto V_t(w)_i$, a linear form on $\mathbb{F}^L$. Let E be the*

span of the forms V_u^{lin} of the revealed nodes, and let e_j also denote the form $w \mapsto w_j$. Then $V_u^{\mathrm{lin}} \in E + \operatorname{span}\{e_j : j \in I_{\Gamma,Q}(u)\}$ for every node u, and $\dim E \le \beta|\Gamma| + |Q|$. If M transposes all $d \times d$ matrices, then $P_{\mathrm{tpos}} = R + S$ with $\operatorname{rank} R \le \beta|\Gamma| + |Q| + |X|$, where row ℓ of S is zero if $\ell \in X$ and is otherwise supported in the union $I(\ell)$ of the sets $I_{\Gamma,Q}(u)$ over the unrevealed output nodes u of ℓ.

(b) Fields. *If $w, w' \in \{0,1\}^L$ agree on every revealed field and on $I_{\Gamma,Q}(f)$, then $f(w) = f(w')$. If M transposes all $d \times d$ matrices, then $w \mapsto \big((v_t(w))_{t\in\Gamma}, (f(w))_{f\in Q}, (w_{\mathrm{tpos}(\ell)})_{\ell\in X}\big)$ is an index code [4, Definition 4.8] with at most $\beta|\Gamma| + \beta|Q| + |X|$ bits for the system $(I(\ell))_\ell$, where $I(\ell) = I_{\Gamma,Q}(o_{t_\ell})$ if $\ell \notin X$ and o_{t_ℓ} is not revealed, and $I(\ell) = \emptyset$ otherwise.*

Outline of the formal proof. (a) The linear parts satisfy $V^{\mathrm{lin}}_{(t,i)} = \sum_{t'\in\mathrm{out}(t)} \sum_{i'} (A_{t,t'})_{ii'} V^{\mathrm{lin}}_{(t',i')} + (a_t)_i e_{x_0(t)}$, the last term present only if $x_0(t) \in [0, L)$; the nonzero terms belong to the out-edges of (t, i) and to its input mark. Unrolling this recursion along the paths from u and stopping at revealed nodes, as in the eighth paper [3, Section 7.1], writes V_u^{lin} as a combination of the forms of the revealed nodes and of the forms e_j, where the coefficient of e_j is a sum over the (Γ, Q)-free paths from u that end at an input-marked node reading position j. There are finitely many such paths, since steps decrease along paths, and none if $j \notin I_{\Gamma,Q}(u)$. This is the first claim.

The forms of the nodes of Q span at most $|Q|$ dimensions. On binary inputs the formal values agree with the encoded values, $V_t(w) = v_t$, by induction on t, so the linear map $\Phi\colon w \mapsto (V_t(w) - V_t(0))_{t\in\Gamma}$ takes at most $2^{\beta|\Gamma|}$ values on $\{0,1\}^L$. A linear map of rank ρ takes at least 2^ρ values there: it is injective on the binary vectors supported on ρ coordinates whose columns are independent, because the difference of two such vectors has coefficients in $\{-1, 0, 1\}$, not all zero. The forms of the nodes at steps of Γ span the row space of Φ, so $\dim E \le \beta|\Gamma| + |Q|$.

If M transposes, $Y_\ell(w) = \varphi_\ell(V_{t_\ell}(w))$ is an affine function on $\mathbb{F}^L$ that equals $w_{\mathrm{tpos}(\ell)}$ on $\{0,1\}^L$. Evaluating it at 0 and at the unit vectors shows that its linear part $\sum_i (\varphi_\ell^{\mathrm{lin}})_i V^{\mathrm{lin}}_{(t_\ell,i)}$ is the form $e_{\mathrm{tpos}(\ell)}$. A term with $(\varphi_\ell^{\mathrm{lin}})_i \neq 0$ belongs to an output node; it lies in E if the node is revealed, and in $E + \operatorname{span}\{e_j : j \in I(\ell)\}$ otherwise. So $e_{\mathrm{tpos}(\ell)} = r_\ell + s_\ell$ with $r_\ell \in E$ and s_ℓ supported in $I(\ell)$; for $\ell \in X$ put $r_\ell = e_{\mathrm{tpos}(\ell)}$ and $s_\ell = 0$. The matrices with rows r_ℓ and s_ℓ are as claimed, since the rows r_ℓ lie in $E + \operatorname{span}\{e_{\mathrm{tpos}(\ell)} : \ell \in X\}$.

(b) The first claim is the cone lemma of the fourth paper [4, Lemma 4.4 and Remark 4.6], with the same proof by induction on the step of f: f is a function of the fields in $P(f)$ and of its kept input bit, whose position lies in $I_{\Gamma,Q}(f)$; a revealed field in $P(f)$ takes the same value on w and w'; and an unrevealed $g \in P(f)$ satisfies $I_{\Gamma,Q}(g) \subseteq I_{\Gamma,Q}(f)$, since a (Γ, Q)-free path from g, preceded by the edge $f \to g$, is a (Γ, Q)-free path from f. Now let w, w' have the same image under the map and agree on $I(\ell)$. They agree on every revealed field, since v_t determines the fields of step t. If $\ell \in X$, the bit $w_{\mathrm{tpos}(\ell)}$ is part of the image. Otherwise o_{t_ℓ} takes the same value on w and w', directly or by the first claim, and its bit is $w_{\mathrm{tpos}(\ell)}$. A step value and a field take at most 2^β values each.

Formally, (a) is stated for a linear network whose output ℓ is an affine function of the values at a set of output nodes. `MergeDepth.expansion_cone` is the decomposition, with S supported in the cone sets `MergeDepth.coneI` of the unrevealed output nodes. The path sums vanish outside these sets (`MergeDepth.C_eq_zero_of_not_mem_coneI`), since the paths from a node have bounded length (`MergeDepth.length_le_of_mem_paths`); with the eighth paper's expansion `BoundedNesting.LinNet.vlin_expand` this is the first claim. The bound on $\dim E$ is `MergeDepth.finrank_revealed_union_le`, with `MergeDepth.finrank_revealed_le` for single coordinates and `MergeDepth.finrank_revealed_le_of_values` with `MergeDepth.finrank_revealed_coord_le` for whole steps. In the uses below, the (Γ, Q)-free paths from the output nodes of the positions outside X have a common length bound r, and `MergeDepth.rigidity_finrank` states the

decomposition with row ℓ of S supported in balls of radius r. Part (b) is `MergeDepth.FunDag.cone` with `MergeDepth.FunDag.isIndexCode` for function DAGs, and `MergeDepth.Pruning.field_cone` with `MergeDepth.Pruning.field_isIndexCode` for the field DAG of a pruning. □

For whole steps, the bound on $\dim E$ is the counting of the eighth paper's per-machine rigidity lemma [3, Lemma 7.3], which `MergeDepth.LinDataPath.rigidity_machine` proves for machines, with the translation of a machine into its linear network. The eighth paper's Lean statement charges β per revealed node of a network, so applied to the coordinate DAG it would give only $\beta(m|\Gamma|+|Q|)$; part (a) charges β per revealed step and one per revealed coordinate.

3.3 Hop charging along transport trees

Definition 3.4 (Transport trees and deep formations)**.** Lean: `MergeDepth.IsCopy`, `MergeDepth.copyForest`, `MergeDepth.root`, `MergeDepth.DeepFormation` [`MergeDepth/Formation.lean`]

Let G be a refinement. A *copy* is a node that has exactly one out-edge and is not input-marked; every other node is a *formation*. Following the out-edges of copies from a node u leads, as steps decrease, to a formation $\mathrm{root}(u)$, with $\mathrm{root}(u) = u$ for a formation; the edges followed form the *chain* of u. The *transport tree* of a formation p is the set of nodes u with $\mathrm{root}(u) = p$, and its edges are the out-edges of its copies. Transport trees partition $V(G)$, and the head of an edge of a transport tree lies in the same tree. A transport tree is *τ-tall* if the chain of one of its nodes contains at least τ deep edges. A formation is *deep* if it is the tail of a deep edge or its transport tree contains a deep edge. For $\Gamma_{\mathrm{F}} \subseteq [0, T)$, the *formation depth* at an output position ℓ is the largest number of deep formations on a Γ_{F}-free path of G from an output node of ℓ.

In the coordinate DAG, a copy u with out-neighbour u' satisfies $V_u^{\mathrm{lin}} = \lambda V_{u'}^{\mathrm{lin}}$ for some $\lambda \neq 0$. Storing a value in a cell, carrying it in the state, moving it between cells and multicasting it create no formations; combining it with anything else does.

Theorem 3.5 (Theorem HC: hop charging)**.** Lean: `MergeDepth.hop_charging_HC` [`MergeDepth/HopMachine.lean`]

Let M have an $\mathbb{F}$-linear data path at length L with a representation of width m, let Γ_{F} be a cold genie at (V, ε) with depth D, let $\tau \geq 1$ and $s \geq 0$ be integers, and let X be the set of output positions at which the coordinate DAG has formation depth greater than s. If M transposes all $d \times d$ matrices, then

$$P_{\mathrm{tpos}} = R + S, \qquad \operatorname{rank} R \leq \varepsilon L + \frac{mkC(L+1)}{\tau} + |X|,$$

and row ℓ of S is zero for $\ell \in X$ and supported in $B^{(s+1)\tau D}(\ell)$ otherwise.

Outline of the formal proof. Give every copy its out-edge as lineage edge and no other node a lineage edge; this is a lineage assignment for every Γ_{F} (Definition 3.7 and Remark 3.8). By Lemma 3.12, its merge depth is at most the formation depth at every output position, so Theorem 3.11(a) applies with the same s and X.

Formally, `MergeDepth.hop_charging_HC` derives the theorem in this way from `MergeDepth.hop_charging_machine`, the formal Theorem 3.11(a), with `MergeDepth.copyForest` and `MergeDepth.dm_le_countP_deepFormation`, in the setting described in the outline of Theorem 3.11; `MergeDepth.LinDataPath.minrank_ballSystem_le_formation` is its minrank form for a machine with a data path, for a real constant C. □

Remark 3.6 (A direct proof)**.** Theorem HC also has a direct proof with a simpler genie, the set Q_0 of roots of the τ-tall transport trees, at the slightly larger radius $r_0 = ((s+1)\tau+1)D$: $|Q_0| \le mN_{\mathcal{R}}/\tau$, and every $(\Gamma_{\mathrm{F}}, Q_0)$-free path that starts at an unrevealed output node of a position $\ell \notin X$ and ends at an input-marked node has fewer than r_0 edges. This proof is not formalized.

Counting tall trees. For every tall tree fix a node whose chain contains at least τ deep edges. The tails of the edges of a chain lie at distinct steps, so the deep edges of a fixed chain project to distinct deep edges of H_M. Fix a deep edge $t \to t'$ of H_M. An edge of a fixed chain that projects to it has its tail at step t and its head at step t', and it lies in the tree of its tail, which contains its head. Trees are disjoint, so the fixed chains of distinct trees that use $t \to t'$ have distinct heads at step t', at most m of them. Counting the pairs (tall tree, deep edge on its fixed chain) gives $\tau|Q_0| \le m \cdot \#\{\text{deep edges of } H_M\} \le mN_{\mathcal{R}}$ by (1).

Paths. Let $\pi = (u_0 \to \cdots \to u_n)$ be $(\Gamma_{\mathrm{F}}, Q_0)$-free, with u_0 an unrevealed output node of a position $\ell \notin X$ and u_n input-marked. First, π meets no tall tree. Indeed, a vertex u_i of a tall tree is not its root, which is revealed, while u_0 is unrevealed and so are the later vertices; so u_i is a copy, π continues along its out-edge inside the tree, and by the same argument through copies of that tree to its end; but u_n is input-marked, hence not a copy. Second, π is Γ_{F}-free, so it contains at most s deep formations, say at positions $i_1 < \cdots < i_{s'}$; put $i_0 = 0$ and $i_{s'+1} = n$. Consider the piece of π from u_{i_a} to $u_{i_{a+1}}$ and a deep edge $u_i \to u_{i+1}$ of it whose tail is not a deep formation; only the edge leaving u_{i_a} can fail this condition. A formation with a deep out-edge is deep, so u_i is a copy, and the edge lies on the chain of u_i, in a tree that is not tall. From there π follows that chain through copies up to the root, which is a deep formation because its tree contains a deep edge, hence is $u_{i_{a+1}}$; or π ends before. So all deep edges of the piece, except possibly the one leaving u_{i_a}, lie on one chain of a tree that is not tall, and there are fewer than τ of them. Each piece thus has at most τ deep edges, and π has at most $h \le (s+1)\tau$. Removing them leaves at most $h+1$ runs; the vertices of a run after its first are not revealed, so a run projects to a Γ_{F}-free path of H_M without deep edges and has at most $D-1$ edges. Hence $n \le h + (h+1)(D-1) \le r_0 - 1$.

Expansion. Apply Lemma 3.3(a) with $\Gamma = \Gamma_{\mathrm{F}}$ and $Q = Q_0$. For $\ell \notin X$ the set $I(\ell)$ lies in $B^{r_0-1}(\ell) \subseteq B^{r_0}(\ell)$ by the bound on paths, and $\operatorname{rank} R \le \beta|\Gamma_{\mathrm{F}}| + |Q_0| + |X| \le \varepsilon L + mN_{\mathcal{R}}/\tau + |X|$; apply (1).

Theorem HC charges every deep hop on a long chain of copies to a checkpoint on that chain: storage, transport and multicast through deep cells cost rank $O(N_{\mathcal{R}}/\tau)$ on every schedule, and only deep formations count. Theorem HC holds for field DAGs as well, with the index code of Theorem HC′(b) in place of the decomposition and the same proof (`MergeDepth.hop_charging_pruning_HC`). The direct proof also applies to field DAGs, with Lemma 3.3(b) in place of (a); the revealed roots are then fields of any kind, a state or an output field among them, and cost up to β bits each.

3.4 Hop charging along lineages

Theorem HC counts every deep formation, so it does not cover a deep cell that accumulates fresh data, $\sigma \leftarrow \sigma + x$, at each visit: every such update is a deep formation. Such an update keeps a single lineage, and lineages can be checkpointed.

Definition 3.7 (Lineages and merge depth)**.** Let G be a refinement and $\Gamma_{\mathrm{F}} \subseteq [0, T)$. A *lineage assignment* for Γ_{F} chooses, for some nodes u, an out-edge e_u of u in G, the *lineage edge* of u, such that every Γ_{F}-free path whose first edge leaves u and differs from e_u contains no deep edge. The nodes with a lineage edge are *continuations*; all other nodes, including those without out-edges, are *merge nodes*. Following lineage edges from a node leads, as steps decrease, to a merge node,

its *lineage root*; the nodes with lineage root p form the *lineage tree* of p, a tree with root p whose edges are the lineage edges of its continuations. Lineage trees partition $V(G)$. A node u_i of a path $\pi = (u_0 \to \cdots \to u_n)$ of G is a *deep merge* of π if it is a merge node and

(α) $i < n$ and the edge $u_i \to u_{i+1}$ is deep, or

(β) for some $j < i$ the edge $u_j \to u_{j+1}$ is deep, and for $j \le j' < i$ the node $u_{j'}$ is a continuation and $u_{j'} \to u_{j'+1}$ is its lineage edge.

The *merge depth* at an output position ℓ is the largest number of deep merges of a Γ_{F}-free path of G from an output node of ℓ. The refinement has *merge depth at most s* if its merge depth is at most s at every output position.

Lineage edges are edges of G, never input marks. A merge node with out-edges typically merges two deep lineages, or deep content with data derived from deep cells. Whether a node is a deep merge depends on the path, which must leave it along a deep edge or arrive through a deep lineage edge followed only by lineage edges. Counting every merge node whose lineage tree contains a deep edge would not satisfy Lemma 3.12 below: such a node need not be a deep formation, since its transport tree may contain no deep edge, and a run of cold edges can pass through many such nodes without using their lineage trees. In Lean the lineage edges form a forest (`MergeDepth.Forest`), the defining condition of a lineage assignment is a hypothesis of the results that use it, and `MergeDepth.dm` counts the deep merges of a path.

Remark 3.8 (Examples)**.** Assigning to every copy its out-edge, and to no other node a lineage edge, is a lineage assignment for every Γ_{F}. A deep-cell update that combines the previous content of the cell with data whose Γ_{F}-free paths reach no deep edge, such as fresh input bits, can be a continuation, with the edge to the previous content as its lineage edge. If every tail of a deep edge is a continuation with that edge as lineage edge, and no lineage root of a tree containing a deep edge reaches a deep edge by a Γ_{F}-free path, then (α) never applies, nothing after a deep merge of type (β) is deep, and every path has at most one deep merge. This covers deep cells that only store, move, multicast or accumulate data not derived from deep cells, and pure data movement. Relays with boundedly many mixing steps, or butterflies of bounded depth, have bounded merge depth. A side edge into a step of Γ_{F} may be deep, as the one-edge path into a revealed step is not Γ_{F}-free; the paths that matter below never use one.

Lemma 3.9 (Covering)**.** Lean: `MergeDepth.covering` [`MergeDepth/Covering.lean`]

Let $\mathcal{T}$ be a finite forest whose edges are directed towards the roots, some of them marked, and let $\tau \ge 1$. There is a set Q of heads of marked edges, with $\tau|Q|$ at most the number of marked edges, such that every walk $x_0 \to x_1 \to \cdots \to x_n$ along edges of $\mathcal{T}$ with $x_1, \dots, x_n \notin Q$ crosses fewer than τ marked edges.

Outline of the formal proof. The *marked height* of a node is the number of marked edges on its walk to its root; along an edge it drops by one if the edge is marked and stays the same otherwise. Sort the heads of the marked edges into τ classes by their marked heights modulo τ, and let Q be a smallest class. There are at most as many heads as marked edges, so $\tau|Q|$ is at most the number of marked edges. A walk that crosses τ marked edges meets, after its first node, heads of marked edges at τ consecutive marked heights, one in each class, so one of them lies in Q. □

Lemma 3.10 (Few deep merges, short paths)**.** Lean: `MergeDepth.short_paths` [`MergeDepth/HopFields.lean`]

Let G be a refinement of width m, let Γ_{F} be a cold genie at (V,ε) with depth D, fix a lineage assignment for Γ_{F}, and let $\tau \geq 1$. There is a set Q of at most $mN_{\mathcal{R}}/\tau$ nodes of G, the checkpoints*, such that every (Γ_{F}, Q)-free path of G with s' deep merges has at most $(s'+1)\tau - 1$ deep edges and fewer than $(s'+1)\tau D$ edges.*

Outline of the formal proof. Counting. The projection of a deep lineage edge is a deep edge of H_M, so its tail lies at a step at which a deep hop ends. There are at most $N_{\mathcal{R}}$ such steps and at most m nodes at each, and every node has at most one lineage edge. Mark the lineage edges whose tails lie at these steps: every deep lineage edge is marked, and at most $mN_{\mathcal{R}}$ edges are. Let Q be the set that Lemma 3.9 gives for the forest of lineage edges; then $|Q| \leq mN_{\mathcal{R}}/\tau$.

Deep edges. Let $\pi = (u_0 \to \cdots \to u_n)$ be (Γ_{F}, Q)-free. If π leaves a continuation u_i by an edge other than its lineage edge, then the rest $u_i \to \cdots \to u_n$ contains no deep edge, since it is Γ_{F}-free. Walk along π and keep the current *stretch*: the longest final segment of the part walked so far that follows lineage edges. Its vertices after the first are vertices of π after u_0, hence not in Q, so by Lemma 3.9 it contains fewer than τ marked edges, hence fewer than τ deep edges. By induction on the length of the rest of the path, the deep edges of the rest together with those of the current stretch number at most $\tau - 1$ plus τ times the deep merges of the rest, where a merge node reached through the current stretch counts as of type (β) if the stretch contains a deep edge. Indeed, at a merge node u_i the stretch ends: if the edge $u_i \to u_{i+1}$ is deep or the stretch contains a deep edge, then u_i is a deep merge, and its share τ pays for the fewer than τ deep edges of the stretch and the edge leaving u_i; otherwise the stretch contains no deep edge. At a continuation that π leaves by its lineage edge the stretch grows by that edge, and at a continuation that π leaves by another edge no deep edge follows. Starting with the stretch (u_0), the path has at most $h \leq \tau - 1 + \tau s' = (s'+1)\tau - 1$ deep edges.

Length. Removing the deep edges leaves at most $h+1$ runs. The vertices of a run after its first are not revealed, so a run projects to a Γ_{F}-free path of H_M without deep edges and has at most $D-1$ edges by the definition of the depth. Hence $n \leq h + (h+1)(D-1) = (h+1)D - 1 < (s'+1)\tau D$. □

Theorem 3.11 (Theorem HC′: hop charging along lineages)**.** Lean: `MergeDepth.hop_charging_pruning`, `MergeDepth.hop_charging_pruning_bits`, `MergeDepth.short_paths_pruning` [`MergeDepth/FieldMachine.lean`]

Let G be either the coordinate DAG of an $\mathbb{F}$-linear data path of M at length L of width m, or the field DAG of a pruning, with $m = k+2$. Let Γ_{F} be a cold genie at (V,ε) with depth D, fix a lineage assignment for Γ_{F}, let $\tau \geq 1$ and $s \geq 0$ be integers, put $r = (s+1)\tau D$, and let X be the set of output positions at which G has merge depth greater than s. There is a set Q of at most $mN_{\mathcal{R}}/\tau$ nodes of G such that every (Γ_{F}, Q)-free path of G from an output node of a position $\ell \notin X$ has fewer than r edges. Suppose that M transposes all $d \times d$ matrices.

(a) If G is the coordinate DAG, then $P_{\mathrm{tpos}} = R + S$ with $\operatorname{rank} R \leq \varepsilon L + mkC(L+1)/\tau + |X|$, where row ℓ of S is zero for $\ell \in X$ and supported in $B^r(\ell)$ otherwise.

(b) If G is a field DAG, there is an index code with at most $\varepsilon L + \beta(k+2)kC(L+1)/\tau + |X|$ bits for a system $(I(\ell))_\ell$ with $I(\ell) \subseteq B^r(\ell)$ for every ℓ.

Outline of the formal proof. Let Q be the set of Lemma 3.10. A (Γ_{F}, Q)-free path from an output node of $\ell \notin X$ is Γ_{F}-free, so it has at most s deep merges and, by Lemma 3.10, at most $r-1$ edges. Hence the sets $I(\ell)$ of Lemma 3.3, applied with $\Gamma = \Gamma_{\mathrm{F}}$, Q and X, lie in $B^{r-1}(\ell) \subseteq B^r(\ell)$ for $\ell \notin X$, and they are empty for $\ell \in X$ in case (b). The rank in (a) is at most $\beta|\Gamma_{\mathrm{F}}| + |Q| + |X| \leq \varepsilon L + mN_{\mathcal{R}}/\tau + |X|$, and the code in (b) has at most $\beta|\Gamma_{\mathrm{F}}| + \beta|Q| + |X| \leq \varepsilon L + \beta(k+2)N_{\mathcal{R}}/\tau + |X|$ bits; apply (1).

Formally, (a) is stated for a linear network of width m whose edges are dependency steps of the work-head walks, whose steps in Γ_{F} take at most 2^{β} values on binary inputs and whose output functionals transpose on binary inputs, and for a cold genie in the sense of `MergeDepth.IsColdGenie`. The network `MergeDepth.LinDataPath.net` of a data path of an oblivious transposer has these properties (`MergeDepth.LinDataPath.net_depStep`, `MergeDepth.LinDataPath.val_card_le`, `MergeDepth.LinDataPath.trans_of_transposer`). `MergeDepth.hop_charging_steps` proves (a) with the rank bound $\beta|\Gamma_{\mathrm{F}}| + q + |X|$, where τq is at most m times the number of deep edges. The coordinate DAG is the network `MergeDepth.coordNet`, whose values are the coordinates of the step values (`MergeDepth.coord_val`) and whose paths and balls project to those of H_M (`MergeDepth.coord_paths`, `MergeDepth.coord_reach_subset`); `MergeDepth.hop_charging` combines `MergeDepth.paths_short` with the decomposition `MergeDepth.rigidity_finrank` of Lemma 3.3(a). `MergeDepth.hop_charging_machine` states (a) with the constants above, and `MergeDepth.hop_charging_machine_real` for a real constant C. For (b), the machine need only be oblivious before a time T by which it has appended L bits (`MergeDepth.ObliviousTransposer`), which holds for the frozen notion of obliviousness, and its entries may have any width. A pruning is `MergeDepth.Pruning`, and its field DAG `MergeDepth.Pruning.toFunDag` is a function DAG that refines the dependency DAG. Keeping everything is a pruning (`MergeDepth.Pruning.full`), by the local recursion of the fourth paper at the level of fields (`MergeDepth.stepValue_eq_of_preds`). `MergeDepth.hop_charging_pruning` and `MergeDepth.hop_charging_pruning_bits` are (b), through `MergeDepth.hop_charging_fields` for refinements, and `MergeDepth.short_paths_pruning` is the path bound.

The checkpoints can be chosen among the heads of deep lineage edges, by marking only the deep lineage edges (`MergeDepth.hop_charging_fields_heads`). In a field DAG these heads are symbol fields, so in (b) a checkpoint costs $\lceil \log_2 g \rceil$ bits instead of β, where g is the number of work-tape symbols, and the term charged to the checkpoints improves to $\lceil \log_2 g \rceil (k+2)kC(L+1)/\tau$ (`MergeDepth.hop_charging_pruning_sym`). □

Theorem HC′ reduces the depth of the code, not of the schedule: every output outside X has bounded depth in G once Γ_{F} and Q are revealed, as for the coordinate genies of [3, Remark 7.6(c)], while paths of H_M, which need not follow the dependencies of the code, may stay long. In (a) the bound is on the rank and does not depend on $\mathbb{F}$; counted in bits, a revealed coordinate would cost up to $\min(\lceil \log_2 |\mathbb{F}| \rceil, \beta)$.

Lemma 3.12 (Merge depth and formation depth)**.** Lean: `MergeDepth.dm_le_countP_deepFormation` [`MergeDepth/Formation.lean`]

Let G be a refinement and $\Gamma_{\mathrm{F}} \subseteq [0, T)$, and fix a lineage assignment for Γ_{F} in which every copy is a continuation. Then every path of G contains at least as many deep formations as it has deep merges. In particular, the merge depth at every output position is at most the formation depth there.

Outline of the formal proof. A copy has only one out-edge, which is therefore its lineage edge, and a merge node is not a copy, hence a formation. Walk along a path with a credit for the current stretch of lineage edges. When the path crosses a deep lineage edge out of a formation, that formation is a deep formation of the path and credits the stretch. When it crosses one out of a copy, the next formation on the path lies on the chain of that copy, so it is the root of a transport tree containing a deep edge, a deep formation; until it is reached, the deep copy edge lies in the transport tree of the current node. A deep merge of type (α) is a formation with a deep out-edge, hence a deep formation itself; one of type (β) ends a stretch that contains a deep lineage edge, and is paid by the credit of that stretch or, if the deep edge left a copy and no formation has followed, is itself the root of the transport tree that contains it. A side edge ends the stretch, and stretches are

disjoint, so no deep formation pays twice. The argument uses only the lineage edges, not the defining condition of a lineage assignment, and `MergeDepth.dm_le_countP_deepFormation` is stated in that generality. □

So Theorem HC′ applies, with the same s, to every code to which Theorem HC applies, which is how Theorem HC is proved, and it covers accumulators of fresh data as well. It is the form used below.

3.5 Exclusion of bounded merge depth

Corollary 3.13 (Corollary HC: the linear crux excludes bounded merge depth)**.** Lean: `MergeDepth.cor_HC`, `MergeDepth.cor_HC_formation`, `MergeDepth.cor_HC_uniform`, `MergeDepth.cor_HC_formation_uniform`, `MergeDepth.LinDataPath`, `MergeDepth.SBall`, `MergeDepth.LinCrux`, `MergeDepth.MergeShallow`, `MergeDepth.FormationShallow` [`MergeDepth/LinearCrux.lean`]

Let $k, m, V \geq 1$, $\beta \geq 2$ and $s \geq 0$ be integers, $C \geq 1$ real, $\mathbb{F}$ a field, $0 < \varepsilon < \rho$ and $0 \leq \eta < 7(\rho-\varepsilon)/8$. Put

$$\tau = \left\lceil \frac{8mkC}{\rho-\varepsilon} \right\rceil, \qquad D = D_{\mathrm{F}}(k, 2V-1, \varepsilon/(4\beta C)) \cdot \lceil 4\beta C/\varepsilon \rceil, \qquad r = (s+1)\tau D,$$

and suppose that $\mathrm{LinCrux}_{\mathbb{F}}(\mathcal{S}^{\mathrm{ball}}_{k,C,\beta,r}, \rho)$ holds [3, Definition 7.4]. Then there is d_0 such that for no $d \geq d_0$ and no machine M with k work tapes and value width at most β do the following hold together at $L = d^2 b(d)$:

(i) M is oblivious at length L with halting time at most $C(L+1)$ and transposes all $d \times d$ matrices;

(ii) M has an $\mathbb{F}$-linear data path at length L with a representation of width at most m whose coordinate DAG, for some cold genie at (V, ε) with depth D and some lineage assignment for it, has merge depth at most s at all but at most ηL output positions.

The same holds with merge depth replaced by formation depth.

Outline of the formal proof. By the choice of τ, $mkC/\tau \leq (\rho-\varepsilon)/8$, so $\eta' = \frac{1}{2}\left(\frac{7}{8}(\rho-\varepsilon) - \eta\right)$ is positive and $\varepsilon + \eta + mkC/\tau \leq \rho - 2\eta'$. Let d_0 be at least the threshold that the linear crux gives for η' and so large that $mkC/\tau < \eta' L$ for $d \geq d_0$. Suppose that (i) and (ii) hold for a machine M at some $d \geq d_0$, with a representation of width $m' \leq m$, a cold genie Γ_{F} and a lineage assignment, and let X be the set of output positions at which the merge depth exceeds s, so $|X| \leq \eta L$. Theorem 3.11(a), which holds for a real C, with m' in place of m, gives $P_{\mathrm{tpos}} = R + S$ with row ℓ of S supported in $B^r(\ell)$ and

$$\operatorname{rank} R \leq \varepsilon L + \frac{m'kC(L+1)}{\tau} + \eta L \leq (\rho - 2\eta')L + \frac{mkC}{\tau} < (\rho - \eta')L.$$

The ball system $(B^r(\ell))_\ell$ of M belongs to $\mathcal{S}^{\mathrm{ball}}_{k,C,\beta,r}$, since M is oblivious with k work tapes, value width at most β and halting time at most $C(L+1)$, and appends exactly L bits. Its minrank over $\mathbb{F}$ is at most $\operatorname{rank} R < (\rho - \eta')L$, contrary to the linear crux. For formation depth, take the lineage assignment that gives every copy its out-edge (Remark 3.8); by Lemma 3.12 its merge depth is at most the formation depth.

Formally, a data path is `MergeDepth.LinDataPath`, the eighth paper's Definition 7.1 for a machine that is oblivious along a trace; the values of its network, `MergeDepth.LinDataPath.net`, are the encoded step values (`MergeDepth.LinDataPath.val_eq_enc`). The class is `MergeDepth.SBall`, the crux is `MergeDepth.LinCrux`, and condition (ii) is `MergeDepth.MergeShallow`, or `MergeDepth.FormationShallow` for formation depth. The corollary is `MergeDepth.cor_HC_uniform`, or

`MergeDepth.cor_HC_formation_uniform` for formation depth, proved through the minrank form `MergeDepth.LinDataPath.minrank_ballSystem_le` of Theorem 3.11(a) at one size. The weaker forms `MergeDepth.cor_HC` and `MergeDepth.cor_HC_formation` have a threshold for each machine and $\eta < (\rho - \varepsilon)/8$. The hypotheses $V \geq 1$, $\beta \geq 2$, $\varepsilon > 0$ and $\eta \geq 0$ are not used, since they matter only for the existence of a cold genie, which (ii) assumes, and $C > 0$ suffices. □

Given the linear crux at the radius r, an oblivious linear-time transposer with an $\mathbb{F}$-linear data path of width at most m must therefore, on every schedule, merge deep lineages more than s times along some path from the output nodes of more than ηL output positions, whatever its cold genie at (V, ε) and its lineage assignment.

Remark 3.14 (Parameters and special cases)**.** (a) The depth in Corollary F depends only on k, V, β, C and ε; the radius $r = (s+1)\tau D$ here depends also on s, and on m and $\rho - \varepsilon$ through τ. Excluding machines of every width, or of every merge depth, needs the linear crux along a sequence of radii. (b) If every work-tape cell is entered at most V times, there are no deep edges and no deep merges, $Q = \emptyset$, and row ℓ of S is supported in the cone of the whole-step genie Γ_{F}, whose full depth is less than D. The argument then needs only a minrank bound for the cone systems of the class $\mathcal{S}_{k,C,\beta,D}$ [4, Definition 4.8]: it is Corollary F restricted to linear codes. As stated, with balls of radius $r \geq D$, Corollary HC contains only a ball form of Corollary F, whose hypothesis is stronger, as ball systems dominate cone systems [3, Remark 7.6(a)]. (c) No depth reduction of the deep part is used, so the corollary applies to the deep-nesting pairs as to every other schedule.

For all codes the crux has to be applied to relations that are not cone relations of whole-step genies. The fifth paper's structural theorem extends to them, because its proof uses the genie only through short paths.

Lemma 3.15 (Relations inside balls)**.** Lean: `MergeDepth.ball_paired_packing_trace`, `MergeDepth.ball_self_demands_trace`, `MergeDepth.ball_rate_trace` [`MergeDepth/HCStarQ91.lean`]
Let $D' \geq 0$ be an integer and $K = \sum_{i=0}^{D'} (k+1)^i$. Suppose that $d \geq 2$ and that M appends exactly L output bits, and let N be a recovery relation on $[0, L)$ with $N(q) \subseteq B^{D'}(\mathrm{tpos}(q))$ for every q.

(a) The conclusions of [10, Theorem 3.3, Corollary 3.7 and Corollary 3.8] hold for N, with D' in place of the full depth of the genie and with constants at least as good. In particular, at most $(2CK(k+2)(k+3)+3)L/\log_2 d$ positions p satisfy $p \in N(p)$.

(b) If Question 9.1 of [10] has an affirmative answer for every fixed degree bound, then for every $\delta > 0$ there is d_1, depending only on k, D', C and δ, such that for $d \geq d_1$ every storage code $\mathcal{C}$ for N [10, Definition 2.1] over a finite alphabet $\mathcal{A}$ with $|\mathcal{A}| \geq 2$ satisfies $\log_{|\mathcal{A}|} |\mathcal{C}| \leq (\frac{1}{2} + \delta)L$.

Outline of the formal proof. (a) We run the argument of Section 4.4 at scale $w = 1$, where the block DAG is the dependency DAG H_M and the relaxation has one head on each of the $k+2$ tapes, following the corresponding head of M, and one register. That argument is the fifth paper's proof of its Theorem 3.3 and Corollaries 3.7 and 3.8, which uses the relation only through the realization of matchings of its incidence graph (the fifth paper's Lemma 3.5, our Lemma 4.9) and through a bound on $|N(q)|$. For a matching of the incidence graph of N, choose for every edge $L_p R_q$ a path of H_M with at most D' edges from $t_{\mathrm{tpos}(q)}$ to a step that reads p; it exists because $p \in B^{D'}(\mathrm{tpos}(q))$. In-degrees in H_M are at most $k+1$, so a step lies on the chosen paths of at most K output steps, each of which appends one position, and the realization along the chosen paths, as in the proof of Lemma 4.9, has occupancy at most K. Likewise $|N(q)| \leq K$, because at most K steps are reachable from a step by paths of at most D' edges, out-degrees being at most $k+1$, and each step reads at

most one position. So Lemma 4.10 and Proposition 4.11 hold for N with $K_1 = k+2$ heads and occupancy K in place of wB. They give the conclusions of Theorem 3.3 and Corollaries 3.7 and 3.8 with $A_s = 2sCK(k+2)(k+3)+3$ in place of the fifth paper's $4sC(B+1)(k+2)(k+6)+3$, and with K in place of B; the last sentence of (a) is Proposition 4.11(b).

(b) We follow the proof of [10, Proposition 9.2], with (a) in place of the statements for cone relations. Put $\delta_0 = \min(\delta, \frac{1}{4})$ and $\vartheta = \lceil 4K/\delta_0 \rceil$, and let H be the set of coordinates p whose left vertex L_p has degree greater than ϑ in the incidence graph [10, Definition 2.3]. The left degrees sum to $\sum_q |N(q)| \le KL$, so $|H| \le \delta_0 L/4$. The relation N_Q induced on $Q = [0,L) \setminus H$ has $|Q| \ge L/2$, an incidence graph of maximum degree at most $\Delta = \max(K, \vartheta)$, and, by (a), for every fixed r at most $O(L/\log d)$ coordinates whose two ports are at distance at most r in that graph. The uniformity argument of the fifth paper's Proposition 9.2 then gives d_1, depending only on k, D', C and δ, such that for $d \ge d_1$ every storage code $\mathcal{D}$ for N_Q over a finite alphabet $\mathcal{A}$ with $|\mathcal{A}| \ge 2$ has $\log_{|\mathcal{A}|}|\mathcal{D}| \le (\frac{1}{2} + \frac{\delta_0}{2})|Q|$: otherwise there are sizes $d_j \to \infty$ with relations of this kind and codes that violate the bound, these relations satisfy the hypotheses of Question 9.1 for the degree bound Δ, and the codes contradict its affirmative answer. Fixing the coordinates in H turns $\mathcal{C}$ into storage codes for N_Q [10, Lemma 2.2], so $\log_{|\mathcal{A}|}|\mathcal{C}| \le |H| + (\frac{1}{2} + \frac{\delta_0}{2})(L - |H|) \le (\frac{1}{2} + \frac{\delta_0}{2})L + \frac{|H|}{2} \le (\frac{1}{2} + \delta)L$.

Formally, (a) is proved through Section 4.4 at scale $w = 1$: the head walks of the machine are the reference walks, and the steps that read input positions and append output bits are those of its trace (`MergeDepth.trace_ball_data`). `MergeDepth.realizes_ball` realizes every matching of the incidence graph of N with occupancy K, and `MergeDepth.ball_paired_packing`, `MergeDepth.ball_self_demands` and `MergeDepth.ball_trimming` are the analogues of the fifth paper's Theorem 3.3 and Corollaries 3.7 and 3.8; `MergeDepth.ball_paired_packing_trace` and `MergeDepth.ball_self_demands_trace` state the first two for a trace. Part (b) is `MergeDepth.ball_rate_trace`, from the uniformity argument, proved once for families of relations (`MergeDepth.uniform_rate`), and paid conditioning (`MergeDepth.logb_card_le_of_conditioning`); `MergeDepth.ball_rate_binary` is the binary form under `MergeDepth.Q91Binary`. The formal statements are more general: both parts hold for every trace that appends at least L bits before a time $T \le C(L+1)$, for every real $C \ge 0$ and every entry width, so neither obliviousness nor the exact output length nor correctness is used. □

Corollary 3.16 (Corollary HC*: Question 9.1 excludes bounded merge depth for all codes)**.** `Lean`: `MergeDepth.cor_HCstar_uniform`, `MergeDepth.cor_HCstar_uniform_frozen`, `MergeDepth.cor_HCstar_uniform_empty` [`MergeDepth/HCStarQ91.lean`]

Suppose that Question 9.1 of [10] has an affirmative answer for every fixed degree bound; the binary alphabet suffices. Let $k, V \ge 1$ and $s \ge 0$ be integers, $C \ge 1$ real, and let $\varepsilon > 0$ and $\eta \ge 0$ satisfy $\varepsilon + \eta < \frac{1}{2}$. Then for every machine M with k work tapes there is d_0 such that for no $d \ge d_0$ do the following hold together at $L = d^2 b(d)$:

(i) M is oblivious at length L with halting time at most $C(L+1)$ and transposes all $d \times d$ matrices;

(ii) for some pruning, the field DAG of M at length L has, for some cold genie at (V, ε), taken with $\beta = \beta(M)$, and some lineage assignment for it, merge depth at most s at all but at most ηL output positions.

In particular, for all $V \ge 1$ and $s \ge 0$ there is d_0 such that at no $d \ge d_0$ at which (i) holds does a pruning with a lineage assignment for the empty genie give the field DAG merge depth at most s with respect to the empty genie.

Outline of the formal proof. Put $\beta = \beta(M)$, $\gamma = \frac{1}{2} - \varepsilon - \eta > 0$, $C' = \lceil C \rceil$ and $\tau = \lceil 4\beta(k+2)kC'/\gamma \rceil$, let D be the depth of the cold genies at (V, ε), and put $r = (s+1)\tau D$; these depend only on M, V,

s, C, ε and η. Let d_1 be given by Lemma 3.15(b) for $D' = r$, the constant C' and $\delta = \gamma/4$, and let $d_0 = \max(d_1, 2)$. Suppose that (i) and (ii) hold at some $d \geq d_0$; then $T \leq C'(L+1)$. Let X be the set of output positions at which the merge depth exceeds s. By Theorem 3.11(b), with C' in place of C, there is an index code with at most

$$\varepsilon L + \frac{\beta(k+2)kC'(L+1)}{\tau} + \eta L \leq (\varepsilon+\eta)L + \frac{\gamma(L+1)}{4} \leq \Big(\frac{1}{2} - \frac{\gamma}{2}\Big)L$$

bits for a system $(I(\ell))_\ell$ with $I(\ell) \subseteq B^r(\ell)$. Some fibre of it has at least $2^{(1/2+\gamma/2)L}$ elements, and it is a binary storage code for the relation $N(q) = I(\mathrm{tpos}(q))$ [4, Remark 4.9(b)], with $N(q) \subseteq B^r(\mathrm{tpos}(q))$. By Lemma 3.15(b) the fibre has at most $2^{(1/2+\gamma/4)L}$ elements, a contradiction. For the last sentence, (i) holds with C' in place of C; a lineage assignment for the empty genie is one for every Γ_{F}, and Γ_{F}-free paths are paths; so the hypothesis gives (ii), with $\eta = 0$ and C' in place of C, for a cold genie at $(V, \frac{1}{4})$, which exists by Lemma 3.1.

Formally, `MergeDepth.cor_HCstar_step` is the corollary at one size: for a refinement of width m with the hypotheses of `MergeDepth.hop_charging_fields`, $T \leq C(L+1)$, at most ηL exceptional positions and $4\beta mkC \leq \gamma\tau$, and with the bound $2^{(1/2+\gamma/4)L}$ on binary storage codes for relations inside the balls of radius $(s+1)\tau D$ as a hypothesis, the hypotheses are contradictory. The large fibre is `MergeDepth.exists_large_fibre_card`, and `MergeDepth.isStorageCode_fibre_of_indexCode` shows that it is a storage code for $q \mapsto I(\mathrm{tpos}(q))$. For the field DAG of a pruning, `MergeDepth.cor_HCstar_pruning` and, from the frozen notion of obliviousness, `MergeDepth.cor_HCstar_frozen` discharge the hypotheses on the refinement, and `MergeDepth.cor_HCstar_pruning_empty` is the last sentence at one size. Under [Q9.1], `MergeDepth.hballs_of_obliviousTransposer` is Lemma 3.15(b) at that radius with $\delta = \gamma/4$, and `MergeDepth.cor_HCstar_uniform` is the corollary at all sizes for an integer C and every $\tau \geq 1$ with $4\beta(k+2)kC \leq \gamma\tau$, with d_0 uniform over the machines with k work tapes and value width at most β and over the entry width. `MergeDepth.cor_HCstar_uniform_frozen` is the corollary as stated, for every $\beta \geq \beta(M)$, every entry width and cold genies of any fixed depth; it uses only $\varepsilon + \eta < \frac{1}{2}$, not $V \geq 1$ or $\eta \geq 0$. `MergeDepth.cor_HCstar_uniform_empty` is the last sentence for an integer C, which suffices because (i) with C implies (i) with C'. With checkpoints among the symbol fields (outline of Theorem 3.11), $4\lceil \log_2 g \rceil (k+2)kC \leq \gamma\tau$ suffices (`MergeDepth.cor_HCstar_pruning_sym`), which shrinks τ and the radius r. □

Corollary HC* is the deep-nesting companion of Corollary F′ [3, Corollary 6.3], which it contains, since bounded entry leaves no deep merges. Given Question 9.1, an oblivious linear-time transposer must therefore, whatever its pruning, cold genie at (V, ε) and lineage assignment, merge deep lineages more than s times along some path from the output nodes of more than ηL output positions, for all fixed s and $\eta < \frac{1}{2} - \varepsilon$ and all large d. The proof uses Question 9.1 only through Lemma 3.15(b). An all-code crux for ball systems would serve as well, since the index code of Theorem 3.11(b) is also one for the ball system: $\mathrm{Crux}(\mathcal{S}^{\mathrm{ball}}_{k,C,\beta,r}, \rho)$, in the sense of [4, Definition 4.8] for the class of [3, Definition 7.4], with $\rho > \varepsilon + \eta$ and $\beta(k+2)kC/\tau < (\rho - \varepsilon - \eta)/4$, excludes the same machines. The crux for the cone class $\mathcal{S}_{k,C,\beta,D}$ does not apply directly, because the systems of Theorem 3.11(b) are not cone systems of whole-step genies of bounded full depth.

Update (November 2026). Question 9.1 has a negative answer, also for binary codes [8, Theorem A and Corollary A′], so Lemma 3.15(b) and Corollary HC* are vacuous as stated, and so are the Lean theorems that state them under [Q9.1]; Lemma 3.15(a) is unconditional and not affected. The proof above needs only the conclusion of Lemma 3.15(b), for $D' = r$ and $\delta = \gamma/4$: a half-rate bound for storage codes on relations inside balls, a native statement that the thirteenth paper does not decide. With that conclusion, or with $\mathrm{Crux}(\mathcal{S}^{\mathrm{ball}}_{k,C,\beta,r}, \rho)$ as above, in place of Question 9.1, the

corollary holds; both hypotheses are open. At one size this is formal: `MergeDepth.cor_HCstar_step` and the other one-size forms named in the outline take the bound for relations inside the balls at that size as a hypothesis, not [Q9.1], and are not affected.

3.6 Routing through deep hops, and the number of deep cells

Proposition 3.17 (Routing through deep hops is hop-limited)**.** Lean: `MergeDepth.hop_routing_trace` [`MergeDepth/HopRoutingTrace.lean`]

Suppose that $d \geq 2$ and that M appends exactly L output bits. Let Γ_{F} be a cold genie at (V, ε) with depth D, let $D'' \geq D$ be an integer, and put

$$K'' = \sum_{i=0}^{D''} (k+1)^i, \qquad A = 2CK''(k+2)(k+3)+3, \qquad h = \left\lceil \frac{D''+2-D}{D} \right\rceil.$$

(a) For all but at most $AL/\log_2 d$ output positions ℓ, every Γ_{F}-free path of H_M from t_ℓ to a step that reads position $\mathrm{tpos}(\ell)$ contains at least h deep edges.

(b) Let $c_0 \geq 0$, and let nonnegative weights $f_\ell(\pi)$ be given on the Γ_{F}-free paths π of H_M from t_ℓ to steps that read $\mathrm{tpos}(\ell)$, such that $\sum_\pi f_\ell(\pi) \leq 1$ for every ℓ and $\sum_\ell \sum_{\pi \ni e} f_\ell(\pi) \leq c_0$ for every deep edge e. Then $\sum_\ell \sum_\pi f_\ell(\pi) \leq AL/\log_2 d + c_0 N_{\mathcal{R}}/h \leq AL/\log_2 d + c_0 kC(L+1)/h$.

Outline of the formal proof. (a) Apply Lemma 3.15(a) to the relation $N(q) = B^{D''}(\mathrm{tpos}(q))$: at most $AL/\log_2 d$ positions p satisfy $p \in B^{D''}(\mathrm{tpos}(p))$. For every other output position ℓ, the position $p = \mathrm{tpos}(\ell)$ satisfies $\mathrm{tpos}(p) = \ell$ and $p \notin B^{D''}(\ell)$, so every path of H_M from t_ℓ to a step that reads p has more than D'' edges. A Γ_{F}-free path with h' deep edges splits at them into $h'+1$ Γ_{F}-free paths without deep edges, each with at most $D-1$ edges by the definition of the depth, so it has at most $(h'+1)D - 1$ edges. Hence $(h'+1)D - 1 \geq D''+1$, that is, $h' \geq (D''+2-D)/D$, and $h' \geq h$.

(b) The exceptional positions of (a) contribute at most $AL/\log_2 d$. For the others every path carries at least h deep edges, so h times their total weight is at most $\sum_e \sum_\ell \sum_{\pi \ni e} f_\ell(\pi)$, summed over the deep edges, which is at most $c_0 N_{\mathcal{R}}$ by (1).

Formally, `MergeDepth.length_le_of_coldDep` is the splitting at deep edges, for a cold genie in the sense of `MergeDepth.IsColdGenie`, and `MergeDepth.hop_routing_a` is (a) for each position ℓ with $\mathrm{tpos}(\ell) \notin B^{D''}(\ell)$, the ball being `MergeDepth.stepBall`. Part (b) is `MergeDepth.hop_routing_b`, for weights on any finite sets of such paths and the positions ℓ with $\mathrm{tpos}(\ell) \in B^{D''}(\ell)$ as the exceptional set; it rests on the double counting `MergeDepth.routing_count` and on `MergeDepth.rCount_le_card_edgesOf`: the edges of a path of H_M are distinct, since steps decrease along it. `MergeDepth.hop_routing_count` adds the count of Lemma 3.15(a): through the involution tpos, the exceptional positions are the self-demands of the relation $q \mapsto B^{D''}(\mathrm{tpos}(q))$ (`MergeDepth.card_exceptional_eq`), which `MergeDepth.ball_self_demands` bounds. `MergeDepth.hop_routing_trace` states the result for every trace that appends at least L bits before a time $T \leq C(L+1)$, for every real $C \geq 0$, with the walks, read steps and append steps supplied by `MergeDepth.trace_ball_data`. □

The proposition is unconditional. Since $A = O((k+1)^{D''})$, the exceptional fraction tends to zero for every fixed D'', and also when D'' grows with d so slowly that $(k+1)^{D''} = o(\log d)$; then h grows too. Read heuristically: after the cold genie, the deep hops are the only long-range carriers, routing through them delivers $o(L)$ once $h \to \infty$, and whatever a transposer delivers beyond the εL revealed bits needs a network code on the deep hops whose gain over routing grows with D''/D.

Proposition 3.18 (The number of deep cells)**.** Lean: `MergeDepth.card_deep_mul_le`, `MergeDepth.card_deepHops_le`, `MergeDepth.two_mul_card_deepHops_le`, `MergeDepth.card_deep_le_of_lt` [`MergeDepth/Capacity.lean`]

We have $|\mathcal{R}| \le kT/(V+1)$ *and* $N_{\mathcal{R}} \le \min\{kT, |\mathcal{R}|(T-1)/2\}$*. In particular,* $|\mathcal{R}|(T-1)/2 < kT$ *only if* $|\mathcal{R}| < 2kT/(T-1)$*, which for* $T > 2k+1$ *means* $|\mathcal{R}| \le 2k$*.*

Outline of the formal proof. Each head makes at most T entries, and each pair of $\mathcal{R}$ takes more than V of them; this gives the first bound, as in [3, Section 2.2]. The bound kT is (1). For $(h,c) \in \mathcal{R}$, the long hops (p,u) of head h at c give pairwise disjoint intervals $(p,u] \subseteq [1, T-1]$, since at most one hop of h at c spans a given time [3, Section 2.2, (P1)]; each has length at least two, so there are at most $(T-1)/2$ of them. Summing over $\mathcal{R}$ gives the second bound. Finally, $2kT/(T-1) = 2k + 2k/(T-1) < 2k+1$ when $T > 2k+1$.

Formally, $\mathcal{R}$ is the eighth paper's `BoundedNesting.heavyPairs` and the deep hops are the long hops outside its `BoundedNesting.lightHops` (`MergeDepth.deepHops`); the first bound is its `BoundedNesting.card_heavyPairs_mul_le`, and the disjointness of the intervals is its hop uniqueness. □

Theorems HC and HC′, Corollaries HC and HC*, Proposition 3.17 and Proposition 3.21 below depend on the deep cells only through $N_{\mathcal{R}}$ and the code; none uses $|\mathcal{R}|$. A scratch of fewer than εL cells, as in [3, Problem 10.5], gives them nothing beyond Proposition 3.18, which improves the hop count only when there are at most $2k$ deep cells.

Remark 3.19 (Heuristic: capacity bounds storage time, not depth)**.** This is a heuristic. The number of deep cells bounds the total length $|\mathcal{R}|T$ of the deep hops, that is, how long the deep part holds values; but cold storage can carry the long-time transport, while the deep part supplies depth through hops that may be short. Demand paths of the form "cold jump, a few short deep hops, cold jump, ..." use deep cell-time of order h each, so capacity allows $|\mathcal{R}|T/O(h)$ of them, the hop count only $O(kT/h)$. We expect no proof to profit from a hypothesis $|\mathcal{R}| \le \delta L$ alone; the results above show only that none of them uses it.

3.7 The merge DAG and code-level depth reduction

After Lemma 3.10, every (Γ_{F}, Q)-free path from an output node alternates stretches with fewer than τ deep lineage edges and deep merges. What remains is the pattern of the merges.

Definition 3.20 (The merge DAG)**.** Lean: `MergeDepth.MEdge`, `MergeDepth.MInit`, `MergeDepth.MCut` [`MergeDepth/MergeDag.lean`]

In the setting of Lemma 3.10, let Q be the set of checkpoints given there. The *merge DAG* $\mathcal{M}$ has as vertices the nodes of G that are deep merges of (Γ_{F}, Q)-free paths starting at unrevealed output nodes, and an edge $f \to f'$ whenever f and f' are consecutive deep merges of such a path. A vertex is *initial* if it is the first deep merge of such a path.

Steps decrease along paths, so $\mathcal{M}$ is acyclic. It is determined by the code, that is, by the representation or the pruning together with the lineage assignment, and by Γ_{F} and Q; paths of H_M that the code does not follow play no role in it.

Proposition 3.21 (Code-level depth reduction suffices)**.** Lean: `MergeDepth.merge_reduction_machine` [`MergeDepth/MergeSteps.lean`]

In the setting of Lemma 3.10, with G *the coordinate DAG of an* $\mathbb{F}$*-linear data path of width* m *or the field DAG of a pruning with* $m = k+2$*, let* $s \ge 0$*,* $r = (s+1)\tau D$*, and let* Z *be a set of*

vertices of $\mathcal{M}$ that meets every path of $\mathcal{M}$ with $s+1$ vertices whose first vertex is initial. Then every $(\Gamma_{\mathrm{F}}, Q \cup Z)$-free path of G from an output node that is not revealed for $(\Gamma_{\mathrm{F}}, Q \cup Z)$ has fewer than r edges. If M transposes all $d \times d$ matrices, the conclusion of Theorem 3.11(a) holds with $X = \emptyset$ and the rank bound increased by $|Z|$, and that of Theorem 3.11(b) with $X = \emptyset$ and the bit count increased by $\beta|Z|$.

Outline of the formal proof. Let π be such a path, from an output node u_0. It is (Γ_{F}, Q)-free and starts at an unrevealed output node, so its deep merges $f_1, \dots, f_{s'}$ form a path of $\mathcal{M}$ whose first vertex is initial. If $s' \geq s+1$, then Z contains one of $f_1, \dots, f_{s+1}$; it is a vertex of π other than u_0, which is not in Z, so π would not be Z-free after its first vertex. Hence $s' \leq s$, and π has fewer than r edges by Lemma 3.10. Apply Lemma 3.3 with $\Gamma = \Gamma_{\mathrm{F}}$, the set $Q \cup Z$ and $X = \emptyset$, as in the proof of Theorem 3.11; a node of Z costs rank at most one in (a) and at most β bits in (b).

Formally, the merge DAG and the cut condition are `MergeDepth.MEdge`, `MergeDepth.MInit` and `MergeDepth.MCut`, for any network, deep relation and lineage assignment. `MergeDepth.dm_le_of_cut` bounds the deep merges of the paths in question by s, and `MergeDepth.length_le_of_cut` adds the bound of `MergeDepth.length_le_of_cov` (Lemma 3.10). For a linear network, `MergeDepth.merge_reduction` uses `MergeDepth.rigidity_core_rev`, the form of Lemma 3.3(a) in which a revealed output node contributes to R only; `MergeDepth.merge_reduction_steps` passes to the coordinate DAG as in Theorem HC′, with the rank bound $\beta|\Gamma_{\mathrm{F}}| + |Q| + |Z|$, and `MergeDepth.merge_reduction_machine` states rank $R \leq \varepsilon L + mkC(L+1)/\tau + |Z|$ with $\tau|Q| \leq mkC(L+1)$. For field DAGs, `MergeDepth.merge_reduction_fields` for refinements and `MergeDepth.merge_reduction_pruning` for the field DAG of a pruning give an index code with $\beta|\Gamma_{\mathrm{F}}| + \beta|Q| + \beta|Z|$ bits and $\tau|Q| \leq (k+2)N_{\mathcal{R}}$. □

The proposition gives a sufficient condition only. It moves the remaining difficulty from the schedule to the code: on any schedule, depth-robust or not, a code whose merge DAG has a cut of $o(L)$ vertices meeting all long merge paths is excluded as in Corollaries HC and HC*. Heuristically, a code that merges deep lineages at every deep visit has a merge DAG as rich as the deep part of the schedule, and then no such cut need exist.

Conjecture 3.22 (Conjecture HV*: code-level depth reduction)**.** *Let $k, V \geq 1$ and $\beta \geq 2$ be integers, $C \geq 1$ and $0 < \varepsilon \leq 1$. There is $c > 0$ such that for every integer $\tau \geq 1$ there is an integer r with the following property. For every $\eta > 0$ there is d_0 such that, for every $d \geq d_0$, every machine with k work tapes and value width at most β that is oblivious at length $L = d^2 b(d)$ with halting time at most $C(L+1)$ and transposes all $d \times d$ matrices has an index code with at most $\varepsilon L + cN_{\mathcal{R}}/\tau + \eta L$ bits for its ball system $(B^r(\ell))_\ell$, where $N_{\mathcal{R}}$ counts its deep hops at V. In the* linear form*, for a field $\mathbb{F}$ and an integer $m \geq 1$, with c allowed to depend on m and $\mathbb{F}$, the same holds for the machines that have an $\mathbb{F}$-linear data path of width at most m at length L, with the minrank over $\mathbb{F}$ of the ball system in place of the length of an index code.*

Remark 3.23 (On Conjecture HV*)**.** (a) Theorem 3.11 proves the conjecture, with $\eta = 0$, $r = (s+1)\tau D$ and $c = \beta(k+2)$, or $c = m$ in the linear form, for machines whose codes have merge depth at most s for some pruning or representation, cold genie and lineage assignment; Proposition 3.21 proves it when the merge DAG has a cut as there of at most $\eta L/\beta$ vertices. (b) Since $N_{\mathcal{R}} \leq kC(L+1)$, the conjecture is equivalent to the statement that for every $\varepsilon' > 0$ there is r such that these ball systems have index codes, or minrank, of at most $\varepsilon' L + o(L)$: take $\varepsilon = \varepsilon'/2$ and $\tau \geq 4ckC/\varepsilon'$ in one direction, any c in the other. The form with $N_{\mathcal{R}}$ records the rate at which (a) charges deep hops; V does not affect the truth of the conjecture. (c) The conjecture and an affirmative answer to Question 9.1 for every degree bound exclude every oblivious linear-time

transposer: for a machine M take $V = 1$, $\beta = \beta(M)$, $\varepsilon = \frac{1}{8}$, $\tau \geq 8ckC$ and $\eta = \frac{1}{16}$; at large d the ball system has an index code with at most $\frac{7}{16}L$ bits, so some fibre, a storage code for the relation $B^r(\mathrm{tpos}(q))$, has at least $2^{9L/16}$ elements, contrary to Lemma 3.15(b) with $\delta = \frac{1}{32}$. Likewise the linear form and $\mathrm{LinCrux}_{\mathbb{F}}(\mathcal{S}^{\mathrm{ball}}_{k,C,\beta,r}, \rho)$, at the radius r attached to some $\tau \geq 8ckC/(\rho - \varepsilon)$ with $\varepsilon < \rho$, exclude linear ones, as in the proof of Corollary HC. Since [Q9.1] is false [8], the first statement of (c) is vacuous as stated. It uses [Q9.1] only through the conclusion of Lemma 3.15(b), which is open; and since the index code is one for the ball system itself, the all-code crux $\mathrm{Crux}(\mathcal{S}^{\mathrm{ball}}_{k,C,\beta,r}, \rho)$ for some $\rho > \frac{7}{16}$, also open, serves in its place, as after Corollary HC*. (d) Heuristically, the conjecture is of network-coding type: after the cold genie the deep hops are the only long-range carriers (Proposition 3.17), and the conjecture bounds what they deliver against side information in balls of bounded radius by a constant times their number, divided by τ. In general networks such bounds fail: coding can beat routing by a factor that grows with the size of an undirected network [2]. So the conjecture can hold only because of the shape of tape schedules and the transposition demand, and a counterexample would be a tape-shaped network code on deep hops that merges deep lineages along a pattern without cheap cuts, with a gain over routing that grows with the radius: the mechanism a construction needs.

4 Skeleton entropy and input steering

The eighth paper excludes oblivious linear-time transposers whose work heads enter every cell boundedly often, given an affirmative answer to Question 9.1 of the fifth paper, and extends the exclusion to adaptive machines of bounded entry whose traces have entropy at most ηL for a fixed $\eta < \frac{1}{2}$ [3, Corollaries 6.3 and 6.4]. It also observes that this entropy hypothesis is not robust: a preliminary sweep of the input head with a one-step detour at every 1 of the first half of the input makes the trace reveal half the input without changing the computation, and a form that survives such padding would bound the entropy of a coarse skeleton of the trace instead [3, Section 6.2]. This section supplies that form, Theorem SE, and draws its consequences for heads that enter every cell boundedly often. At large scales Theorem SE is not affected by padding of bounded entry before the run, such as that of the eighth paper; padding whose heads revisit cells unboundedly often can still affect it (Remark 4.18).

The skeleton at scale w records, at every step, which block of w consecutive cells contains each head, and the output count rounded down to a multiple of w (Definition 4.1). The tool is the block machine (Definition 4.4 and Lemma 4.5): on the inputs that share a skeleton, the machine is an oblivious machine whose heads move on blocks of w cells, its dependency DAG is the walk DAG of the coarse walks, and bounded entry of a head gives bounded entry of its coarse walk. Theorem F, the cone lemma and the paired-distance argument of the fifth paper then apply to one recovery relation per skeleton class, which gives Theorem SE (Theorem 4.13). At large scales, heads of bounded entry carry almost no skeleton entropy (Lemmas 4.14 and 4.15). Hence, given [Q9.1], a linear-time transposer whose work tapes have bounded entry must steer its input head by about half of the input at every scale (Corollary IS, Corollary 4.16), and no linear-time transposer, adaptive or not, has bounded entry on every tape, the input tape included (Corollary RV, Corollary 4.17).

Every result of this section is formalized in Lean, with [Q9.1] as a hypothesis of Lemma 4.12, Theorem SE and Corollaries IS and RV only. The formal statements of these four results hold for every entry width $b \geq 1$ in place of $b(d)$, at all dimensions d above a threshold that does not depend on b.

4.1 Skeletons

We use the machine model and the notation of Section 2, with the following details [3, Section 2.1]. A machine M has a finite state set $\mathcal{Q}$ and $k \geq 1$ work tapes over a finite alphabet Σ that contains the blank, and every step writes the cell under each work head. At a length $L = d^2 b(d)$ the input is a word $X \in \{0,1\}^L$ (we write X because w will denote a scale), with bits $X[p]$ in the cells $p \in [0, L)$ of the input tape; every other cell of the input tape reads as an end marker. On input X the machine executes the steps $0, \ldots, T(X) - 1$ and then halts. For $0 \leq t \leq T(X)$ we write $x_0(t)$ for the position of the input head at time t, that is, before step t, $x_j(t)$ for the position of work head j ($1 \leq j \leq k$), and $n(t)$ for the number of bits appended at the steps before t; we write $x_j(t; X)$ and $n(t; X)$ when the input varies. M is a *transposer* at length L if on every input it appends exactly L bits and its output bit at position r is $X[\mathrm{tpos}(r)]$, where $\mathrm{tpos}((di + j)b + s) = (dj + i)b + s$ for $0 \leq i, j < d$ and $0 \leq s < b = b(d)$; the map tpos is an involution, written Π in the fifth paper [10, (3.1)]. It works *within* $c(L + 1)$ *steps* if $T(X) \leq c(L + 1)$ for every input. Walks, hops, entries, owner-load and walk DAGs are those of Section 2. The machine has *entry at most* V *on tape* j on input X if the walk $x_j(\cdot\,; X)$ on $[0, T(X))$ enters every cell at most V times. Entropies $\mathrm{H}(\cdot)$ are Shannon entropies in bits, for a uniformly random input $X \in \{0,1\}^L$. In the formal statements, machines and their runs are those of the first paper's library, in which (T) is stated [6]; walks are maps $\mathbb{N} \to \mathbb{Z}$ with steps in $\{-1, 0, 1\}$, and entries are the eighth paper's `BoundedNesting.entries`.

Definition 4.1 (Skeletons)**.** Lean: `MergeDepth.Skeleton`, `MergeDepth.InClass`, `MergeDepth.skelClass`, `MergeDepth.runSkeleton` [`MergeDepth/BlockMachine.lean`]

Let $w \geq 1$ be an integer, the *scale*. The *block* of $a \in \mathbb{Z}$ is $\mathcal{B}_a = \{wa, \ldots, wa + w - 1\}$, and on the input tape $\mathcal{B}_a^{\mathrm{in}} = \mathcal{B}_a \cap [0, L)$. The *coarse walk* of a walk x is $\lfloor x/w \rfloor$; its value at t is the index of the block that contains $x(t)$. The *skeleton* at scale w of the run on input X is the sequence

$$\sigma_w(X) = \Big(\big(\lfloor x_0(t)/w \rfloor, \lfloor x_1(t)/w \rfloor, \ldots, \lfloor x_k(t)/w \rfloor, \lfloor n(t)/w \rfloor \big) \Big)_{0 \leq t < T(X)},$$

and its *input skeleton* is $\sigma_w^{\mathrm{in}}(X) = \big(\lfloor x_0(t)/w \rfloor \big)_{0 \leq t < T(X)}$; both have length $T(X)$. The *skeleton class* of a sequence σ is $\mathcal{X}_\sigma = \{X \in \{0,1\}^L : \sigma_w(X) = \sigma\}$. For a skeleton σ we write T_σ for its length and $s_0, s_1, \ldots, s_k, s_{\mathrm{out}} \colon [0, T_\sigma) \to \mathbb{Z}$ for its components; $s_1, \ldots, s_k$ are the *coarse work walks* of σ.

All members of a class $\mathcal{X}_\sigma$ halt at time T_σ, and at every time $t < T_\sigma$ their heads on one tape lie in one block, as do their output counts; so their positions on each tape, and their output counts, differ by at most $w - 1$. Formally, the class is the fibre of σ_w over σ among the halting runs (`MergeDepth.inClass_iff_runSkeleton`). If $\mathcal{X}_\sigma \neq \emptyset$, every component s_j is a walk with $s_j(0) = 0$, because $\lfloor (z \pm 1)/w \rfloor - \lfloor z/w \rfloor \in \{-1, 0, 1\}$, and s_{out} is nondecreasing with steps 0 and 1. The trace of a run, that is, its head moves, append times and halting time, determines its skeleton at every scale; at $w = 1$ the skeleton determines the trace up to the moves of the last step and to whether that step appends. Finally, since $\mathrm{H}(Y) \geq -\log_2 \max_y \Pr[Y = y]$ for every random variable Y with finitely many values (`MergeDepth.exists_large_fibre`),

$$\mathrm{H}(\sigma_w(X)) \leq \left(\tfrac{1}{2} - \eta\right) L \quad \Longrightarrow \quad |\mathcal{X}_\sigma| \geq 2^{\left(\frac{1}{2} + \eta\right) L} \text{ for some skeleton } \sigma. \tag{2}$$

Lemma 4.2 (Coarse walks)**.** Lean: `MergeDepth.entries_skel_subset`, `MergeDepth.card_entries_skel_le`, `MergeDepth.card_block_changes_le` [`MergeDepth/SkeletonWalk.lean`]

Let $V, w, T \geq 1$ be integers, let x be a walk of length T with $x(0) = 0$ that enters every cell at most V times, and let $s = \lfloor x/w \rfloor$.

(a) *If t is an entry of an integer a by s, then $x(t) \in \{wa, wa+w-1\}$ and t is an entry of the cell $x(t)$ by x.*

(b) *The walk s enters every integer at most $2V$ times.*

(c) *There are at most $2V\big(\lfloor (T-1)/w \rfloor + 2\big) - 1$ times $t \in [1,T)$ with $s(t) \neq s(t-1)$.*

Outline of the formal proof. (a) If $t = 0$, then $a = s(0) = 0$ and t is the entry of the cell $x(0) = 0 = wa$. If $t \geq 1$, then $s(t-1) \neq a = s(t)$, so $x(t-1) \notin \mathcal{B}_a \ni x(t)$. Hence $x(t) = x(t-1) \pm 1$ is an end cell of $\mathcal{B}_a$, namely wa or $wa+w-1$, entered from outside the block, and t is an entry of $x(t)$ by x.

(b) By (a), distinct entries of a by s are distinct entries by x of the at most two cells wa and $wa+w-1$, each of which is entered at most V times.

(c) The cells visited by x form an interval of at most T consecutive integers, and such an interval meets at most $\lfloor (T-1)/w \rfloor + 2$ blocks. Every time $t \geq 1$ with $s(t) \neq s(t-1)$ is an entry of $s(t)$ by s. By (b) each of the visited blocks has at most $2V$ entries, and one of these entries, the entry of 0 at time 0, is not such a time.

Formally, the coarse walk is `MergeDepth.skel`, and part (a) needs no entry bound. □

Remark 4.3 (Hops of two members need not match)**.** A natural route to a robust form of Corollary F″ applies Theorem F to the trace of each member of a skeleton class and merges the resulting cuts and cones. That route needs the hops of two members to correspond within $O(w)$ steps, and this fails because a head can linger near a cell without leaving its block. Let $w \geq 2$, $\lambda \geq 2$ and $h \geq w$ be integers and $a = 2w+1$. Let y be the walk that climbs from 0 to a, stays at a for λ steps, climbs to $a+h$, which it reaches at the time $o = a+\lambda+h$, and descends to 0; for $a < s' < a+\lambda$ let $y_{s'}$ agree with y except that $y_{s'}(s') = a-1$. The cells $a-1 = 2w$ and a lie in $\mathcal{B}_2$, so all these walks have the same coarse walk; y enters every cell at most twice and $y_{s'}$ at most three times. The upper long hops at the cell $2w$ that contain the time o start at the time $2w$ for y and at the time s' for $y_{s'}$, so their starts differ by $s'-2w$, which takes every value from 2 to λ, although the cell $2w$ is farther than w from both ends of the family of y with owner o, whose base is 0 and whose height is $a+h$. Pauses are not essential when $w \geq 3$. For $h > \lambda$ let y' climb from 0 to $2w+2$, make an excursion of 2λ steps up to $2w+2+\lambda$ and back, visit $2w+1, 2w+2, 2w+1, 2w+2$, then climb to $2w+2+h$ and descend to 0, and let y'' agree with y' except that it visits $2w+1, 2w, 2w+1, 2w+2$ after the excursion. The cells $2w, 2w+1, 2w+2$ lie in $\mathcal{B}_2$ because $w \geq 3$, so y' and y'' have the same coarse walk; neither pauses, both enter every cell at most five times, and their upper long hops at the cell $2w$ that contain the time of the top start $2\lambda+4$ steps apart. The block machine below avoids per-trace structure altogether: its cut is determined by the skeleton.

4.2 The block machine

Fix a scale w and a skeleton σ with $\mathcal{X}_\sigma \neq \emptyset$, and write $T = T_\sigma$.

Definition 4.4 (The block machine)**.** Lean: `MergeDepth.BlockVal`, `MergeDepth.blockVal`, `MergeDepth.blockWidth` [`MergeDepth/BlockMachine.lean`]

For $X \in \mathcal{X}_\sigma$ and $0 \leq t < T$, the *block value* $\bar{v}_t(X)$ consists of

(i) the state after step t;

(ii) for every work tape j, the contents of the w cells of $\mathcal{B}_{s_j(t)}$ after step t;

(iii) for every head $j \in \{0, 1, \ldots, k\}$, the offset $x_j(t+1) - ws_j(t)$;

(iv) the word $O_t(X)$ of the output bits at the positions in $[ws_{\mathrm{out}}(t), n(t+1))$, in order.

For $1 \le j \le k$ and $0 \le t < T$ let $\mathrm{prev}_j^\sigma(t) = \max\{t' < t : s_j(t') = s_j(t)\}$ when this set is nonempty. The *block DAG* $\bar{H}_\sigma$ has vertex set $[0,T)$, a *state edge* $t \to t-1$ for every $1 \le t < T$, and a *block edge* $t \to \mathrm{prev}_j^\sigma(t)$ for every work tape j and every t for which $\mathrm{prev}_j^\sigma(t)$ is defined; an edge is determined by its endpoints, so a block edge of length one is a state edge. The block values on $\mathcal{X}_\sigma$, the block DAG and the components of σ form the *block machine* of M on σ. Its *value width* is

$$\beta_w = \lceil \log_2 |\mathcal{Q}| \rceil + kw \lceil \log_2 |\Sigma| \rceil + (k+1)\lceil \log_2(w+2) \rceil + w + 1.$$

The offset in (iii) lies in $\{-1, 0, \dots, w\}$, because $x_j(t) \in \mathcal{B}_{s_j(t)}$ and heads move by at most one cell, and the word in (iv) has length at most w, because $n(t+1) \le n(t)+1 \le w s_{\mathrm{out}}(t)+w$. So a block value is described by at most β_w bits: (ii) takes $kw\lceil \log_2|\Sigma| \rceil$ bits, (iii) takes $(k+1)\lceil \log_2(w+2) \rceil$ bits, and (iv), one of the $2^{w+1}-1$ binary words of length at most w, takes $w+1$ bits. Formally, block values take at most 2^{β_w} values on every input and at every time (`MergeDepth.card_admissible_le`, `MergeDepth.blockVal_mem_admissible`).

Genies, Γ-free paths and full depth are defined for $\bar{H}_\sigma$ as for the dependency DAG of a trace in the eighth paper [3, Section 2.4]: a *genie* is a set $\Gamma \subseteq [0,T)$, it reveals the block values $\bar{v}_u(X)$, $u \in \Gamma$, with β_w bits each, at *cost* $\beta_w|\Gamma|$; a path $t_0 \to \cdots \to t_m$ is Γ-*free* if $t_1, \dots, t_m \notin \Gamma$; and Γ has *full depth at most* D if every Γ-free path has at most D edges.

Lemma 4.5 (The block machine)**.** Lean: `MergeDepth.blockVal_eq_of_args`, `MergeDepth.block_oblivious`, `MergeDepth.outTime_spec`, `MergeDepth.card_outTime_eq_le`, `MergeDepth.skeleton_entries_ownerLoad` [`MergeDepth/BlockMachine.lean`]

Let σ be a skeleton at scale w with $\mathcal{X}_\sigma \neq \emptyset$, and $T = T_\sigma$.

(a) (Dependency DAG.) *$\bar{H}_\sigma$ is the walk DAG of the coarse work walks $s_1, \dots, s_k$, and every vertex has in-degree and out-degree at most $k+1$.*

(b) (Obliviousness.) *For every $t < T$ there is a function Φ_t, determined by M, σ and t, such that for every $X \in \mathcal{X}_\sigma$*

$$\bar{v}_t(X) = \Phi_t\Big(\bar{v}_{t-1}(X),\ \bar{v}_{\mathrm{prev}_1^\sigma(t)}(X), \dots, \bar{v}_{\mathrm{prev}_k^\sigma(t)}(X),\ X|_{\mathcal{B}^{\mathrm{in}}_{s_0(t)}}\Big),$$

where an argument that is not defined, because $t = 0$ or because $\mathrm{prev}_j^\sigma(t)$ is not defined, is replaced by a fixed symbol.

(c) (Output decoding.) *Suppose that every member of $\mathcal{X}_\sigma$ appends exactly L bits. For $r \in [0, L)$ the time $\hat{t}_r = \max\{t < T : s_{\mathrm{out}}(t) = \lfloor r/w \rfloor\}$ exists, and for every $X \in \mathcal{X}_\sigma$ the output bit at position r is the letter of $O_{\hat{t}_r}(X)$ at place $r - w\lfloor r/w \rfloor$, counted from 0. At most w positions r share one time $\hat{t}_r$.*

(d) (Entries.) *If some member of $\mathcal{X}_\sigma$ has entry at most V on every work tape, then every coarse work walk s_j enters every integer at most $2V$ times, and the set of all long hops of $s_1, \dots, s_k$ has owner-load at most $4V-1$ [3, Definition 2.2].*

Outline of the formal proof. (a) For a walk y and a time u, the hop of y ending at u is $(\mathrm{prev}_y(u), u)$, where $\mathrm{prev}_y(u) = \max\{t < u : y(t) = y(u)\}$, and it is long if $u - \mathrm{prev}_y(u) \ge 2$ [3, Section 2.2]. For $y = s_j$ this is prev_j^σ. So the block edges of length at least two are the edges $u \to p$ for the long hops (p, u) of $s_1, \dots, s_k$, which are the hop edges of their walk DAG, and the block edges of length one are pauses, which the walk DAG identifies with state edges [3, Section 2.3], as $\bar{H}_\sigma$ does. So the two DAGs coincide. A vertex t has out-edges only to $t-1$ and to the at most k vertices $\mathrm{prev}_j^\sigma(t)$. Its

in-edges come from $t+1$ and, for each j, from at most one t' with $\mathrm{prev}_j^\sigma(t') = t$: if $t < t' < t''$ and $\mathrm{prev}_j^\sigma(t') = \mathrm{prev}_j^\sigma(t'') = t$, then $s_j(t'') = s_j(t) = s_j(t')$, so $\mathrm{prev}_j^\sigma(t'') \geq t' > t$.

(b) Fix $t < T$ and $X \in \mathcal{X}_\sigma$. Step t applies the transition function of M to the state at time t, the input symbol under $x_0(t)$ and the symbols under $x_1(t), \ldots, x_k(t)$. These data, and the positions $x_j(t)$, are functions of the arguments of Φ_t through maps that depend only on M, σ and t:

- The state at time t is the initial state if $t = 0$, and item (i) of $\bar{v}_{t-1}(X)$ otherwise.
- $x_j(0) = 0$, and for $t \geq 1$, $x_j(t) = ws_j(t-1) + e_j$, where e_j is the offset of head j in item (iii) of $\bar{v}_{t-1}(X)$ and $s_j(t-1)$ is read from σ.
- Let $1 \leq j \leq k$. If $t' = \mathrm{prev}_j^\sigma(t)$ is defined, the contents of $\mathcal{B}_{s_j(t)}$ on tape j at time t are those after step t', which form item (ii) of $\bar{v}_{t'}(X)$: for $t' < \tau < t$ we have $s_j(\tau) \neq s_j(t)$, so $x_j(\tau) \notin \mathcal{B}_{s_j(t)}$, and tape j is written only by head j and only in the cell under it. If $\mathrm{prev}_j^\sigma(t)$ is not defined, head j has not been in $\mathcal{B}_{s_j(t)}$ before time t, so the block is blank. In both cases the symbol under head j is the one at place $x_j(t) - ws_j(t)$ of the block.
- If $x_0(t) \in [0, L)$, then $x_0(t) \in \mathcal{B}^{\mathrm{in}}_{s_0(t)}$, and the input symbol $X[x_0(t)]$ is a component of $X|_{\mathcal{B}^{\mathrm{in}}_{s_0(t)}}$; otherwise it is the end marker.

The transition yields the new state, the symbols written, the moves and the appended bit, if any. Item (i) of $\bar{v}_t(X)$ is the new state; item (ii) for tape j is the contents of $\mathcal{B}_{s_j(t)}$ at time t with the cell $x_j(t)$ overwritten by the symbol written there; item (iii) for head j is $x_j(t) - ws_j(t)$ plus the move of head j. For item (iv): if $t \geq 1$ and $s_{\mathrm{out}}(t) = s_{\mathrm{out}}(t-1)$, then $O_{t-1}(X)$ is the word of the positions in $[ws_{\mathrm{out}}(t), n(t))$, and $O_t(X)$ is $O_{t-1}(X)$ followed by the bit appended at step t, if any, whose position is $n(t)$. Otherwise $O_t(X)$ consists of that bit alone, or is empty, because $n(t) = ws_{\mathrm{out}}(t)$: for $t = 0$ both sides are 0, and for $t \geq 1$ the count grows by at most one per step, so $s_{\mathrm{out}}(t) \neq s_{\mathrm{out}}(t-1)$ gives $n(t-1) < ws_{\mathrm{out}}(t) \leq n(t) \leq n(t-1) + 1$. The values of s_{out} are read from σ.

(c) Fix $X \in \mathcal{X}_\sigma$. Since $n(0) = 0$, $n(T) = L$ and n grows by 0 or 1 per step, for every $r < L$ there is a step $t_r < T$ with $n(t_r) = r < n(t_r + 1)$, the step that appends bit r. Then $s_{\mathrm{out}}(t_r) = \lfloor r/w \rfloor$, so $\hat{t}_r$ exists and $\hat{t}_r \geq t_r$, whence $n(\hat{t}_r + 1) \geq n(t_r + 1) = r + 1$. So $O_{\hat{t}_r}(X)$, the word of the positions in $[w\lfloor r/w \rfloor, n(\hat{t}_r + 1))$, contains position r, at place $r - w\lfloor r/w \rfloor$. The time $\hat{t}_r$ depends only on σ and r, and $s_{\mathrm{out}}(\hat{t}_r) = \lfloor r/w \rfloor$; so positions with distinct values of $\lfloor r/w \rfloor$ have distinct times, and at most w positions share a value.

(d) The coarse work walks of σ are those of the head walks $x_j(\cdot\,; X)$ on $[0, T)$ of that member X, which start at 0. By Lemma 4.2(b) they enter every integer at most $2V$ times, and the eighth paper's Corollary 3.2, which is Lean-verified there, applied with $2V$ in place of V, bounds the owner-load of the set of all their long hops by $2 \cdot 2V - 1$ [3, Corollary 3.2].

Formally, prev_j^σ is `MergeDepth.prevS`, the block DAG is `MergeDepth.BlockStep`, and the dependency DAG of the coarse walks is `MergeDepth.DepStep`; part (a) is `MergeDepth.blockStep_iff`, with the degree bounds `MergeDepth.card_blockOut_le` and `MergeDepth.card_blockIn_le`. Part (b) is `MergeDepth.block_oblivious`, derived from its relational form `MergeDepth.blockVal_eq_of_args`; part (c) is `MergeDepth.outTime_spec` and `MergeDepth.card_outTime_eq_le`, with $\hat{t}_r$ given by `MergeDepth.outTime`; and part (d) is `MergeDepth.skeleton_entries_ownerLoad`. The formal proofs give more: (b) holds for inputs whose runs follow σ up to time t, halting or not (`MergeDepth.blockVal_eq_of_coarseAgree`); (c) uses only the output component of σ and decodes bit r of every member that appends more than r bits; and (a) and (b) do not need $\mathcal{X}_\sigma \neq \emptyset$. □

So on a skeleton class, M is an oblivious machine of value width β_w: its work heads move along the coarse work walks on tapes whose cells are blocks, its input head reads one block of w input

cells per step, its output is written one block at a time, and its dependency DAG is $\bar{H}_\sigma$. Motion inside a block, however data-dependent, creates no edge: it is carried by the items (ii) and (iii) of the block values along state edges. This is what makes adaptivity tractable: the tools for oblivious machines apply to each skeleton class, at the price of the value width β_w, and only the coarse walks enter the depth analysis.

4.3 One recovery relation per skeleton class

Lemma 4.6 (A genie for a skeleton class)**.** Lean: `MergeDepth.skel_genie_cost` [`MergeDepth/SkeletonGenie.lean`]

Let M be a machine with k work tapes, let $V, w \geq 1$ be integers, $c > 0$ and $\varepsilon > 0$, and put $C = \max(c, 1)$,

$$\varepsilon'' = \frac{\varepsilon}{4\beta_w C}, \qquad P = \left\lceil \frac{4\beta_w C}{\varepsilon} \right\rceil, \qquad D_1 = D_{\mathrm{F}}(k, 4V-1, \varepsilon''), \qquad D = D_1 P - 1,$$

where D_{F} is the bound of Theorem F [3, Theorem 3.6]. Let σ be a skeleton of M at scale w at a length $L \geq 1$, with $T_\sigma \leq c(L+1)$, and suppose that some member of $\mathcal{X}_\sigma$ has entry at most V on every work tape. Then there is a genie Γ, depending only on σ, of cost $\beta_w|\Gamma| \leq \varepsilon L$ and full depth at most D in $\bar{H}_\sigma$.

Outline of the formal proof. Write $T = T_\sigma$. By Lemma 4.5(d) the set of all long hops of the coarse work walks $s_1, \ldots, s_k$, which are walks of length T, has owner-load at most $4V - 1$. Theorem F [3, Theorem 3.6], which allows pauses and any coupling of the walks and whose Lean-verified form holds for every $\varepsilon'' > 0$, gives a cut F of their walk DAG with $|F| \leq \varepsilon'' T$ such that every path that uses no edge of F has at most $D_1 - 1$ hop edges. By Lemma 4.5(a) this walk DAG is $\bar{H}_\sigma$, and its hop edges are its edges of length at least two. Fix such a cut; it depends only on σ.

Let Γ_0 be the set of the vertices u such that some edge $t \to u$ lies in F; so $|\Gamma_0| \leq |F|$. Every edge of a Γ_0-free path ends outside Γ_0, so it is not in F, and such a path has at most $D_1 - 1$ edges of length at least two. Put $\Gamma = \Gamma_0 \cup \{t < T : t \equiv P - 1 \pmod P\}$ and let π be a Γ-free path. Removing its at most $D_1 - 1$ edges of length at least two leaves at most D_1 maximal runs of state edges. The vertices of such a run other than its first are vertices of π other than its first, hence lie outside Γ, and they are consecutive integers; so there are at most $P - 1$ of them, and the run has at most $P - 1$ edges. Hence π has at most $(D_1 - 1) + D_1(P - 1) = D$ edges. Finally $|\Gamma| \leq |F| + \lfloor T/P \rfloor$, so, using $T \leq C(L+1)$ and $L \geq 1$,

$$\beta_w|\Gamma| \leq \beta_w \varepsilon'' T + \frac{\beta_w T}{P} \leq \frac{\varepsilon T}{4C} + \frac{\varepsilon T}{4C} \leq \frac{\varepsilon(L+1)}{2} \leq \varepsilon L.$$

This is the construction of the cold genie (Lemma 3.1), for the coarse work walks at the entry bound $2V$, at which no hop is deep.

Formally, the proof applies `MergeDepth.cold_genie` to the coarse walks of the member, with $2V$ in place of V; none of their steps is deep (`MergeDepth.rCount_deep_coarse_eq_zero`), and `MergeDepth.skel_genie` is the resulting path bound. `MergeDepth.skeleton_genie` states the lemma for the block DAG of σ, whose walks agree with the coarse walks of the member only before T_σ. □

The genie is the same set of times for every member of the class; only the revealed values depend on the input.

Lemma 4.7 (Cones on a skeleton class)**.** Lean: `MergeDepth.blockMachine`, `MergeDepth.blockMachine_card_cone_le`, `MergeDepth.blockMachine_cone`, `MergeDepth.blockMachine_output_eq`, `MergeDepth.blockMachine_isStorageCode_fibre` [`MergeDepth/BlockMachine.lean`]

Let σ be a skeleton at scale w with $\mathcal{X}_\sigma \neq \emptyset$, every member of which appends exactly L bits, and let $\Gamma \subseteq [0, T_\sigma)$ have full depth at most D in $\bar{H}_\sigma$. For $t < T_\sigma$ let $C_\Gamma(t)$ be the set of endpoints of Γ-free paths from t in $\bar{H}_\sigma$, so that $t \in C_\Gamma(t)$, and for $r, q \in [0, L)$ put

$$J_\Gamma(t) = \bigcup_{u \in C_\Gamma(t)} \mathcal{B}^{\mathrm{in}}_{s_0(u)}, \qquad I_\Gamma(r) = J_\Gamma(\hat{t}_r), \qquad N_\Gamma(q) = I_\Gamma(\mathrm{tpos}(q)).$$

Let $G_\Gamma(X) = (\bar{v}_u(X))_{u \in \Gamma}$ be the revealed word, and $B = \sum_{h=0}^{D} (k+1)^h$.

(a) $|C_\Gamma(t)| \leq B$ for every t, and $|N_\Gamma(q)| \leq wB$ for every q.

(b) If $X, Y \in \mathcal{X}_\sigma$ satisfy $G_\Gamma(X) = G_\Gamma(Y)$ and $X|_{J_\Gamma(t)} = Y|_{J_\Gamma(t)}$, then $\bar{v}_t(X) = \bar{v}_t(Y)$. Consequently, if $G_\Gamma(X) = G_\Gamma(Y)$ and $X|_{I_\Gamma(r)} = Y|_{I_\Gamma(r)}$, the output bits of X and Y at position r agree.

(c) If M transposes every input of $\mathcal{X}_\sigma$, then for every value g of the revealed word the fibre $\{X \in \mathcal{X}_\sigma : G_\Gamma(X) = g\}$ is a binary storage code for the recovery relation N_Γ on $[0, L)$ in the sense of the fifth paper [10, Definition 2.1].

Outline of the formal proof. (a) Every vertex of $C_\Gamma(t)$ is the endpoint of a Γ-free path from t with at most D edges, and by Lemma 4.5(a) at most $(k+1)^h$ paths with h edges start at t. Every block has at most w input cells, and $N_\Gamma(q) = J_\Gamma(\hat{t}_{\mathrm{tpos}(q)})$.

(b) By strong induction on t, as in the fourth paper's cone lemma [4, Lemma 4.4], with Lemma 4.5(b) in place of its local recursion. We show that the arguments of Φ_t agree on X and Y. The constant arguments agree. Since $t \in C_\Gamma(t)$, the block $\mathcal{B}^{\mathrm{in}}_{s_0(t)}$ is contained in $J_\Gamma(t)$, on which X and Y agree. Every other argument is $\bar{v}_{t'}$ for an edge $t \to t'$ of $\bar{H}_\sigma$. If $t' \in \Gamma$, then $\bar{v}_{t'}(X) = \bar{v}_{t'}(Y)$ is a component of the common revealed word. If $t' \notin \Gamma$, then every Γ-free path from t', preceded by the edge $t \to t'$, is a Γ-free path from t; so $C_\Gamma(t') \subseteq C_\Gamma(t)$ and $J_\Gamma(t') \subseteq J_\Gamma(t)$, and the induction hypothesis for $t' < t$ gives $\bar{v}_{t'}(X) = \bar{v}_{t'}(Y)$. Hence $\bar{v}_t(X) = \bar{v}_t(Y)$. The second sentence is the case $t = \hat{t}_r$, by Lemma 4.5(c).

(c) Let X and Y lie in the fibre and agree on $N_\Gamma(q) = I_\Gamma(\mathrm{tpos}(q))$. By (b) their output bits at position $\mathrm{tpos}(q)$ agree; since M transposes and tpos is an involution, these bits are $X[q]$ and $Y[q]$.

Formally, the lemma is proved for an abstract structure, `MergeDepth.BlockFunDag`: a DAG on the times with out-edges to earlier times, an input block read at every time, and values that are determined on a class of inputs by the block read and the values at the out-neighbours, as Lemma 4.5(b) provides. There $C_\Gamma(t)$ is `MergeDepth.coneV` and $J_\Gamma(t)$ is `MergeDepth.BlockFunDag.coneJ`; part (a) is `MergeDepth.card_coneV_le` and `MergeDepth.BlockFunDag.card_coneJ_le`, part (b) is `MergeDepth.BlockFunDag.cone` and `MergeDepth.BlockFunDag.output_eq`, and part (c) is `MergeDepth.BlockFunDag.isStorageCode_fibre`, for the storage codes `MotionDepthCodes.IsStorageCode` of the fourth paper's library, which the fifth paper's library uses. The block machine of a skeleton class is such a structure (`MergeDepth.blockMachine`), and the other declarations of the status line are (a)–(c) for it; (c) needs only that the members output the transpose, with no bound on their running time. □

The relation N_Γ depends only on σ and Γ. Compared with the cone relation of a single trace, every cone vertex contributes a block of w input cells instead of one, and the at most w output positions of one output block share their cone.

4.4 Paired distances on a skeleton class

The fifth paper proves that, after the paid deletion of the coordinates of high reuse, the cone relations of every linear-time schedule with a genie of fixed full depth satisfy the paired-distance condition [10, Theorem 3.3 and Corollaries 3.7 and 3.8], and that the argument applies to a single frozen trace [10, Remark 3.9]. The relation N_Γ of a skeleton class is not the cone relation of one trace, so we prove the steps of that argument for it. There are three changes: the relaxation carries $2w-1$ heads on each tape around the heads of one member of the class, the occupancy bound becomes wB (the fifth paper has $B+1$), and the initial potential becomes $m\log_2 3$ instead of m. In addition, no realization needs to be reversed (Lemma 4.10).

We use the geometric relaxation of the fifth paper [10, Section 3.2], with several heads allowed on one tape. There are finitely many tapes with cells indexed by $\mathbb{Z}$, among them an *input tape* and an *output tape*; K unit-speed heads, distributed over the tapes in any way; and ρ registers, which have no addresses. The *slots* are the cells and the registers. A *tick* first transfers labels arbitrarily among the cells under heads and the registers, leaving all other slots unchanged, and then moves every head by at most one cell; moving a head does not move labels. Every label follows one trajectory through the slots; trajectories may share a slot and separate later. The *occupancy* of a slot is the number of labels in it; occupancy bounds are required at every *placement*, that is, before and after the transfers of every tick. The fifth paper uses one head per tape, all starting at cell 0.

Lemma 4.8 (Transport with several heads per tape)**.** Lean: `MergeDepth.transport_tpos`, `MergeDepth.transport`, `MergeDepth.transport_sharp` [`MergeDepth/Transport.lean`]

Let $d \ge 2$ and $b \ge 1$ be integers, $L = d^2 b$ and $W = db$, let $\omega \ge 0$ be an integer with $2\omega + 1 \le W$, and let $S \subseteq [0, L)$ have $m > 0$ elements. In the relaxation with K heads and ρ registers, suppose that trajectories start at the distinct input cells $p \in S$, with no labels elsewhere, and end at the distinct output cells $\mathrm{tpos}(p)$; that every head of the input tape starts in $[-\omega, \omega]$; that every slot has occupancy at most κ at every placement; and that U ticks are used. Then

$$m \log_2 d \le \kappa K(K+\rho)U + 3L.$$

Outline of the formal proof. We follow the proof of the fifth paper's Lemma 3.4 [10]. Fix the trajectories. For $\theta \in \{0, \dots, W-1\}$ partition every tape into the *bins* $\{z : \lfloor (z-\theta)/W \rfloor = i\}$, $i \in \mathbb{Z}$, the tape being part of the name of a bin. At every placement the *active class* is the union of the registers and of the bins that contain at least one head, and every other bin is a class of its own. The active class has at most $KW + \rho$ slots, hence at most $\kappa(KW+\rho)$ labels. Let p^* be uniform in S, let $A = \lfloor p^*/W \rfloor$ be its input row, let Z_θ be the class that contains the label of p^* at a given placement, and put $\Phi = m\,\mathbb{E}_\theta \mathrm{H}(A \mid Z_\theta)$. For random variables Z, Z' with finitely many values, $\mathrm{H}(A \mid Z') - \mathrm{H}(A \mid Z) \le \mathrm{H}(Z \mid Z')$, because $\mathrm{H}(A \mid Z') \le \mathrm{H}(A, Z \mid Z') \le \mathrm{H}(Z \mid Z') + \mathrm{H}(A \mid Z)$ [10, (3.6)].

Transfers. Both ends of every transfer are head cells or registers, hence in the active class, so no label changes class and Φ does not change.

Moves. Process the moves of a tick one head at a time. A unit move of a head from z to $z \pm 1$ changes the bin of that head for exactly one θ; for every other θ the classes are unchanged. For that θ, the entered bin joins the active class if it is not already in it, the left bin leaves it if no other head is in it, and all other classes are unchanged. So every label of the new active class had one of two old class names, the old active class or the entered bin, and every label of a released bin had the old active name. Hence $\mathrm{H}(Z_{\mathrm{old}} \mid Z_{\mathrm{new}})$ is at most the probability that Z_{new} is the active class, which is at most $\kappa(KW+\rho)/m$, and $m\,\mathrm{H}(A \mid Z_{\mathrm{new}}) - m\,\mathrm{H}(A \mid Z_{\mathrm{old}}) \le \kappa(KW+\rho)$. Averaged over θ, the move raises Φ by at most $\kappa(KW+\rho)/W \le \kappa(K+\rho)$. A tick has at most K unit moves, so over the U ticks Φ grows by at most $\kappa K(K+\rho)U$.

Start. All labels lie in their input cells. The heads of the input tape lie in $[-\omega, \omega]$, at most W consecutive cells, which meet at most two adjacent bins; their union, $2W$ consecutive cells, meets at most three of the rows $\{iW, \dots, iW + W - 1\}$. A bin that is not active meets at most two rows, and the active bins of other tapes and the registers hold no labels. So for every θ, given the class, A takes at most three values, and initially $\Phi \le m \log_2 3$.

End. The label of p sits in output cell $\mathrm{tpos}(p)$. For $p = (di + j)b + s$ the row $\lfloor p/W \rfloor$ is i, and $\mathrm{tpos}(p) = (dj+i)b+s$ gives $i = \lfloor (\mathrm{tpos}(p) \bmod W)/b \rfloor$: the row is a W-periodic function of the output cell that takes every value on exactly b cells of a period. So a bin of the output tape that holds n' labels has row multiplicities $n'_a \le b$, and $n'\,\mathrm{H}(A \mid \text{this bin}) = n' \log_2 n' - \sum_a n'_a \log_2 n'_a \ge n' \log_2 n' - n' \log_2 b$. At most $L/W + 1 = d + 1$ bins meet $[0, L)$, so by the convexity of $y \log_2 y$, if every bin is a class of its own, then $m\,\mathrm{H}(A \mid Z_\theta) \ge m \log_2(m/(b(L/W+1))) = m \log_2 d - m \log_2((L+W)/m)$. The actual partition merges the active bins and the registers into one class; this coarsening cannot lower the conditional entropy. So finally $\Phi \ge m \log_2 d - m \log_2((L+W)/m)$.

Combination. Hence $m \log_2 d \le \kappa K(K+\rho)U + m \log_2 3 + m \log_2((L+W)/m)$. Since $d \ge 2$, we have $L + W \le \frac{3}{2}L$, and $y \log_2(a/y) \le a/(e \ln 2)$ for $0 < y \le a$; so $m \log_2((L+W)/m) \le 3L/(2e \ln 2) < 0.8L$, while $m \log_2 3 < 1.59L$ because $m \le L$. This gives the claim.

Formally, runs of the relaxation are `MergeDepth.IsRun`, and the potential is the sum over θ of counting conditional entropies; its bound per tick holds in both directions of time (`MergeDepth.pot_run`), which Lemma 4.10 uses. `MergeDepth.transport` needs only that the heads of the input tape start in W consecutive cells and that the labels end in distinct cells $z \in [0, L)$ of the output tape with $\lfloor (z \bmod W)/b \rfloor$ equal to the row of their source; `MergeDepth.transport_sharp` is the bound before the last step, $m \log_2 d \le m \log_2 3 + m \log_2((L+W)/m) + UK\kappa(K + \rho/W)$, for every $d \ge 1$. □

Lemma 4.9 (Realizing a matching of incidences)**.** `Lean`: `MergeDepth.realizes_cone` [`MergeDepth/SkeletonPaired.lean`]

Let σ, Γ, D, B and N_Γ be as in Lemma 4.7, write $T = T_\sigma$, and let E be the incidence graph of N_Γ: it has a vertex L_p and a vertex R_p for every $p \in [0, L)$, and an edge L_pR_q exactly when $p \in N_\Gamma(q)$ [10, Definitions 2.3 and 3.2]. Fix $X^* \in \mathcal{X}_\sigma$, with head walks $z_j(t) = x_j(t; X^*)$ for $0 \le j \le k$ and output count $z_{\mathrm{out}}(t) = n(t; X^*)$, $0 \le t \le T$. Consider the relaxation with an input tape, k work tapes and an output tape, each carrying $2w - 1$ heads, which at tick t occupy the cells $z(t) + i$, $|i| \le w - 1$, where z is the tape's reference walk z_0, z_j or z_{out}; and with one register. So $K = K_w := (2w-1)(k+2)$, $\rho = 1$, every head starts in $[-(w-1), w-1]$, and the head configurations at all ticks are fixed in advance. Let $\mathcal{M}$ be a matching in E. Then there is a realization in T ticks that carries, for every edge $L_pR_q \in \mathcal{M}$, one label from input cell p to output cell $\mathrm{tpos}(q)$, with occupancy at most wB.

Outline of the formal proof. We follow the proof of the fifth paper's Lemma 3.5 [10].

Coverage. Since $X^* \in \mathcal{X}_\sigma$, we have $z_j(t) \in \mathcal{B}_{s_j(t)}$ for $0 \le j \le k$ and $z_{\mathrm{out}}(t) \in \mathcal{B}_{s_{\mathrm{out}}(t)}$ for $t < T$. The $2w - 1$ cells $z(t) + i$, $|i| \le w - 1$, around a cell of a block contain the whole block. So at tick t every cell of $\mathcal{B}_{s_j(t)}$ on tape j, for $0 \le j \le k$, and every cell of $\mathcal{B}_{s_{\mathrm{out}}(t)}$ on the output tape lies under a head.

Paths and slots. For every edge $L_pR_q \in \mathcal{M}$ we have $p \in N_\Gamma(q) = J_\Gamma(\hat{t}_{\mathrm{tpos}(q)})$; choose a Γ-free path $\hat{t}_{\mathrm{tpos}(q)} = t_0 \to t_1 \to \cdots \to t_\ell = u$ in $\bar{H}_\sigma$ with $p \in \mathcal{B}^{\mathrm{in}}_{s_0(u)}$, so $\ell \le D$. Assign a slot to every edge of $\bar{H}_\sigma$: to a state edge the register, and to an edge $t \to t'$ of length at least two the *base cell* $ws_j(t) = ws_j(t')$ of tape j, for a fixed j with $t' = \mathrm{prev}^\sigma_j(t)$. By coverage, the slot of an edge $t \to t'$ is the register or a head cell at the ticks t' and t.

The realization. Each label moves along its path read forwards in time. The label of L_pR_q stays in input cell p until tick u. At tick t_i, before the moves, it is transferred from the slot by which

it arrives (input cell p if $i = \ell$, and the slot of the edge $t_i \to t_{i+1}$ otherwise) to the slot by which it leaves (the slot of the edge $t_{i-1} \to t_i$ if $i \geq 1$, and output cell $\mathrm{tpos}(q)$ if $i = 0$). All these slots are the register or head cells at tick t_i: input cell p lies in $\mathcal{B}^{\mathrm{in}}_{s_0(u)}$; output cell $\mathrm{tpos}(q)$ lies in the block $\mathcal{B}_{s_{\mathrm{out}}(\hat{t}_{\mathrm{tpos}(q)})}$ by the definition of $\hat{t}_{\mathrm{tpos}(q)}$; and edge slots are covered as said. Between the ticks t' and t of an edge of length at least two, its labels wait in its base cell $ws_j(t)$. That cell may lie under a head at the ticks strictly between, because the heads reach into neighbouring blocks, but no transfer touches it then: a transfer at tick τ uses, on tape j, only the base cell $ws_j(\tau)$, and $s_j(\tau) \neq s_j(t)$ for $t' < \tau < t$. All transfers of a tick precede its moves, so T ticks suffice.

Occupancy. A chosen path through a vertex t starts at a time $\hat{t}_r$ from which t can be reached by a path with at most D edges. Since in-degrees are at most $k+1$ (Lemma 4.5(a)), there are at most B such times; at most w positions r share each of them (Lemma 4.5(c)); and every R_q, hence every $r = \mathrm{tpos}(q)$, lies on at most one edge of $\mathcal{M}$. So at most wB chosen paths pass through t. Before the transfers of tick τ the register holds only labels of the edge $\tau \to \tau - 1$, and after them only labels of the edge $\tau + 1 \to \tau$; all these paths pass through one vertex, so there are at most wB such labels. Before the transfers of tick τ, the base cell of block a on tape j holds only labels of an edge of tape j that comes from a long hop of s_j at a spanning τ, and after them only labels of such an edge from a hop spanning $\tau + 1$; by property (P1) of the walk s_j [3, Section 2.2], at most one hop of s_j at a spans a given time, so these labels belong to paths through one vertex, and there are at most wB of them. An input cell holds at most its own label, and an output cell at most the label that ends there, because $\mathcal{M}$ is a matching. Input and output cells are never edge slots, so the occupancy is at most wB at every placement.

Formally, the relation is `MergeDepth.coneRel`, defined from the reference walks alone: $p \in N(q)$ if a Γ-free path of the dependency DAG of the coarse work walks from the time $o(\mathrm{tpos}(q))$ ends at a time u with $\lfloor p/w \rfloor = \lfloor z_0(u)/w \rfloor$, where the output time $o(r)$ may be any time before T at which the output count lies in the block of r. With $o(r) = \hat{t}_r$ this relation contains N_Γ (`MergeDepth.classRel_subset_coneRel`), and realizations pass to subrelations. `MergeDepth.realize_paths` realizes every family of paths of the block DAG along which the slots are as above, and `MergeDepth.realizes_cone` applies it to all matchings at once. □

Lemma 4.10 (Disjoint short paired paths)**.** Lean: `MergeDepth.skel_disjoint_paths`, `MergeDepth.skel_disjoint_paths_transport` [`MergeDepth/SkeletonPaired.lean`]

In the setting of Lemma 4.9, suppose that $L = d^2 b$ with $d \geq 2$ and $b \geq 1$, that $db \geq 2w - 1$, and that $T \leq c(L+1)$. Let $s \geq 1$ be odd. If E contains m pairwise vertex-disjoint paths with s edges each, every one of which joins L_p to R_p for some p, then

$$m \leq A_s^{(w)} \frac{L}{\log_2 d}, \qquad A_s^{(w)} = 2scwB\,K_w(K_w+1) + 3.$$

Outline of the formal proof. We follow the proof of the fifth paper's Lemma 3.6 [10], with one simplification: the bounds on the potential in the proof of Lemma 4.8 hold in both directions of time, so no realization needs to be reversed. List the vertices of each path from its left end: $L_{y_0}, R_{y_1}, L_{y_2}, \ldots, R_{y_s}$, with $y_0 = y_s = p$. Since the paths are vertex-disjoint, for each $i < s$ their ith edges form a matching $\mathcal{M}_i$ of E, consisting of the edges $L_{y_i} R_{y_{i+1}}$ if i is even and $L_{y_{i+1}} R_{y_i}$ if i is odd. Lemma 4.9 gives for each $\mathcal{M}_i$ a realization in T ticks from the initial head configuration to the final one, with occupancy at most wB. For even i it carries the label of each path from input cell y_i to output cell $\mathrm{tpos}(y_{i+1})$; for odd i, read backwards, it carries it from output cell $\mathrm{tpos}(y_i)$ to input cell y_{i+1}. At every junction the labels occupy the same cells and the heads the same configuration, so the potential of the end of one stage is that of the start of the next. Each stage changes the potential by at most $\kappa K(K+\rho)T$, forwards or backwards, so the start and end bounds

of the proof of Lemma 4.8 give, with $K = K_w$, $\rho = 1$, $\kappa = wB$, $\omega = w - 1$, $W = db \geq 2w - 1$ and $T \leq c(L + 1) \leq 2cL$,

$$m \log_2 d \leq wB\, K_w(K_w + 1) \cdot 2scL + 3L,$$

which is the claim. Formally, `MergeDepth.disjoint_paths_bound` is this argument for any relation with such realizations and any walks of length s, not necessarily simple, with pairwise disjoint vertex sets; `MergeDepth.skel_disjoint_paths` and `MergeDepth.skel_disjoint_paths_transport` are its forms for the relation `MergeDepth.coneRel`. □

Proposition 4.11 (Paired distances on a skeleton class)**.** Lean: `MergeDepth.skel_paired_packing`, `MergeDepth.skel_self_demands` [`MergeDepth/SkeletonPaired.lean`]

Let $k, w \geq 1$ and $D \geq 0$ be integers and $c > 0$. Let σ be a skeleton at scale w of a machine with k work tapes at a length $L = d^2 b(d)$ with $d \geq 2$ and $db(d) \geq 2w - 1$, such that $\mathcal{X}_\sigma \neq \emptyset$, $T_\sigma \leq c(L + 1)$ and every member appends exactly L bits; let Γ have full depth at most D in $\bar{H}_\sigma$, let E be the incidence graph of N_Γ, and for $Q \subseteq [0, L)$ let E_Q be its induced subgraph on $\{L_p, R_p : p \in Q\}$ [10, Definition 3.2]. With B and $A_s^{(w)}$ as above:

(a) For all integers $r \geq 1$ and $\Delta \geq 0$ and every $Q \subseteq [0, L)$ such that E_Q has maximum degree at most Δ,

$$|\{p \in Q : \mathrm{dist}_{E_Q}(L_p, R_p) \leq r\}| \leq \sum_{\substack{1 \leq s \leq r \\ s \text{ odd}}} (s + 1)\, \nu_r\, A_s^{(w)}\, \frac{L}{\log_2 d}, \qquad \nu_r = \sum_{j=0}^{r} \max(1, \Delta)^j.$$

(b) The set $S_\Gamma = \{p : p \in N_\Gamma(p)\}$ of self-demands has at most $A_1^{(w)} L / \log_2 d$ elements.

(c) For every integer $\vartheta \geq 1$, the set $\mathcal{H}_\vartheta = \{p : \deg_E(L_p) > \vartheta\}$ has at most wBL/ϑ elements, and $E_{[0,L) \setminus \mathcal{H}_\vartheta}$ has maximum degree at most $\max(wB, \vartheta)$.

The bounds depend only on k, D, c, w, Δ and r; no correctness and no bound on $|\Gamma|$ is assumed.

Outline of the formal proof. (a) This is the packing argument of the fifth paper's proof of its Theorem 3.3 [10]. For each $p \in Q$ with $\mathrm{dist}_{E_Q}(L_p, R_p) \leq r$ choose a shortest path from L_p to R_p in E_Q; it has an odd number $s \leq r$ of edges, because every edge of E joins an L-vertex to an R-vertex. A vertex y lies on at most ν_r chosen paths: their left ends are distinct and lie within distance r of y in a graph of maximum degree at most Δ. For each odd $s \leq r$ choose a maximal vertex-disjoint family of chosen paths with s edges. Every chosen path with s edges meets a member of the family, and each member, having $s + 1$ vertices, meets at most $(s + 1)\nu_r$ chosen paths. The members are paths in E, so by Lemma 4.10 there are at most $A_s^{(w)} L / \log_2 d$ of them. Summing over s gives (a).

(b) The edges $L_p R_p$, $p \in S_\Gamma$, are pairwise vertex-disjoint paths with one edge each; apply Lemma 4.10 with $s = 1$.

(c) Counting edges, $\sum_p \deg_E(L_p) = \sum_q |N_\Gamma(q)| \leq wBL$ by Lemma 4.7(a), so at most wBL/ϑ coordinates have $\deg_E(L_p) > \vartheta$. Deleting both ports of these coordinates does not raise any degree; the remaining left degrees are at most ϑ, and the right degrees $\deg_E(R_q) = |N_\Gamma(q)|$ are at most wB.

Formally, (a) and (b) are stated for the relation `MergeDepth.coneRel` of the reference walks (Lemma 4.9); for every relation inside it, such as N_Γ, they are `MergeDepth.subrel_paired_packing` and `MergeDepth.subrel_self_demands`. Part (a) applies the fifth paper's `DirectedRecovery.short_paired_count_log_bound`, which is Lean-verified there, to the induced relation, with the transport bound `MergeDepth.disjoint_paths_transport`; (b) is

the fifth paper's `DirectedRecovery.self_demands_count_of_disjoint_transport`; and (c) is `MergeDepth.skel_trimming`, or `MergeDepth.paid_trimming` for any relation with $|N(q)| \le wB$. Along any sequence of such data with $d \to \infty$ and bounded incidence degree the relations satisfy the fifth paper's paired-distance condition (`MergeDepth.skel_paired_separation`). □

4.5 Theorem SE

The *induced relation* of a relation N on $Q \subseteq [0, L)$ is $N_Q(q) = N(q) \cap Q$ for $q \in Q$ [10, Section 2].

Lemma 4.12 (A uniform half-rate bound on class relations)**.** Lean: `MergeDepth.skel_uniform_class` [`MergeDepth/SkeletonSE.lean`]

Assume [Q9.1]; binary codes suffice. Let M be a machine with k work tapes, let $w \ge 1$ and $D \ge 0$ be integers, $c > 0$ and $0 < \delta < 1$, and put $B = \sum_{h=0}^{D} (k+1)^h$ and $\vartheta = \lceil wB/\delta \rceil$. For a length $L = d^2 b(d)$ let $\mathcal{N}_L$ be the set of the induced relations $(N_\Gamma)_Q$, on the coordinate sets $Q = [0, L) \setminus (\mathcal{H}_\vartheta \cup S_\Gamma)$, for all skeletons σ of M at scale w at length L such that $\mathcal{X}_\sigma \ne \emptyset$, $T_\sigma \le c(L+1)$ and every member appends exactly L bits, and all $\Gamma \subseteq [0, T_\sigma)$ of full depth at most D in H_σ; here S_Γ and $\mathcal{H}_\vartheta$ are as in Proposition 4.11. Then for every $\zeta > 0$ there is L_ζ, depending only on k, w, D, c, δ and ζ, such that for every length $L \ge L_\zeta$, every relation of $\mathcal{N}_L$, on its coordinate set Q, and every binary storage code $\mathcal{C} \subseteq \{0,1\}^Q$ for it, $\log_2 |\mathcal{C}| \le (\frac{1}{2} + \zeta)|Q|$.

Outline of the formal proof. Suppose not. Then there are $\zeta > 0$ and infinitely many lengths L, each with a machine with k work tapes, a relation N'_L in its set $\mathcal{N}_L$ on a coordinate set Q_L, coming from some σ and Γ, and a binary storage code $\mathcal{C}_L$ for N'_L with $\log_2 |\mathcal{C}_L| > (\frac{1}{2} + \zeta)|Q_L|$. Along these lengths $d \to \infty$, so for all large ones $d \ge 2$ and $db(d) \ge 2w - 1$, and Proposition 4.11 applies. By its parts (b) and (c), $|Q_L| \ge (1 - \delta - A_1^{(w)} / \log_2 d)L$, which tends to infinity. The incidence graph of N'_L is E_{Q_L}, which has maximum degree at most $\max(wB, \vartheta) = \vartheta$ by part (c), since $\vartheta \ge wB/\delta > wB$. For every fixed $r \ge 1$, part (a) with $\Delta = \vartheta$ and $Q = Q_L$ shows that at most $O_r(L/\log_2 d) = o(|Q_L|)$ coordinates $p \in Q_L$ have $\mathrm{dist}_{E_{Q_L}}(L_p, R_p) \le r$; this is the paired-distance condition [10, Definition 2.3]. So the relations N'_L, the degree bound ϑ and the codes $\mathcal{C}_L$ satisfy the hypotheses of Question 9.1 with the binary alphabet, and its affirmative answer gives $\limsup \log_2 |\mathcal{C}_L| / |Q_L| \le \frac{1}{2}$, a contradiction. The relations are loopless, since Q_L avoids S_Γ, so the argument does not depend on whether recovery relations in Question 9.1 may contain loops. This is the uniformity argument of the fifth paper's Proposition 9.2 [10]; the bounds of Proposition 4.11 depend on the machines only through k.

Formally, `MergeDepth.skel_uniform_binary` is this argument, through the uniformity argument `MergeDepth.uniform_rate_binary`, for every relation inside the relation `MergeDepth.coneRel` of Lemma 4.9, with its own heavy coordinates and self-demands removed; `MergeDepth.skel_uniform_class` applies it to N_Γ (`MergeDepth.classRel_subset_coneRel`). The formal statement allows every $\vartheta \ge wB/\delta$ and every $c \ge 0$, and needs only one member that appends at least L bits. □

Theorem 4.13 (Theorem SE: skeleton entropy)**.** Lean: `MergeDepth.thm_SE`, `MergeDepth.thm_SE_class` [`MergeDepth/SkeletonSE.lean`]

Assume [Q9.1]; binary codes suffice. Let $k, V, w \ge 1$ be integers, $c > 0$ and $0 < \eta < \frac{1}{2}$. For every machine M with k work tapes there is L_0 such that at no length $L = d^2 b(d) \ge L_0$ does M have the following three properties:

(i) on every input $X \in \{0,1\}^L$ it halts within $c(L+1)$ steps with output X^{T};

(ii) on every input, every work-tape cell is entered at most V times;

(iii) $\mathrm{H}(\sigma_w(X)) \le (\frac{1}{2} - \eta)L$.

More precisely, at no length $L \ge L_0$ is there a skeleton class $\mathcal{X}_\sigma$ of M at scale w with at least $2^{(\frac{1}{2}+\eta)L}$ elements such that M halts within $c(L+1)$ steps with output X^{T} on every $X \in \mathcal{X}_\sigma$ and has entry at most V on every work tape on some $X \in \mathcal{X}_\sigma$.

Outline of the formal proof. The second statement implies the first: by (2), (iii) gives a class with at least $2^{(\frac{1}{2}+\eta)L}$ elements, (i) holds on it, and (ii) holds on each of its members.

Put $\varepsilon = \delta = \eta/2$. Let β_w be the value width of the block machines of M, let D be given by Lemma 4.6 for k, V, w, c and ε, and let $B = \sum_{h=0}^{D}(k+1)^h$ and $\vartheta = \lceil wB/\delta \rceil$; all depend only on M, V, w, c and η. Let $L_{\eta/8}$ be given by Lemma 4.12 for these D and δ, and let $L_0 \ge L_{\eta/8}$ be such that every length $L \ge L_0$ has $d \ge 2$, $db(d) \ge 2w-1$ and $A_1^{(w)}/(2\log_2 d) < \eta/8$.

Suppose that a class $\mathcal{X}_\sigma$ as described exists at a length $L \ge L_0$. Its members append exactly L bits and halt within $c(L+1)$ steps, so $T_\sigma \le c(L+1)$. By Lemma 4.6 there is a genie Γ of full depth at most D in $\bar{H}_\sigma$ with $\beta_w|\Gamma| \le \varepsilon L$. The revealed word G_Γ takes at most $2^{\varepsilon L}$ values, so some fibre $W = \{X \in \mathcal{X}_\sigma : G_\Gamma(X) = g\}$ has

$$\log_2|W| \ge (\tfrac{1}{2}+\eta)L - \varepsilon L = (\tfrac{1}{2}+\tfrac{\eta}{2})L.$$

By Lemma 4.7(c), W is a binary storage code for N_Γ. Let $Q = [0,L) \setminus (\mathcal{H}_\vartheta \cup S_\Gamma)$. By the fifth paper's paid conditioning [10, Lemma 2.2], which is Lean-verified there, fixing the coordinates in $\mathcal{H}_\vartheta \cup S_\Gamma$ and restricting to Q turns every nonempty fibre of W into a storage code for $(N_\Gamma)_Q \in \mathcal{N}_L$, which has at most $2^{(\frac{1}{2}+\eta/8)|Q|}$ words by Lemma 4.12 with $\zeta = \eta/8$; and there are at most $2^{|\mathcal{H}_\vartheta \cup S_\Gamma|}$ such fibres. By Proposition 4.11(b),(c), $|S_\Gamma| \le A_1^{(w)}L/\log_2 d$ and $|\mathcal{H}_\vartheta| \le wBL/\vartheta \le \delta L = \eta L/2$. Hence

$$\log_2|W| \le |\mathcal{H}_\vartheta \cup S_\Gamma| + (\tfrac{1}{2}+\tfrac{\eta}{8})(L - |\mathcal{H}_\vartheta \cup S_\Gamma|) \le (\tfrac{1}{2}+\tfrac{\eta}{8})L + \tfrac{1}{2}|\mathcal{H}_\vartheta \cup S_\Gamma| \le (\tfrac{1}{2}+\tfrac{3\eta}{8})L + \frac{A_1^{(w)}L}{2\log_2 d},$$

which is less than $(\frac{1}{2}+\frac{\eta}{2})L$, a contradiction.

Formally, `MergeDepth.thm_SE_class` is the second statement and `MergeDepth.thm_SE` the first. The proof uses the genie `MergeDepth.skeleton_genie`, the count `MergeDepth.card_revealed_le` of revealed words, the storage codes `MergeDepth.blockMachine_isStorageCode_fibre`, paid conditioning (`MergeDepth.logb_card_le_of_conditioning`), and `MergeDepth.skel_uniform_binary`, applied to N_Γ inside the relation `MergeDepth.coneRel` of a member (`MergeDepth.classRel_subset_coneRel`). □

The hypotheses enter the proof as follows: (iii) only through the existence of one large class; (ii) only for one member of that class, through its coarse work walks; and (i) for the time bound, for the number of appended bits and for correctness. Given [Q9.1], Theorem SE is a necessary condition for adaptive transposers: a linear-time transposer whose work heads enter every cell boundedly often must, at every fixed scale w and for every $\eta > 0$, have skeleton entropy above $(\frac{1}{2}-\eta)L$ at all large lengths. Its cut is determined by the skeleton alone, so no structure of the individual traces is paid for, and motion inside blocks, however data-dependent, is invisible to it.

Relation to Corollary F″. Theorem SE contains the eighth paper's Corollary F″ [3, Corollary 6.4]. Suppose that M has the three properties of that corollary at a length L, with a constant $\eta' < \frac{1}{2}$ in place of η. Some trace τ_0 has probability at least $2^{-\eta' L}$, and it enters every work-tape cell at most V times. A trace determines the skeleton, so the at least $2^{(1-\eta')L}$ inputs that follow τ_0 lie in one

skeleton class at any scale w. With $\eta = \min(\frac{1}{2} - \eta', \frac{1}{4})$, this class has at least $2^{(\frac{1}{2}+\eta)L}$ elements and contains an input on which M has entry at most V on every work tape, which the second statement of Theorem SE excludes at all large lengths.

4.6 Bounded entry and input steering

Lemma 4.14 (Few coarse walks)**.** Lean: `MergeDepth.card_coarse_le`, `MergeDepth.card_monotone_le` [`MergeDepth/CoarseCount.lean`]

(a) Let $V \geq 1$, $w \geq 4V$ and $T \geq 16V$ be integers. The coarse walks $\lfloor x/w \rfloor$ of all walks x of length T with $x(0) = 0$ that enter every cell at most V times are at most $(ew/V)^{m_1}$ in number, where $m_1 = 2V(T/w + 2)$.

(b) Let $T \geq 2$ and $1 \leq m \leq T - 1$ be integers. The maps $s\colon [0, T) \to \mathbb{Z}$ with $s(0) = 0$, $s(t+1) - s(t) \in \{0, 1\}$ for $t + 1 < T$, and $s(T-1) \leq m$ are at most $(eT/y)^y$ in number, for every real number y with $m \leq y \leq T$.

Outline of the formal proof. We use $\sum_{i=0}^{m} \binom{n}{i} \leq (en/m)^m$ for integers $1 \leq m \leq n$.

(a) By Lemma 4.2(c), $s = \lfloor x/w \rfloor$ changes value at most $m_0 = 2V(\lfloor (T-1)/w \rfloor + 2) - 1$ times, each time by ± 1, at times in $[1, T)$; since $s(0) = 0$, s is determined by the set of these times and the signs of the changes. Since $w \geq 4V$ and $T \geq 16V$, we have $3 \leq 4V - 1 \leq m_0 \leq (T-1)/2 + 4V - 1 \leq T - 1$. So the number of coarse walks is at most

$$\sum_{i=0}^{m_0} \binom{T-1}{i} 2^i \leq 2^{m_0} \Big(\frac{e(T-1)}{m_0}\Big)^{m_0} \leq \Big(\frac{2eT}{m_0}\Big)^{m_0}.$$

The function $y \mapsto y \ln(2eT/y)$ increases for $0 < y < 2T$, its derivative being $\ln(2T/y)$, and $m_0 < m_1 \leq T/2 + 4V \leq 3T/4$, because $w \geq 4V$ and $T \geq 16V$. So the count is at most $(2eT/m_1)^{m_1} \leq (ew/V)^{m_1}$, because $m_1 \geq 2VT/w$.

(b) Such a map is determined by the set of times $t \in [1, T)$ with $s(t) \neq s(t-1)$, which has at most m elements. So there are at most $\sum_{i \leq m} \binom{T-1}{i} \leq (e(T-1)/m)^m \leq (eT/m)^m$ of them, and $y \mapsto y \ln(eT/y)$ increases on $(0, T]$, its derivative being $\ln(T/y)$.

Formally, the encoding by change times and up-steps is `MergeDepth.card_unit_le`, the binomial estimate is `MergeDepth.sum_choose_le`, and the monotonicity of $y \mapsto (Ae/y)^y$ on $(0, A]$ is `MergeDepth.rpow_mono_of_le`. □

A head that enters every cell at most V times has a coarse walk that carries only $O(V \log(w/V)/w)$ bits per step; so at large scales the skeleton of a machine with bounded entry is almost determined by its length, and only heads that revisit cells unboundedly often can carry a constant fraction of the input in their coarse motion.

Lemma 4.15 (Coarse entropy of bounded-entry heads)**.** Lean: `MergeDepth.coarse_entropy_le`, `MergeDepth.theta_tendsto` [`MergeDepth/SkeletonEntropy.lean`]

Let M have k work tapes, let $V \geq 1$ and $w \geq 4V$ be integers, $c > 0$, and let $L \geq \max(2w, 16V)$ be a length. Suppose that on every input $X \in \{0,1\}^L$, M appends exactly L bits, halts within $c(L+1)$ steps and has entry at most V on every work tape. Then

$$\mathrm{H}(\sigma_w(X)) \leq \mathrm{H}(\sigma_w^{\mathrm{in}}(X)) + \theta_{k,V,c}(w)\, L + 4kV \log_2 \frac{ew}{V},$$

where $\theta_{k,V,c}(w) = (4kVc \log_2(ew/V) + \log_2(2ecw))/w$. For fixed k, V and c, $\theta_{k,V,c}(w) \to 0$ as $w \to \infty$.

Outline of the formal proof. The skeleton $\sigma_w(X)$ is determined by the input skeleton, which fixes $T = T(X)$, together with the k coarse work walks and the output component. So $\mathrm{H}(\sigma_w(X)) \le \mathrm{H}(\sigma_w^{\mathrm{in}}(X)) + \log_2 N$, where N is the largest number, over the values of T, of possible values of these $k+1$ components given T. Since L bits are appended at distinct steps, $L \le T \le c(L+1) \le 2cL$; in particular $T \ge 16V$. Each coarse work walk is the coarse walk of a walk of length T that starts at 0 and enters every cell at most V times, so by Lemma 4.14(a) it takes at most $(ew/V)^{m_1}$ values, with $m_1 = 2V(T/w+2) \le 4VcL/w + 4V$. The output component starts at 0, grows by 0 or 1 per step, and ends at most at $m = \lfloor L/w \rfloor$, where $1 \le m \le L/w \le T-1$ because $L \ge 2w$ and $w \ge 4$; by Lemma 4.14(b) with $y = L/w$ it takes at most $(eTw/L)^{L/w} \le (2ecw)^{L/w}$ values. Hence $\log_2 N \le k(4VcL/w + 4V)\log_2(ew/V) + (L/w)\log_2(2ecw)$, which is the claim. The limit holds because $(\log_2 w)/w \to 0$.

Formally, the statement concerns abstract run data, the halting time, the head walks and the output count on every input, with $L \le T(X) \le c(L+1)$ and $n(t) \le L$ for $t < T(X)$ in place of the number of appended bits; the skeleton and the input skeleton are `MergeDepth.skelData` and `MergeDepth.inSkel`, entropies are those of functions of a uniform input (`MergeDepth.entropy`), the first inequality is `MergeDepth.entropy_le_of_fibres`, and the limit holds for real w. For a machine that, on every input, halts within $c(L+1)$ steps with exactly L output bits, `MergeDepth.frozen_coarse_entropy_le_of_halts` supplies these data from its runs, and `MergeDepth.frozen_coarse_entropy_le` is the special case of a transposer. □

The output component alone contributes at most $(L/w)\log_2(2ecw)$ bits, whatever the machine does, as long as it appends exactly L bits within $c(L+1)$ steps; it could therefore be omitted from the skeleton, at the price of passing to a coarser scale and a smaller η in Theorem SE.

Corollary 4.16 (Corollary IS: input steering)**.** Lean: `MergeDepth.cor_IS` [`MergeDepth/InputSteering.lean`]

Assume [Q9.1]; binary codes suffice. Let $k, V, w \ge 1$ be integers, $c > 0$ and $0 < \eta < \frac{1}{2}$. For every machine M with k work tapes there is L_0 such that at no length $L = d^2 b(d) \ge L_0$ does M have the following three properties:

(i) on every input $X \in \{0,1\}^L$ it halts within $c(L+1)$ steps with output X^{T};

(ii) on every input, every work-tape cell is entered at most V times;

(iii) $\mathrm{H}(\sigma_w^{\mathrm{in}}(X)) \le (\frac{1}{2} - \eta)L$.

Outline of the formal proof. Choose an integer $m \ge 1$ such that $w' = mw$ satisfies $w' \ge 4V$ and $\theta_{k,V,c}(w') \le \eta/2$, which is possible by Lemma 4.15. Let L_0 be at least the length that Theorem SE gives for M, V, w', c and $\eta/4$, and such that every length $L \ge L_0$ satisfies $L \ge \max(2w', 16V)$ and $4kV\log_2(ew'/V) \le \eta L/4$. Suppose that M has (i)–(iii) at a length $L \ge L_0$. Since $\lfloor z/(mw) \rfloor = \lfloor \lfloor z/w \rfloor / m \rfloor$ for every integer z, the input skeleton at scale w' is a function of that at scale w, so $\mathrm{H}(\sigma_{w'}^{\mathrm{in}}(X)) \le (\frac{1}{2} - \eta)L$. By (i), (ii) and Lemma 4.15 at scale w',

$$\mathrm{H}(\sigma_{w'}(X)) \le (\tfrac{1}{2} - \eta)L + \tfrac{\eta}{2}L + \tfrac{\eta}{4}L = (\tfrac{1}{2} - \tfrac{\eta}{4})L.$$

So M has the three properties of Theorem SE at scale w' with $\eta/4$ in place of η, which is impossible. Formally, `MergeDepth.cor_IS` takes $w' = wm$ with $\theta_{k,V,c}(w') < \eta/2$ (`MergeDepth.theta_tendsto`), compares the input skeletons at the scales w and w' by data processing (`MergeDepth.entropy_comp_le`), and applies `MergeDepth.frozen_coarse_entropy_le` and `MergeDepth.thm_SE`. □

Corollary IS reduces adaptivity, for machines whose work tapes have bounded entry, to the coarse motion of one head: given [Q9.1], such a linear-time transposer must move its input head, at every fixed scale, in a way that depends on the data and carries about half of the input. For a construction this is a requirement; for a proof of ¬(T) it leaves, among such machines, only those whose input head is steered in this way. For an oblivious machine the input skeleton is constant, so Corollary IS contains the eighth paper's Corollary F′ [3, Corollary 6.3] and extends it to adaptive machines.

Corollary 4.17 (Corollary RV: revisits)**.** `Lean`: `MergeDepth.cor_RV` [`MergeDepth/InputSteering.lean`]

Assume [Q9.1]; binary codes suffice. For all integers $k, V \geq 1$ and every $c > 0$, no machine with k work tapes is, at infinitely many lengths $L = d^2 b(d)$, a transposer of all $d \times d$ matrices within $c(L+1)$ steps such that on every input every cell of every tape, the input tape included, is entered at most V times.

Outline of the formal proof. Let M be such a machine. Since $(\log_2 w)/w \to 0$, there is an integer $w \geq 4V$ with $4Vc\log_2(ew/V) \leq w/8$; fix it. At a length $L \geq 16V$ at which M has the stated properties, every input satisfies $L \leq T(X) \leq c(L+1)$, and the input skeleton at scale w is the coarse walk of a walk of length $T(X)$ that starts at 0 and enters every cell at most V times. By Lemma 4.14(a), for each value of $T(X)$ it takes at most $(ew/V)^{m_1}$ values, where

$$m_1 = 2V\big(T(X)/w + 2\big) \leq 2V(2cL/w + 2),$$

and $T(X)$ takes at most $c(L+1)+1$ values. Hence

$$\mathrm{H}(\sigma_w^{\mathrm{in}}(X)) \leq \frac{L}{8} + 4V\log_2\frac{ew}{V} + \log_2(c(L+1)+1) \leq \frac{L}{4}$$

at all large such lengths. The work tapes satisfy (ii) of Corollary IS, so Corollary IS with this w and $\eta = \frac{1}{4}$ excludes M at all large lengths. Formally, `MergeDepth.cor_RV` bounds the entropy of the input skeleton by counting coarse walks (`MergeDepth.inSkel_entropy_le`) and applies `MergeDepth.cor_IS` with $\eta = \frac{1}{4}$. □

Given [Q9.1], every linear-time transposer, adaptive or not, revisits cells unboundedly often: for every V, at all large lengths, some input makes some head enter some cell more than V times. Corollary RV is incomparable with the eighth paper's Corollary F′: it needs bounded entry on the input tape, which F′ does not, and no obliviousness, which F′ needs; the extension of F′ to adaptive machines is Corollary IS. Since bounded entry allows unboundedly many reversals [3, Remark 6.2], Corollary RV does not follow from reversal or pass bounds. It is specific to transposition, which enters through the row potential of Lemma 4.8: copying the input to the output takes linear time and enters every cell of every tape at most once.

Update (November 2026). [Q9.1] is false, also for binary codes [8, Theorem A and Corollary A′], so Lemma 4.12, Theorem SE and Corollaries IS and RV are vacuous as stated, and so are their Lean forms, which assume `MergeDepth.Q91Binary`. Theorem SE uses [Q9.1] only through Lemma 4.12, and Corollaries IS and RV use it only through Theorem SE. So all three hold with the conclusion of Lemma 4.12, a half-rate bound for binary storage codes on the relations of skeleton classes, as hypothesis in place of [Q9.1]. That native bound is open, since the thirteenth paper's relations are not known to arise from tape machines, and the requirements that this section states for constructions hold only under it. For Corollary RV a deficit bound, rate at most $1 - \rho(\Delta) + o(1)$ with ρ decaying slowly enough, would suffice in place of rate $\frac{1}{2}$ (Remark 4.19, a proof sketch only); the thirteenth paper does not decide such bounds either, since its codes are not shown to have rate near one.

Remark 4.18 (Padding)**.** (a) Padding of bounded entry before the run affects neither Theorem SE nor Corollary IS at large scales. Let $c_0, V_0 \geq 1$ be integers, and let M' first move its input head along a walk that starts and ends at cell 0, has length between L and $c_0 L$ and enters every cell at most V_0 times, chosen in any way as a function of the input, and then run M from its initial configuration. The padding of the eighth paper [3, Section 6.2], a sweep across the input with a step out and back at every cell of the first half that holds a 1, followed by a return to cell 0, is of this kind with $c_0 = 3$ and $V_0 = 4$. The prefix moves neither the work heads nor the output count, so M' inherits (ii) of Theorem SE and of Corollary IS from M, and it satisfies (i) with $c + c_0$ in place of c. At a scale w' the skeleton of M' is the coarse walk of the prefix, with the work heads and the output count in block 0, followed by the skeleton $\sigma_{w'}(X; M)$ of M, and likewise for the input skeletons. So, by Lemma 4.14(a) applied to each length of the prefix, for $w' \geq 4V_0$ and $L \geq 16V_0$,

$$\mathrm{H}(\sigma_{w'}(X; M')) \leq \mathrm{H}(\sigma_{w'}(X; M)) + 2V_0\Big(\frac{c_0 L}{w'} + 2\Big) \log_2 \frac{ew'}{V_0} + \log_2(c_0 L + 1),$$

and the same bound holds for the input skeletons. If M satisfies (iii) of Theorem SE or of Corollary IS at a scale w with η, then for every multiple $w' \geq 4V_0$ of w with $2V_0 c_0 \log_2(ew'/V_0)/w' \leq \eta/2$, the machine M' satisfies the same condition at w' with $\eta/4$ at all large lengths, because skeletons and input skeletons at scale w' are functions of those at scale w. So Theorem SE and Corollary IS exclude M' whenever their hypotheses (ii) and (iii) hold for M.

(b) Padding in which the input head revisits cells unboundedly often can break hypothesis (iii) of Theorem SE and of Corollary IS at every scale. Before running M, a machine may copy its input to an extra work tape and then, reading the copy, step its input head from cell 0 to cell -1 and back at every 1 and pause at every 0. This costs $O(L)$ steps and keeps the entries on the work tapes bounded, and $\sigma^{\mathrm{in}}_w(X)$ then determines X for every w; but the input head enters the cells 0 and -1 once for each 1 of the input. By (a), padding before the run needs such revisits to have this effect at large scales. So Theorem SE and Corollary IS are necessary conditions that padding of this kind meets at no cost to a construction. Corollary RV, whose hypotheses are entry bounds and involve no entropy, is not affected by padding: padding adds entries and removes none.

Remark 4.19 (A weak coding bound suffices for Corollary RV)**.** Proof sketch only: Corollary RV does not need rate $\frac{1}{2}$. Under its hypotheses all $k+1$ heads have bounded entry, so Lemma 4.15, together with the bound on the input skeleton in the proof of Corollary RV, gives $\mathrm{H}(\sigma_w(X)) \leq \theta'(w)L + O(\log L)$ with $\theta'(w) = (4(k+1)Vc\log_2(ew/V) + \log_2(2ecw))/w$, and the largest skeleton class has at least $2^{(1-\theta'(w))L - O(\log L)}$ elements. Suppose that along every sequence of relations of incidence degree at most Δ that satisfies the paired-distance condition, all binary storage codes have rate at most $1 - \rho(\Delta) + o(1)$. The proof of Theorem SE, run on that class with $\delta = \varepsilon$, then gives a contradiction as soon as $\rho(\Delta(w, \varepsilon)) > \theta'(w) + 2\varepsilon$ for some $w \geq 4V$ and $\varepsilon \in (0, 1)$, where $\Delta(w, \varepsilon) = \lceil wB/\varepsilon \rceil$ is the trimming threshold and B the cone bound at the depth of Lemma 4.6 for ε; here $\log\log \Delta(w, \varepsilon)$ is of order $k^3 V^3 \beta_w C/\varepsilon$, and β_w is of order $kw \log|\Sigma|$ for large w. Compare [3, Remark 6.5].

5 Localization

The two preceding sections combine into one statement about where a linear-time transposer could still live, given an affirmative answer to Question 9.1. It is the paper's main conditional conclusion: every transposer must revisit some tape unboundedly often; an oblivious one must revisit a work tape and merge deep lineages unboundedly often in its code; and one whose work tapes have bounded

entry must revisit input cells and carry about half of the input in the coarse motion of its input head.

Theorem 5.1 (Theorem L: localization)**.** Lean: `MergeDepth.thm_L_a`, `MergeDepth.thm_L_b`, `MergeDepth.thm_L_c` [`MergeDepth/Localization.lean`]

Assume [Q9.1]. Let M be a machine with k work tapes, let $c > 0$, and let $\mathcal{D}$ be an infinite set of dimensions d at which M transposes all $d \times d$ matrices with $b(d)$-bit entries within $c(L+1)$ steps, $L = d^2 b(d)$. Then the following hold at all large $d \in \mathcal{D}$, in each part from a threshold that depends only on M, c and the parameters named there.

(a) (Revisits.) *For every integer $V \geq 1$: on some input, some cell of some tape, the input tape or a work tape, is entered more than V times.*

(b) (Input steering.) *For all integers $V, w \geq 1$ and every $\eta \in (0, \frac{1}{2})$: if on every input every work-tape cell is entered at most V times, then $\mathrm{H}(\sigma_w^{\mathrm{in}}(X)) > (\frac{1}{2} - \eta)L$ for a uniformly random input X.*

(c) (Merge depth.) *For all integers $V \geq 1$ and $s \geq 0$ and all $\varepsilon \in (0, 1]$ and $\eta \geq 0$ with $\varepsilon + \eta < \frac{1}{2}$: if M is oblivious at length L, then for every pruning of its field DAG, every cold genie at (V, ε), taken with $\beta = \beta(M)$ and $C = \max(1, c)$, and every lineage assignment for it, more than ηL output positions have merge depth greater than s.*

Outline of the formal proof. Part (a) is Corollary 4.17 for the entry bound V: the dimensions in $\mathcal{D}$ at which every input enters every cell at most V times are finitely many. Part (b) is Corollary 4.16 for k, V, w, c and η: at all large lengths, its properties (i) and (ii) exclude (iii), and (i) holds at every $d \in \mathcal{D}$. Part (c) is Corollary 3.16 for k, V, s, $C = \max(1, c)$, ε and η: at all large d, its properties (i) and (ii) exclude each other. Formally, `MergeDepth.thm_L_a`, `MergeDepth.thm_L_b` and `MergeDepth.thm_L_c` are the three parts, from `MergeDepth.cor_RV`, `MergeDepth.cor_IS` and `MergeDepth.cor_HCstar_uniform_frozen`, with the binary form of [Q9.1] as a hypothesis and the field DAGs of prunings of the machine in the frozen model in (c). The formal statements are slightly stronger. They hold at all large $d \in \mathcal{D}$ for any set $\mathcal{D}$, finite or not. In (b) every $\eta > 0$ is allowed. In (c) only $\varepsilon + \eta < \frac{1}{2}$ is used, and the cold genies may be taken with any integer $\beta \geq \beta(M)$ and with any depth bound in place of the depth of Lemma 3.1, the threshold then depending also on β and on that bound. □

Update (November 2026). [Q9.1] is false [8, Theorem A and Corollary A′], so Theorem L is vacuous as stated, and so are `MergeDepth.thm_L_a`, `MergeDepth.thm_L_b` and `MergeDepth.thm_L_c`. Its proof uses [Q9.1] only through Lemma 4.12, in parts (a) and (b), and Lemma 3.15(b), in part (c). So Theorem L holds with the conclusions of these two lemmas, the native half-rate bounds for the relations of skeleton classes and for relations inside balls, as hypotheses in place of [Q9.1]; these are open, and the thirteenth paper, which constructs no transposer, does not decide them. Part (c) holds as well under the all-code crux $\mathrm{Crux}(\mathcal{S}^{\mathrm{ball}}_{k,C,\beta,r}, \frac{1}{2})$ for all radii r (after Corollary 3.16), and part (a) under a deficit bound of the kind described in Remark 4.19, which is a proof sketch only. Read with these hypotheses in place of [Q9.1], the discussion below is unchanged.

Part (c) implies that an oblivious machine revisits a work tape unboundedly often: for every V, at all large $d \in \mathcal{D}$ at which M is oblivious, some work-tape cell is entered more than V times. Otherwise there are no deep edges at V, hence no deep merges, and every output position has merge depth 0; but a cold genie at $(V, \frac{1}{4})$ exists (Lemma 3.1), keeping everything is a pruning, and giving every copy its out-edge is a lineage assignment (Remark 3.8), so (c) with $s = 0$, $\varepsilon = \frac{1}{4}$ and $\eta = 0$ is violated.

Given [Q9.1], Theorem L leaves three kinds of machines, and the open problems of Section 9 concern them.

- Oblivious machines that revisit a work tape unboundedly often and whose code merges deep lineages unboundedly often. Hop charging does not exclude them. Conjecture 3.22 asks that the ball systems of oblivious machines have short index codes at a rate set by their deep hops; together with [Q9.1] it would exclude them (Remark 3.23), and Proposition 3.21 proves it for codes whose merge DAG has a cheap cut.

- Adaptive machines whose work tapes have bounded entry and which revisit input cells unboundedly often. Part (b) puts about half of the input into the coarse motion of the input head, but padding meets this at no cost (Remark 4.18): a machine can copy its input to an extra work tape and then step its input head between two cells according to the bits. For a construction, part (b) therefore adds nothing to the revisits that part (a) requires.

- Adaptive machines that revisit some work tape unboundedly often. Neither hop charging, which needs one schedule, nor skeleton entropy, which needs bounded entry on the work tapes, applies to them (Problem 9.2).

At block scale all three are subject to the same unconditional requirement (Theorem 6.4): on a set of $2^{L-O(d)}$ inputs they run one oblivious external-memory transposition with linearly many I/Os.

6 Unconditional reductions

This section proves two reductions that use no coding hypothesis. Both reduce adaptive machines to oblivious ones, at a price in input entropy. Theorem 6.1 freezes one likely trace: the exclusions of the ninth paper [11], which are proved there for one fixed computation geometry, then apply to every machine of sufficiently low adaptivity whose likely trace has the family structure they require. Theorem 6.4 works at block scale, where no hypothesis on adaptivity is needed: every linear-time transposer yields an external-memory transposition with $O(d)$ I/Os, linear in the number of blocks, that is oblivious on a set of $2^{L-O(d)}$ inputs. Consequently an I/O lower bound of order $\omega(d)$ for this form of transposition, proved without the indivisibility assumption of the classical external-memory bounds, would refute (T) (Corollary 6.5).

Machines and traces. We use the machine model and the notation of Section 2, with the following details [3, Section 2.1]. A machine M has state set Q and k work tapes over the alphabet Γ, which contains the blank; every step writes the cell under each work head, and the input cells outside $[0, L)$ hold an end marker. For X uniform on $\{0,1\}^L$, the *probability* of a trace τ_0 is $\Pr[\tau(X) = \tau_0]$. The value $v_t(X)$ of step t consists of the state after the step, the k symbols written and the output bit appended at the step, or a symbol for "none". It takes at most $2^{\beta(M)}$ values, where $\beta(M) = \lceil \log_2 |Q| \rceil + k \lceil \log_2 |\Gamma| \rceil + 2$ is the *value width* of M [4, Section 4.1]. We say that M *transposes at dimension d within $c(L+1)$ steps* if on every input $X \in \{0,1\}^L$, $L = d^2 b(d)$, it halts within $c(L+1)$ steps with output X^{T}.

6.1 Low adaptivity

We recall the return families of the ninth paper. Let $x_1, \dots, x_k$ be the work-head walks of a trace with halting time T, on the times $0, \dots, T-1$. A long hop (p, u) of head h (Section 2) has $u - p \geq 2$, $x_h(p) = x_h(u)$ and $x_h(t) \neq x_h(u)$ for $p < t < u$. Its *side* is upper or lower according as the head

stays above or below the cell $x_h(u)$ between p and u, and its *owner* is the first time in (p, u) at which the head is farthest from that cell [3, Section 2.2]. A *complete family* is the set of all long hops of one head, on one side, with one owner; its *hop count* is its number of hops, and its *full span* is the interval $(p_G, U_G]$ of times, where (p_G, U_G) is its outermost hop. The complete families of the trace admit an *N-class span partition* if they can be partitioned into N classes whose full spans are pairwise disjoint within each class, and a set of complete families is a Q_2 *core* if it admits such a partition into two classes [11, Definition 2.1]. An N-class span partition exists exactly when no time lies in more than N full spans: one direction is clear, and for the other, colour the families greedily in the order of the left ends of their spans; when a family is coloured, the families already coloured whose spans meet its span all contain the first time of its span, so they use at most $N - 1$ colours (`MergeDepth.hasSpanColoring_iff_activeFamilies_card_le`).

Theorem 6.1 (Theorem LA: low adaptivity)**.** `Lean`: `MergeDepth.low_adaptivity_a`, `MergeDepth.low_adaptivity_b`, `MergeDepth.low_adaptivity_a_exists`, `MergeDepth.low_adaptivity_b_exists` [`MergeDepth/LowAdaptivity.lean`]

Let M be a machine, let $\beta \geq \beta(M)$ and $c > 0$, and let $N \geq 1$ and $q \geq 0$ be integers.

(a) Let $\varepsilon > 0$ and d_0 be constants that [11, Corollary 4.5] provides for the parameters $(2c, \beta, N, q)$, and let $\eta < \varepsilon$. Then at no $d \geq \max(d_0, 1)$ does M have both of the following properties at $L = d^2 b(d)$:

(i) M transposes at dimension d within $c(L + 1)$ steps;

(ii) some trace τ_0 has probability at least $2^{-\eta L}$, its complete families admit an N-class span partition, and at most q of them have more than εL hops.

(b) Let $\varepsilon > 0$ and d_0 be constants that [11, Corollary 4.3] provides for the parameters $(2c, \beta, N)$, and let $\eta < \varepsilon$. Then the conclusion of (a) holds when the last clause of (ii) is replaced by: the complete families of τ_0 split into a Q_2 core, at most one further family, and families of at most εL hops each.

Outline of the formal proof. We prove (a). Suppose that M has properties (i) and (ii) at some $d \geq \max(d_0, 1)$, and let W be the set of inputs $X \in \{0,1\}^L$ with $\tau(X) = \tau_0$. Then

$$|W| = 2^L \Pr[\tau(X) = \tau_0] \geq 2^{(1-\eta)L}. \tag{3}$$

The ninth paper allows its fixed geometry to be one trace shared by a set of inputs [11, Section 2.1]; we show that τ_0 and W form an admissible geometry in the sense of [11, Definition 2.2].

The geometry. The trace τ_0 fixes the halting time T, the input-head walk x_0 and the work-head walks $x_1, \ldots, x_k$ on $\{0, \ldots, T - 1\}$, all of which move by at most one cell per step, and the append times. By (i), the machine appends exactly L bits on every $X \in W$, at most one per step, so the append times are $t_0 < t_1 < \cdots < t_{L-1} < T$, and output position r is appended at step t_r. Moreover $T \leq c(L + 1) \leq 2cL$, because $L \geq 1$; this is why the runtime constant is $2c$, as the ninth paper's geometries have $T \leq cL$. Let $x(t)$ be $x_0(t)$ clamped to $[0, L - 1]$.

Local determination. Let $X \in W$ and $t < T$. Step t applies the transition function of M to the state at time t, which is the initial state if $t = 0$ and the state component of $v_{t-1}(X)$ otherwise; to the input symbol, which is $X_{x_0(t)}$ if $0 \leq x_0(t) < L$ and the end marker otherwise; and to the symbols under the work heads. All runs on W move their heads as τ_0 prescribes, and every step writes the cell under each work head. Hence the cell under work head h at time t holds the symbol written at step $\mathrm{prev}_h(t) = \max\{u < t : x_h(u) = x_h(t)\}$, a component of $v_{\mathrm{prev}_h(t)}(X)$, and the blank if that set is empty. If $\mathrm{prev}_h(t) \neq t - 1$, then $(\mathrm{prev}_h(t), t)$ is a long hop of head h. Consequently, for

$X, Y \in W$: if $X_{x(t)} = Y_{x(t)}$ and $v_u(X) = v_u(Y)$ for $u = t - 1$ (when $t \geq 1$) and for the lower ends u of all long hops ending at t, then $v_t(X) = v_t(Y)$. This is the local recursion of [4, Lemma 4.2], for all inputs that follow τ_0. Its parents are the state edge and the work previous-visit edges of [11, Section 2.1], and the input bit at the clamped position $x(t)$ is the direct read there; when $x_0(t)$ lies outside the input, that read is superfluous, which only enlarges the source sets, as noted there.

Correctness and capacity. For $X \in W$ and $0 \leq u < L$, the output bit at position $\mathrm{tpos}(u)$ is appended at step $t_{\mathrm{tpos}(u)}$ and is a component of $v_{t_{\mathrm{tpos}(u)}}(X)$; by (i) it equals X_u. This is the transpose demand of [11, Section 2.1]. The records v_t take at most 2^β values.

Thus the walks and append times of τ_0, the correctness set W, the values v_t as records and the map that extracts the output bit from a record form an admissible geometry with entry width $b(d)$, runtime constant $2c$, capacity β and the N-class span partition of (ii). Its core and exceptional families are labels only; for (a) we take both empty and let a_0 be the largest hop count of any family, or 0 if there is none. Its correctness deficit is $\Delta = L - \log_2|W| \leq \eta L$, by (3).

Let $\mathcal{E}$ be the set of complete families of τ_0 with more than εL hops. By (ii), $|\mathcal{E}| \leq q$. Every other family has at most $a = \lfloor \varepsilon L \rfloor$ hops, and $a \leq \varepsilon L$. Since $d \geq d_0$, [11, Corollary 4.5], which permits a skeleton $\mathcal{E}$ that differs from the geometry's labels, gives $\Delta \geq \varepsilon L$. Hence $\varepsilon L \leq \eta L$, contradicting $\eta < \varepsilon$.

For (b), take the given Q_2 core as the core and the further family, unless it lies in the core, as the exceptional set, and put $a_0 = \lfloor \varepsilon L \rfloor$, which bounds the hop count of every remaining family. The same modelling gives an admissible geometry with at most one exception and $a_0 \leq \varepsilon L$, and [11, Corollary 4.3] gives $\Delta \geq \varepsilon L$, which contradicts $\Delta \leq \eta L$ as before.

Formally, the machines are those of the frozen model, and (i) is `MergeDepth.TransposesAt`, the time bound of the frozen statement of (T) at the one dimension d. The trace of a run records, for every step before the halting time, the moves and whether a bit is appended (`MergeDepth.trace`); its probability is `MergeDepth.traceProb`, and the step values and the value width are `MergeDepth.stepValue` and `MergeDepth.valueWidth`. The constants are any (ε, d_0) that satisfy the conclusion of the ninth paper's corollaries (`MergeDepth.IsSkeletonConstant`, `MergeDepth.IsOneExceptionConstant`), which provide them as `SourceCovers.finite_skeleton_uniform_deficit` and `SourceCovers.one_exception_uniform_deficit`; `MergeDepth.low_adaptivity_a_exists` and `MergeDepth.low_adaptivity_b_exists` choose them depending only on (c, β, N, q) and (c, β, N), for all machines of value width at most β at once. The modelling step is `MergeDepth.fibreGeometry`, with local determination `MergeDepth.stepValue_eq_of_parents` and the deficit bound `MergeDepth.fibreGeometry_deficit_le`. The bound $d \geq 1$ is needed: at $d = 0$ the input is empty, and a machine that halts at once has properties (i) and (ii). The proof gives more. It needs the inputs to follow τ_0 only up to a time by which τ_0 has appended L bits, and correctness only on those inputs: with the same constants, for every machine whose step values have capacity $\log_2(3|Q||\Gamma|^k) \leq \beta$, every entry width $b \geq 1$ in place of $b(d)$, and every time $T' \leq c(L+1)$ before which τ_0 appends at least L bits, at most $2^{(1-\varepsilon)L}$ inputs follow τ_0 up to time T' and are eventually transposed, if the families of τ_0 formed before T' have the structure of (ii) (`MergeDepth.card_correctSet_le_a`, `MergeDepth.card_correctSet_le_b`). The motion after T' and the halting time are unconstrained. □

Hypothesis (ii) holds, for instance, if at most $2^{\eta L}$ distinct traces occur and each of them has the stated family structure; more generally, it holds if the trace has Shannon entropy $\mathrm{H}(\tau(X)) \leq \eta L$ for a uniformly random input and every trace of probability at least $2^{-\eta L}$ has that structure, since $\mathrm{H}(\tau(X)) \geq -\log_2 \max_\tau \Pr[\tau(X) = \tau]$. In probability form, for a machine that transposes at dimension $d \geq \max(d_0, 1)$ within $c(L+1)$ steps, every trace with the family structure of (ii) has probability at most $2^{-\varepsilon L}$ (`MergeDepth.traceProb_le_a`, `MergeDepth.traceProb_le_b`), which

gives (a) and (b) for all $\eta < \varepsilon$ at once. In particular, a transposer all of whose traces have the structure of (a) has at least $2^{\varepsilon L}$ distinct traces (`MergeDepth.card_traces_ge_a`).

The theorem is the freezing argument of the proof of [3, Corollary 6.4] applied to the unconditional exclusions of the ninth paper: the information of the trace is paid once, through the probability of τ_0, and it is smaller than the deficit εL that the frozen geometry forces. It thereby moves the classes excluded by the ninth paper from oblivious machines to machines of low adaptivity, without a coding hypothesis. The ninth paper's formal object for a fixed geometry takes the geometric data as hypotheses and does not derive them from a Turing machine [11, Section 2.1]; here that modelling step is formal too, since `MergeDepth.frozenGeometry` derives every field of the object from a trace of a machine in the frozen model.

Remark 6.2 (Scope of Theorem LA)**.** (1) The constant ε is not explicit. The ninth paper obtains its uniform constants by contradiction, extracting convergent hop profiles by compactness [11, Section 4.1]. Theorem 6.1 therefore excludes adaptivity below an unspecified threshold, whereas the conditional Corollary F″ of the eighth paper [3, Corollary 6.4] allows every $\eta < \frac{1}{2}$.

(2) Like the entropy hypothesis of Corollary F″, hypothesis (ii) concerns the full trace, including the motion of the input head, so padding that motion with data-dependent detours that leave the computation unchanged can destroy it [3, Section 6.2]. Theorem SE (Theorem 4.13) avoids this for Corollary F″ by passing to the coarse skeleton; the admissible geometries of the ninth paper, however, fix the fine walks of all heads, the input head included.

(3) The proof uses [11, Corollary 4.5] only through its bound on the correctness set, which holds uniformly over a class of admissible geometries. The same freezing therefore transfers the other class-uniform bounds of [11, Section 4], its Corollaries 4.4 and 4.7, with their hypotheses imposed on the frozen geometry of τ_0 (`MergeDepth.card_correctSet_le_of_class`).

6.2 Block scale: an oblivious external-memory image

A simulation that keeps two consecutive blocks of each tape in internal memory sees the motion of a machine only when a head leaves its two blocks. A linear-time machine does so $O(d)$ times, each time carrying a constant number of bits; so fixing this coarse motion costs only $O(d)$ bits of input entropy, and on the inputs that share it the simulation is oblivious. We use the following form of the external-memory model of Aggarwal and Vitter [1].

Definition 6.3 (External-memory algorithms)**.** Lean: `MergeDepth.EM.Alg`, `MergeDepth.EM.Runs`, `MergeDepth.EM.inputDisk`, `MergeDepth.EM.HaltsWith`, `MergeDepth.EM.CorrectAt`, `MergeDepth.EM.ObliviousOn` [`MergeDepth/EMModel.lean`]

Fix a finite alphabet Σ containing 0, 1 and a blank $\sqcup$, and integers $B, m \geq 1$ and $r \geq 0$. A *block* is a word in Σ^B. A *disk* is a map $D\colon \mathbb{Z} \to \Sigma^B$ whose values are blank blocks at all but finitely many *addresses*. The *internal memory* consists of m *slots*, each holding one block, and a *register* of r bits; its contents form the set $\mathcal{M} = (\Sigma^B)^m \times \{0,1\}^r$, and its size is $mB\log_2|\Sigma| + r$ bits. An *I/O* is a triple (σ, i, a) with a direction $\sigma \in \{\mathrm{read}, \mathrm{write}\}$, a slot $i \in \{1, \dots, m\}$ and an address $a \in \mathbb{Z}$. An *external-memory algorithm* with these parameters is a map

$$\mathcal{A}\colon \mathcal{M} \to \{\mathrm{halt}\} \cup (\{\mathrm{read}, \mathrm{write}\} \times \{1, \dots, m\} \times \mathbb{Z} \times \mathcal{M}).$$

A *run* from a disk D starts with blank slots and a zero register and repeats the following. If $\mathcal{A}(\mu) = \mathrm{halt}$ for the current contents μ, the run stops. Otherwise $\mathcal{A}(\mu) = (\sigma, i, a, \mu')$; a read replaces μ by μ' with slot i replaced by $D(a)$, and a write sets $D(a)$ to slot i of μ' and replaces μ by μ'. The

arbitrary map $\mu \mapsto \mu'$ models free internal computation. The *I/O sequence* of a run is the sequence of the I/Os (σ, i, a) it performs, and their number is its number of I/Os.

For transposition at dimension d the block size is $B = d\,b(d)$, one matrix row, so that $L = dB$. On input $X \in \{0,1\}^L$ the initial disk holds row i of X, that is the bits at input positions $iB, \ldots, iB + B - 1$, at address i for $0 \le i < d$, and is blank elsewhere. The algorithm is *correct* on X if its run halts with row j of X^{T} at address $d + j$ for every $0 \le j < d$. It is *oblivious on a set* $\mathcal{C}$ of inputs if its runs on all inputs in $\mathcal{C}$ halt with the same I/O sequence.

In the formal model a disk is any map $\mathbb{Z} \to \Sigma^B$; nothing changes, since runs from the input disks keep all but finitely many addresses blank.

Theorem 6.4 (Theorem EM: an oblivious external-memory image)**.** `Lean`: `MergeDepth.EM.theoremEM`, `MergeDepth.EM.theoremEM_sharp`, `MergeDepth.EM.em_core`, `MergeDepth.EM.em_oblivious` [`MergeDepth/ExternalMemory.lean`]

Let M be a machine with k work tapes, state set Q and work alphabet Γ, let $c > 0$, and put

$$c_1 = 4(k+1)c + 2, \qquad c_2 = \bigl(2(k+1)c + 1\bigr)\log_2(2k+4).$$

There is $\gamma > 0$, depending only on k, c and $|Q|$, with the following property. Let $d \ge 1$ be a dimension at which M transposes within $c(L+1)$ steps, and put $B = d\,b(d)$. Then there is an external-memory algorithm over the alphabet Σ_M that consists of 0, 1, the end marker and the symbols of Γ, with the blank of Γ as its blank, with block size B, $2k+3$ slots and a register of at most $\gamma \log_2(2L)$ bits, such that:

(i) it is correct on every input $X \in \{0,1\}^L$ and performs at most $c_1 d$ I/Os on each;

(ii) it is oblivious on a set $\mathcal{C}$ of at least $2^{L - c_2 d}$ inputs.

Outline of the formal proof. Fix d, and let $T_0 = \lfloor c(L+1) \rfloor$, so that M halts within T_0 steps on every input. Call the input tape tape 0 and the work tapes tapes $1, \ldots, k$. Block $j \in \mathbb{Z}$ of a tape consists of its cells $jB, \ldots, jB + B - 1$.

Layout. Block i of the input tape, for $0 \le i < d$, is row i of the input, stored at address i. The other blocks of the input tape consist of end markers and are never stored. Block j of work tape h is stored at address $2d + k\,z(j) + h - 1$, where $z(j) = 2j$ for $j \ge 0$ and $z(j) = -2j - 1$ for $j < 0$; these addresses are distinct and at least $2d$, and they are blank initially, as the work tapes are. The n-th group of B consecutive output bits, $0 \le n < d$, is written to address $d + n$.

Memory. Slots $2\tau + 1$ and $2\tau + 2$ hold the *window* of tape $\tau \in \{0, \ldots, k\}$: its blocks j_τ and $j_\tau + 1$, the lower one in slot $2\tau + 1$. Slot $2k + 3$ is an output buffer. The register holds the state of M, the head positions x_τ, the window indices j_τ, the number n of flushes so far, the number of bits in the buffer, and a *phase*: before the first I/O, running, or waiting for the second I/O of a shift of work tape h in one of the two directions, $2k + 2$ phases in all. Since a run starts with the zero register, the register values are stored by an injective encoding that maps the initial value, in the phase before the first I/O, to the zero word.

Invariants. Each window contains the current contents of its blocks; every work-tape block outside its window has its current contents at its address; the head of tape τ lies in its window $[j_\tau B, (j_\tau + 2)B)$, or, right after a step of M, one cell outside it; and the output bits appended so far are those at the addresses $d, \ldots, d + n - 1$ followed by those in the buffer. Initially $j_\tau = -1$ for every τ. The work windows are then blank and the lower input slot consists of end markers, both without an I/O, and the algorithm reads address 0 into slot 2; this first I/O is the same for all inputs.

Simulation. With the windows in memory, the algorithm simulates the steps of M without I/Os: the symbols under the heads are in the windows, every work symbol written goes into its window, and every output bit appended goes into the buffer. After each step it processes the *events* of that step, first the shifts in the order of the tapes, then a flush, and then it simulates the next step, or halts if M has halted.

- If the head of tape τ has reached $(j_\tau + 2)B$ (a *right shift*), the algorithm writes the lower slot to the address of block j_τ if $\tau \geq 1$, copies the upper slot into the lower one, fills the upper slot with block $j_\tau + 2$, and increases j_τ by one. The block is read from its address if $\tau \geq 1$ or if $\tau = 0$ and $0 \leq j_\tau + 2 < d$; otherwise it consists of end markers and is produced without an I/O. A *left shift*, when the head has reached $j_\tau B - 1$, is symmetric: write back block $j_\tau + 1$ if $\tau \geq 1$, move the lower slot up, fill the lower slot with block $j_\tau - 1$, and decrease j_τ.
- If the buffer holds B bits (a *flush*), it is written to address $d + n$, and n increases by one.

A write sends slot i of the new contents μ' (Definition 6.3), so an application of the map cannot both write a slot and change it. A flush therefore empties the buffer by resetting the count of buffered bits in the register, and a shift of a work tape takes two applications: the first writes back the leaving block and enters the waiting phase, and the second moves the remaining window block to its new slot and reads the entering block. A shift performs at most two I/Os, a shift of the input tape at most one, and a flush one.

Correctness. The invariants are preserved: a shift writes back the block that leaves the window and fills the block that enters it with its current contents, after which the head, which moves by at most one cell per step, lies inside the window again; and a flush moves the buffer to the next output address. Hence the simulation follows the run of M exactly. By transposition, M appends exactly $L = dB$ bits, so there are exactly d flushes, all before M halts, and address $d + n$ receives output bits $nB, \ldots, nB + B - 1$, which form row n of X^{T}. The algorithm is correct on every input.

Counting I/Os. After a shift of tape τ the head lies at offset B (right shift) or $B - 1$ (left shift) in the new window, and at the start it lies at offset B. In each case it is at distance at least B from both cells just outside the window, so the next shift of that tape comes after at least B moves of its head. A head moves at most T_0 times, so tape τ shifts at most

$$\frac{c(L+1)}{B} = c\Big(d + \frac{1}{B}\Big) \leq 2cd$$

times. The number of I/Os is therefore at most $1 + d + 2(k+1) \cdot 2cd \leq c_1 d$, since $d \geq 1$.

Obliviousness. Record each event by a letter: $(\tau, +)$ or $(\tau, -)$ for a right or left shift of tape τ, and ϕ for a flush. The *event word* of a run is the sequence of these letters, a word over an alphabet of $2k + 3$ letters, of length at most $(k+1) \cdot 2cd + d = (2(k+1)c + 1)d =: n_0$. The I/Os performed for an event are determined by its letter and by the current window indices and flush count, which are determined by the earlier letters. Hence the I/O sequence of a run is a function of its event word. There are at most $\sum_{n=0}^{\lfloor n_0 \rfloor} (2k+3)^n \leq (2k+4)^{n_0}$ words of length at most n_0, so some event word is shared by a set $\mathcal{C}$ of at least $2^L (2k+4)^{-n_0} = 2^{L - c_2 d}$ inputs, and the algorithm is oblivious on $\mathcal{C}$.

Register. The register holds the state of M, the phase, and $2k + 4$ integers: the x_τ, the j_τ, the flush count and the buffer count. Each has absolute value at most $T_0 + 2$: $|x_\tau| \leq T_0$, $|j_\tau| \leq |x_\tau| + 2$ because the head is within one cell of its window, and both counts are at most the number of bits appended so far, hence at most T_0. So each integer takes at most $2T_0 + 5 \leq (4c+5)L$ values, and since $\log_2(2L) \geq 1$, the register needs $\lceil \log_2(|Q|\,(2T_0+5)^{2k+4}(2k+2)) \rceil \leq \gamma \log_2(2L)$ bits with $\gamma = (2k+4)\big(\log_2(4c+5) + 1\big) + \log_2|Q| + \log_2(2k+2) + 1$.

Formally, the algorithm is `MergeDepth.EM.emAlg`: a step function that performs at most one I/O per step, collapsed so that one application of the map follows the internal steps up to the next I/O. Its invariant relates each state to the configuration of M at the same time. `MergeDepth.EM.em_core` shows that on every input on which M computes an output of length dB within T_0 steps, the run halts with that output at the addresses $d, \dots, 2d-1$ and performs the first read followed by the I/Os of an event word with at most $\lfloor T_0/B \rfloor$ shifts per tape and exactly d flushes, and `MergeDepth.EM.em_oblivious` chooses $\mathcal{C}$ by pigeonhole over the event words, padded to a common length. The proof gives more. It counts exactly: at most $1 + (2k+1)\lfloor T_0/B \rfloor + d$ I/Os, since a shift of the input tape costs at most one, and $2^L \le |\mathcal{C}|\,(2k+4)^{(k+1)\lfloor T_0/B \rfloor + d}$ (`MergeDepth.EM.theoremEM_nat`). As $\lfloor T_0/B \rfloor \le c(d+1)$, this gives at most $((2k+1)c+1)(d+1)$ I/Os and $|\mathcal{C}| \ge 2^{L-(((k+1)c+1)d+(k+1)c)\log_2(2k+4)}$ (`MergeDepth.EM.theoremEM_sharp`), so that for large d the constants c_1 and c_2 improve to about $(2k+1)c+1$ and $((k+1)c+1)\log_2(2k+4)$. Transposition is used only through the output length: for every machine that computes outputs of length dB from inputs of length dB within T_0 steps, and every set W of such inputs, the algorithm is oblivious on a subset $\mathcal{C} \subseteq W$ with $|W| \le |\mathcal{C}|\,(2k+4)^{(k+1)\lfloor T_0/B \rfloor + d}$ (`MergeDepth.EM.em_oblivious`). □

Theorem 6.4 thus places every linear-time transposer, adaptive or not, inside one concrete external-memory question: a transposition with $O(d)$ I/Os and $O(B)$ bits of internal memory that is oblivious on a set of $2^{L-O(d)}$ inputs. The constants c_1, c_2 do not depend on the alphabet, which affects only the memory size in bits.

Corollary 6.5 (An external-memory route to the negation of (T))**.** Lean: `MergeDepth.EM.corollaryEM`, `MergeDepth.EM.EMLowerBound`, `MergeDepth.EM.corollaryEM_some` [`MergeDepth/ExternalMemory.lean`]

Suppose that for every finite alphabet Σ containing 0, 1 and a blank, and all constants $m \ge 1$, $\gamma \ge 0$ and $A \ge 0$, there is d_0 such that for every $d \ge d_0$ the following holds at $L = d^2 b(d)$ and $B = d\,b(d)$: every external-memory algorithm over Σ with block size B, m slots and a register of at most $\gamma \log_2(2L)$ bits that is correct and oblivious on some set of at least 2^{L-Ad} inputs performs more than Ad I/Os on the inputs of that set. Then (T) fails: no machine, with any number of work tapes, transposes in linear time.

Outline of the formal proof. Suppose that (T) holds, witnessed by a machine M with k work tapes and a constant c. Let γ, c_1 and c_2 be as in Theorem 6.4, and put $\Sigma = \Sigma_M$, $m = 2k+3$ and $A = \max(c_1, c_2)$. Let d_0 be given by the hypothesis for these constants, and let $d \ge \max(d_0, 1)$. By Theorem 6.4 there is an algorithm over Σ with block size $d\,b(d)$, m slots and a register of at most $\gamma \log_2(2L)$ bits that is correct on every input, oblivious on a set of at least $2^{L-c_2 d} \ge 2^{L-Ad}$ inputs, and performs at most $c_1 d \le Ad$ I/Os on each input. This contradicts the hypothesis. Formally, the conclusion is the negation of the frozen statement of (T), `Transpose.LinearTimeTransposable`, and the hypothesis `MergeDepth.EM.EMLowerBound` is needed at one dimension $d \ge 1$ only, for each alphabet and choice of constants (`MergeDepth.EM.corollaryEM_some`). □

At block scale, then, one indivisibility-free I/O lower bound would refute (T), with no hypothesis on adaptivity, nesting or coding.

Remark 6.6 (What the hypothesis of Corollary 6.5 asks)**.** The lower bounds of Aggarwal and Vitter for permuting and for matrix transposition [1] assume that records are indivisible: they are moved and copied but never combined. Under that assumption, transposition with a constant number of blocks of internal memory needs more I/Os than linearly many, by a logarithmic factor; so the algorithm that Theorem 6.4 builds from a linear-time machine must combine records, that is, it must code. The hypothesis of Corollary 6.5 asks for a bound of order $\omega(d)$ without indivisibility,

for algorithms that need only be correct and oblivious on a set of $2^{L-O(d)}$ inputs, which need not be a product set. Farhadi, Hajiaghayi, Larsen and Shi prove external-memory lower bounds for sorting without indivisibility, by network coding [14]. They reduce non-oblivious algorithms to a fixed I/O graph by a pigeonhole over I/O graphs, the step that produces $\mathcal{C}$ in Theorem 6.4. Their bounds are conditional on the conjecture that network coding gives no advantage over multicommodity flow in undirected graphs, and that conjecture is false in general [2]. We know of no unconditional bound of the required kind. Such a bound need not deal with adaptivity: fixing the I/O sequence costs only $O(d) = o(L)$ bits of input entropy, so on $\mathcal{C}$ a machine's adaptivity shapes only its motion within blocks. The reduction goes one way only: since internal computation is free and blocks are accessed at random, an external-memory transposition with $O(d)$ I/Os would not by itself give a linear-time machine.

7 Evidence at deep nesting: block pairs

The deep-nesting pairs of the eighth paper of the series [3, Section 2.5] are the sharpest open instance of TCP(2): its Conjecture A″ [3, Conjecture 5.24], that at a suitable odd shift every label of cost at most $n_{\mathrm{h}}/8$ on every cover doubly θ_0-serves at most $C \log_2 q + C' + \eta(m+1)$ levels with $\eta < 1$, would make them depth-robust, so that TCP(2) fails [3, Corollary 5.26]. For pairs of block labels of bounded length, the tenth paper [9, Section 5] proved failure witnesses in three regimes (its Theorem A, which rests on its Theorems V* and LU, all Lean-verified there) and named four further steps towards Conjecture A″ [9, Remark 5.6]: (a) one odd shift serving both directions; (b) half-blindness of the fine levels at that shift; (c) from witnesses to counts of served levels; (d) unbounded block lengths and general labels.

This section closes steps (b) and (c) for bounded block length and reduces step (a) to Conjecture S (Conjecture 7.3): the straddle counts of most coarse classes concentrate as the shift varies. The result is Theorem BP (Theorem 7.10): assuming Conjecture S, Conjecture A″ holds for pairs of block labels of length at most a_0, at one odd shift, with no $\log q$ term and no cost hypothesis. Theorem BP is evidence that the deep-nesting pairs are depth-robust; it does not decide TCP(2), since Conjecture A″ concerns every label of cost at most $n_{\mathrm{h}}/8$ (Remark 7.11). Section 7.5 gives partial results on Conjecture S and numerical evidence for it up to $m = 4095$.

The written proofs use the tenth paper's Theorems V*, LU and A (Lean-verified there) and the eighth paper's lemmas on the deep-nesting pair (written proofs there). Every numbered result of this section is formalized in Lean, with Conjecture S as a hypothesis where the statement assumes it. The formalization includes the deep-nesting walk, the eighth paper's Lemma 5.1(a)–(c) and its blindness bound (Proposition 5.17, without its level-0 term), so no step of the formal proofs rests on a written proof of the eighth paper (Section 7.1).

7.1 Setting

We use the notation of [3, Sections 2.5 and 5]: $s_0 > m \geq 1$, $\Lambda = 2^{s_0}$, $R_i = 2^{s_0 - i}$, $n_i = m - i + 1$, $\Pi_i = n_i R_i$ (the length of a stroke of level i), $R_{m+1} = 0$, and we put $N = m + 1$, so $P = 4N\Lambda = 4\Pi_0$. The pair is $x_1 = U = U(s_0, m, 1)$ and $x_2 = x_1(\cdot + \xi)$ for an odd shift ξ, so that no time is a turn of both heads. On the r-fold cover a label G with q values gives the hop (p, u) the error $e(p, u) = (G(u) - G(p) - 1) \bmod q$; the hops of head h of level k form the class (h, k); and G *θ-serves* level k for head h if the class has total error at most $\theta q n_{\mathrm{h}}/(2N)$, and *doubly θ-serves* it if it does so for both heads. Every class has exactly $n_{\mathrm{h}}/(2N) = 4r\Lambda$ hops [3, Lemma 5.1(c)], so θ-service means mean error at most θq. A hop is *blind* if its half-open interval $(p, u]$ contains no turn of the other head.

Block pairs. For $a \geq 1$ and $K \in \mathbb{Z}$, $B(a,K)$ is the block label of [3, Definition 5.11]; that definition takes $K \geq 0$, but it and the identities of [3, Lemma 5.12] used below hold verbatim for every integer K, as in the tenth paper. Its *cancellers* are the levels l with $m - l \equiv a \pmod{a+1}$; it has level parameters A_l, E_l, block mass $Q(a,K) = (2^a - 1)(2K+2)$ and path-sum increments $c_{\mathrm{b}}(l) = 1 + A_l$ and $c_{\mathrm{f}}(l) = 2 + A_l + E_l$, with $c_{\mathrm{f}}(\kappa) - c_{\mathrm{b}}(\kappa) = -Q(a,K) - 1$ at every canceller κ. For $q \mid r$ the *block-pair label* of $(B(a_1,K_1), B(a_2,K_2))$ modulo q on the r-fold cover is $G = (\Phi_1 + \Phi_2) \bmod q$, where Φ_h is the potential of head h carrying $B(a_h,K_h)$ at its own turns [3, Section 5.3]; it is well defined because each Φ_h grows by an integer per period and $q \mid r$, as in [3, Proposition 5.23], and it is turn-supported. For a head h, a level k and $j \geq 0$ with $k + j \leq m$, the *blind errors* of $B(a_h,K_h)$ are

$$a^{(h)}(k) = A_k^{(h)} \bmod q, \qquad b_j^{(h)}(k) = \Big(E_k^{(h)} - \sum_{i=k+1}^{k+j} \big(A_i^{(h)} + E_i^{(h)} + 2\big)\Big) \bmod q$$

and $\tilde{Q}_h = Q(a_h, K_h \bmod q)$; in the tenth paper's library these errors are `DirectedReciprocity.blindApexErr` and `DirectedReciprocity.blindBandErr`. By the band decomposition [3, Lemma 5.10], the apex hop $(\alpha - d, \alpha + d)$, $1 \leq d < R_k$, of a level-k V of head h has error $(A_k^{(h)} + F) \bmod q$, and its valley hop of band j, at a cell $v + \sigma x$ with $R_{k+j+1} < x < R_{k+j}$, has error $(b_j^{(h)}(k) + F) \bmod q$, where F is the other head's increment over the hop; on a blind hop $F = 0$.

Four results of the tenth paper. All four are Lean-verified there; the second and third are the shift-free components `DirectedReciprocity.block_overflow_band_witness` and `DirectedReciprocity.block_straddle_residue_ge_half_q` of the proof of [9, Theorem 5.5] (its Regimes 2 and 3). Define $a(k)$, $b_j(k)$ as above for any integer sequences (A_l), (E_l) and $q \geq 3$.

(I) *Run bound* [9, Theorem 5.3(2)]. If $s \geq 1$ and the levels $k_0, \ldots, k_0 + s - 1$ satisfy $8a(k_0 + i) \leq q$ and $8b_0(k_0 + i) \leq q$ for $i < s$, $8b_1(k_0 + i) \leq q$ for $i + 1 < s$, and $8b_2(k_0 + i) \leq q$ for $i + 2 < s$, then $s \leq 2\lfloor \log_2 q \rfloor + 2$.

(II) *Overflow bound.* If $a \geq 1$, $K \in \mathbb{Z}$, $q \geq 32$ and $4Q(a, K \bmod q) + 4 > q$, then there is $\mathrm{rem} \in \{0, \ldots, a-1\}$ such that every level $k < m$ with $(m - k) \bmod (a+1) = \mathrm{rem}$ has $16a(k) \geq q$, $8b_0(k) \geq q$ or $8b_1(k) \geq q$, for the blind errors of $B(a,K)$.

(III) *Residue bound.* If $a_1, a_2 \geq 1$, $K_1, K_2 \in \mathbb{Z}$, $q \geq 1$, $\tilde{Q}_1 \leq \tilde{Q}_2$, $4\tilde{Q}_2 + 4 \leq q$, and κ is a canceller of $B(a_2, \cdot)$, then for every level k, $(2A_k^{(1)} + 2(c_{\mathrm{f}}^{(2)}(\kappa) - c_{\mathrm{b}}^{(2)}(\kappa))) \bmod q \geq q/2$, where $A^{(1)}$ belongs to $B(a_1,K_1)$ and $c_{\mathrm{b}}^{(2)}, c_{\mathrm{f}}^{(2)}$ to $B(a_2,K_2)$.

(IV) *Theorem V** [9, Theorem 5.2(2)]. Let $\Phi\colon \mathbb{Z} \to \mathbb{Z}$, let $k \geq 1$ and κ satisfy $\Pi_\kappa \leq R_k$, let $\alpha \in \mathbb{Z}$, and suppose that there are integers $C_V, c_{\mathrm{b}}, c_{\mathrm{f}}$ and a function Y with $\Phi(\alpha + o) = C_V + c_{\mathrm{b}} + Y(o)$ and $\Phi(\alpha + \Pi_\kappa + o) = C_V + c_{\mathrm{f}} + Y(o)$ for $0 \leq o < \Pi_\kappa$. For an integer c_0 let $S = \mathrm{Strad}(c_0, \alpha, \Pi_\kappa, R_k)$ be the set of $d \in [1, R_k - 1]$ with $\alpha \leq c_0 - d < \alpha + \Pi_\kappa \leq c_0 + d < \alpha + 2\Pi_\kappa$, and for $A' \in \mathbb{Z}$ and $q \geq 1$ put $e(d) = (A' + \Phi(c_0 + d) - \Phi(c_0 - d)) \bmod q$. Then $|S| \cdot ((2A' + 2(c_{\mathrm{f}} - c_{\mathrm{b}})) \bmod q) \leq 2\sum_{d \in S} e(d)$.

Formalization. The walk U is `MergeDepth.dnpWalk`. Its turns are the apex and valley times of its V's and two tips per period (`MergeDepth.isTurnZ_dnpWalk_iff`), and the elements of the tenth paper's apex sets, used for $\mathcal{A}_i$ in Definition 7.1, are apices of V's of level i (`MergeDepth.dnpWalk_V`). The eighth paper's Lemma 5.1 is proved: each head has $2^{m+3} - 6$ turns per period (`MergeDepth.card_turns_period`), the hops of level $k \geq 1$ are those listed there (`MergeDepth.levelHops_eq_listed`), and every class has 4Λ hops per period and $4r\Lambda = n_{\mathrm{h}}/(2N)$ on the r-fold cover

(`MergeDepth.levelCount_eq`, `MergeDepth.card_coverClass`). A block label is a jump function with the properties of [3, Definition 5.11] (`MergeDepth.IsBlockLabel`); there is exactly one, and its potential is that of the tenth paper's stroke recursion (`MergeDepth.IsBlockLabel.potentialOf_eq`). The block-pair label, its errors, θ-service and blindness are `MergeDepth.blockPairLabel`, `MergeDepth.hopError`, `MergeDepth.ThetaServes` and `MergeDepth.IsBlind`. Lemmas 7.7 to 7.9 and Theorem BP are first proved in one period, for the families of Definition 7.6 listed explicitly as pairs of times, with these potentials, and with service measured on the four families of a class. They transfer to the walk pair because the families consist of hops of their classes (`MergeDepth.family_inHopClass`), the errors are those of the block-pair label (`MergeDepth.hopError_blockPairLabel`), and a level that is θ-served for a head on some cover is also served when service is measured on its four families (`MergeDepth.ThetaServes.servedOnWitness`). The formal proofs do not use $q \mid r$, which is needed only for G to be a label of the cover (`MergeDepth.blockPairLabel_add_mul`).

7.2 Straddle counts and Conjecture S

Definition 7.1 (Straddle counts and good levels)**.** Lean: `MergeDepth.coarseZone`, `MergeDepth.jZero`, `MergeDepth.goodLevels`, `MergeDepth.goodCount` [`MergeDepth/BlockPairsProfile.lean`]

Let $\mathcal{K} = \{1, \dots, \lfloor N/4 \rfloor\}$ be the *coarse zone* and $j_0 = \lfloor \log_2 N \rfloor + 1$. For $a \geq 1$ and $k \geq 1$ let $\kappa_a(k)$ be the least integer $\kappa \geq k + j_0$ with $m - \kappa \equiv a \pmod{a+1}$ (the tenth paper's `DirectedReciprocity.cancellerBelow`(m, a, k) if $k + j_0 \leq m$); thus $j_0 \leq \kappa_a(k) - k \leq j_0 + a$, and $\kappa_a(k)$ is a canceller of $B(a, \cdot)$ if $\kappa_a(k) \leq m$. For $1 \leq i \leq m$ let $\mathcal{A}_i \subseteq [0, P)$ be the set of the 2^{i+1} apex times of the level-i V's of U in one period. For $1 \leq k < \kappa \leq m$ with $\Pi_\kappa \leq R_k$ and $\xi \in \mathbb{Z}$ put

$$X^{\pm}_{k,\kappa}(\xi) = \sum_{\alpha_1 \in \mathcal{A}_k} \sum_{\alpha \in \mathcal{A}_\kappa} |\mathrm{Strad}((\alpha_1 \pm \xi) \bmod P,\ \alpha,\ \Pi_\kappa,\ R_k)|,$$

which for $0 \leq \xi \leq P$ is the tenth paper's `DirectedReciprocity.classStraddleCount` at $(s_0, m, k, \kappa, \mathrm{dir}, \xi)$, with dir true for $+$ and false for $-$. A level $k \in \mathcal{K}$ is *a-good in direction $\pm$ at ξ* if $X^{\pm}_{k,\kappa_a(k)}(\xi) \geq 2^{k+2} R_k / 2^{a+7}$, and $\Gamma_{a,\pm}(\xi)$ is the number of such levels.

In head 2's own time, where it is U, the apex hop $(\alpha_1 - d, \alpha_1 + d)$ of head 1 is the window of half-width d around $\alpha_1 + \xi$; so $X^{+}_{k,\kappa}(\xi)$ counts the apex hops with $d < R_k$ of the class $(1, k)$ that start in the back stroke and end in the forward stroke of a level-κ V of head 2, and X^{-} does the same with the heads exchanged. The threshold is that of [9, Theorem 5.5(3)]. Given $a_0 \geq 1$, we use the *size condition* $4(j_0 + a_0) \leq N \quad (*)$; under it, for $k \in \mathcal{K}$, $1 \leq a \leq a_0$ and $\kappa = \kappa_a(k)$, the tenth paper's `DirectedReciprocity.cancellerBelow_props` gives $\kappa \leq m$, $\Pi_\kappa \leq R_k$ and $N \leq 2n_\kappa$.

Definition 7.2 (Straddle profile)**.** Lean: `MergeDepth.piHat`, `MergeDepth.apexHat`, `MergeDepth.valleyHat`, `MergeDepth.tent`, `MergeDepth.perTent`, `MergeDepth.profile`, `MergeDepth.profileMean`, `MergeDepth.profileVariance`, `MergeDepth.relVariance`, `MergeDepth.relVarianceA`, `MergeDepth.meanRelVariance` [`MergeDepth/BlockPairsProfile.lean`]

Let $1 \leq k < \kappa \leq m$ and $\hat{\Pi}_i = n_i 2^{-i}$, so that $\Pi_i = \Lambda \hat{\Pi}_i$. For $S \subseteq \{1, \dots, k-1\}$ and $S' \subseteq \{1, \dots, \kappa - 1\}$ put

$$\hat{a}_S = \sum_{i=0}^{k-1} 2^{-i} + \sum_{i \in S} \hat{\Pi}_i, \qquad \hat{v}_{S'} = \sum_{i=0}^{\kappa-1} 2^{-i} + \hat{\Pi}_\kappa + \sum_{i \in S'} \hat{\Pi}_i,$$

let $\Delta(y) = \max(0, \hat{\Pi}_\kappa - 2|y|)$, and define the N-periodic function

$$x_k(t) = \frac{1}{2^{k-1} \hat{\Pi}_\kappa} \sum_S \sum_{S'} \sum_{l \in \mathbb{Z}} \Delta(\hat{a}_S + t - \hat{v}_{S'} - lN) \qquad (t \in \mathbb{R}).$$

For t uniform on $[0,N)$ let $\mu_k = \mathbb{E}\,x_k(t)$ and $V_k = \operatorname{Var} x_k(t)/\mu_k^2$. Write $V_k^{(a)}$ for V_k at $\kappa = \kappa_a(k)$, which is at most m for $k \in \mathcal{K}$ once $4(j_0+a) \le N$, and $\bar{V}_m^{(a)} = |\mathcal{K}|^{-1}\sum_{k\in\mathcal{K}} V_k^{(a)}$.

These depend on m, k and κ, not on s_0. In units of Λ, the $\hat{a}_S$ and $\hat{v}_{S'}$ are the level-k apex and level-κ valley times of one level-0 stroke, and x_k is the normalized straddle count at a real shift (Lemma 7.4). Each tent has integral $\hat{\Pi}_\kappa^2/2$, so $\mu_k = n_\kappa/(4N)$.

Conjecture 7.3 (Conjecture S)**.** *For every integer $a \ge 1$, $\bar{V}_m^{(a)} \to 0$ as $m \to \infty$.*

In Lean, Conjecture S for one a is `MergeDepth.ConjectureS`.

By Proposition 7.12, Conjecture S is an averaged L^2 bound for Riesz products along the mixed-radix digits $n_i 2^{-i}$; by Proposition 7.13, its analogue for single classes fails.

Lemma 7.4 (Straddle counts and the profile)**.** Lean: `MergeDepth.prob_straddle_vs_profile` [`MergeDepth/BlockPairsTransfer.lean`]
Let $1 \le k < \kappa \le m < s_0$ with $\Pi_\kappa \le R_k$, and put $M_k = 2^{k+1}\Pi_\kappa$.

(a) For all integers c_0 and α, $|\mathrm{Strad}(c_0,\alpha,\Pi_\kappa,R_k)| = \max\bigl(0,\Pi_\kappa - |2(c_0-\alpha-\Pi_\kappa)+1|\bigr)$.

(b) For every integer $0 \le \xi \le P$, $X^+_{k,\kappa}(\xi) = M_k\,x_k\bigl((\xi+\tfrac12)/\Lambda\bigr)$ and $X^-_{k,\kappa}(\xi) = X^+_{k,\kappa}(P-\xi)$.

(c) If ξ is uniform over the odd residues modulo P, then for every real τ and each sign

$$\Bigl|\Pr\bigl[X^{\pm}_{k,\kappa}(\xi) < \tau M_k\bigr] - \frac{1}{N}\,\mathrm{Leb}\{t\in[0,N) : x_k(t) < \tau\}\Bigr| \;\le\; 2^{\kappa+2-s_0}.$$

Outline of the formal proof. (a) With $z = c_0 - \alpha$ the conditions say $\max(1, z-\Pi_\kappa+1, \Pi_\kappa - z) \le d \le \min(R_k-1, z, 2\Pi_\kappa - 1 - z)$. The integers $z-\Pi_\kappa+1$, $\Pi_\kappa - z$ sum to 1 and z, $2\Pi_\kappa - 1 - z$ sum to $2\Pi_\kappa - 1$, so the bounds 1 and $R_k - 1 \ge \Pi_\kappa - 1$ never bind; with $y = z - \Pi_\kappa$ the remaining bounds are $\frac12 + |y+\frac12|$ and $\Pi_\kappa - \frac12 - |y+\frac12|$, leaving $\max(0, \Pi_\kappa - |2y+1|)$ values of d.

(b) By (a), $|\mathrm{Strad}(c_0,\alpha,\Pi_\kappa,R_k)| = \Lambda\Delta((c_0+\frac12-\nu)/\Lambda)$ with $\nu = \alpha + \Pi_\kappa$ the valley time, and it vanishes unless $c_0 \in [\alpha, \alpha+2\Pi_\kappa)$. The four level-0 strokes of a period have length $\Pi_0 = N\Lambda$ and the same turn times relative to their starts, and every V lies inside one of them [3, Section 2.5]; a stroke of level $l < m$ starting at t_0 has the apex of its V at $t_0 + R_l$, and the strokes of that V start at $t_0 + R_l$ and $t_0 + R_l + \Pi_{l+1}$. So in the stroke starting at 0 the level-k apex times are the $\Lambda\hat{a}_S$ and the level-κ valley times are the $\Lambda\hat{v}_{S'}$, and for integers $c_0 \in [0,P)$, $\sum_{\alpha\in\mathcal{A}_\kappa}|\mathrm{Strad}(c_0,\alpha,\Pi_\kappa,R_k)| = \Lambda H((c_0+\frac12)/\Lambda)$ with the N-periodic $H(y) = \sum_{S'}\sum_{l\in\mathbb{Z}}\Delta(y - \hat{v}_{S'} - lN)$. As $\mathcal{A}_k = \{\Lambda\hat{a}_S + j\Pi_0 : 0 \le j \le 3\}$, $X^+_{k,\kappa}(\xi) = 4\Lambda\sum_S H(\hat{a}_S + t) = M_k x_k(t)$ with $t = (\xi+\frac12)/\Lambda$. The second identity holds as $(\alpha_1 - \xi) \bmod P = (\alpha_1 + (P-\xi)) \bmod P$.

(c) Let $g(u) = M_k x_k(u/\Lambda)$, so $X^+_{k,\kappa}(\xi) = g(\xi+\frac12)$ by (b). As the $\hat{a}_S, \hat{v}_{S'}$ are multiples of $2^{-\kappa}$ and $\hat{\Pi}_\kappa/2$ is a multiple of $2^{-\kappa-1}$, g is affine on each of the $N2^{\kappa+3}$ intervals $[iR_\kappa/2, (i+1)R_\kappa/2)$ that tile $[0,P)$. So each of them meets $\{g < \tau M_k\}$ in an interval of some length λ, which contains $\lambda/2 \pm 1$ of the points $\xi + \frac12$ with ξ odd, as these have spacing 2. Summing over the intervals, the number of odd $\xi \in [0,P)$ with $X^+_{k,\kappa}(\xi) < \tau M_k$ differs from half the measure of $\{u \in [0,P) : g(u) < \tau M_k\}$, which is $4\Lambda\,\mathrm{Leb}\{t\in[0,N) : x_k(t) < \tau\}$ by periodicity, by at most $N2^{\kappa+3}$; dividing by the number $P/2 = 2N\Lambda$ of odd residues gives the bound. The sign $-$ follows from (b), as $\xi \mapsto P-\xi$ permutes the odd residues. Formally, (a) is `MergeDepth.card_straddleSet_eq`, which needs only $\Pi_\kappa \le R_k$; (b) is `MergeDepth.classStraddleCount_eq_profile` and `MergeDepth.classStraddleCount_false_eq`; and (c) is `MergeDepth.prob_straddle_vs_profile`. □

Proposition 7.5 (Conjecture S yields a good shift)**.** Lean: `MergeDepth.notGoodProb_le`, `MergeDepth.prob_some_bad_le`, `MergeDepth.good_shift_prob` [`MergeDepth/BlockPairsShift.lean`]

Let $a_0 \geq 1$ and $0 \leq \gamma < 1$, let $m < s_0$ satisfy $(*)$, and let ξ be uniform over the odd residues modulo P. For $1 \leq a \leq a_0$ and $k \in \mathcal{K}$ put $\kappa = \kappa_a(k)$ and $\theta_k = N2^{\kappa-k}/(2^{a+4}n_\kappa^2)$, and let $\beta_k^{(a)}$ be the probability that k is not a-good in direction $+$.

(a) $\theta_k \leq 1/2$; k fails to be a-good in direction $-$ also with probability $\beta_k^{(a)}$; and $\beta_k^{(a)} \leq V_k^{(a)}/(V_k^{(a)} + (1-\theta_k)^2) + 2^{\kappa+2-s_0}$.

(b) With $\beta_a = |\mathcal{K}|^{-1}\sum_{k\in\mathcal{K}}\beta_k^{(a)}$, the probability that $\Gamma_{a,\pm}(\xi) < \gamma|\mathcal{K}|$ for some $1 \leq a \leq a_0$ and some sign is at most $\frac{2}{1-\gamma}\sum_{a=1}^{a_0}\beta_a$.

(c) If Conjecture S holds for $a = 1, \dots, a_0$, there is $m_0 = m_0(a_0, \gamma)$ such that for every $m \geq m_0$ $(*)$ holds and, for every $s_0 > m$, with probability at least $1/2$, $\Gamma_{a,\pm}(\xi) \geq \gamma|\mathcal{K}|$ for every $1 \leq a \leq a_0$ and both signs.

Outline of the formal proof. (a) As $N \leq 2n_\kappa$, and $2^{\kappa-k} \leq 2^{j_0+a} \leq 2^{a+1}N$ because $2^{j_0} \leq 2N$, we get $\theta_k \leq 1/2$. The threshold of a-goodness is $\tau_k M_k$ with $\tau_k = R_k/(2^{a+6}\Pi_\kappa) = \theta_k\mu_k$. So by Lemma 7.4(c), and by Cantelli's inequality $\Pr[x \leq \mu - \lambda] \leq \sigma^2/(\sigma^2+\lambda^2)$ $(\lambda > 0)$ with $\lambda = (1-\theta_k)\mu_k$, the bound holds; the sign $-$ follows from Lemma 7.4(b).

(b) By (a), $|\mathcal{K}| - \Gamma_{a,\pm}(\xi)$ has expectation $\beta_a|\mathcal{K}|$, so by Markov's inequality it exceeds $(1-\gamma)|\mathcal{K}|$ with probability at most $\beta_a/(1-\gamma)$; take a union bound over the $2a_0$ pairs $(a, \pm)$.

(c) For $k \in \mathcal{K}$, $\kappa \leq N/4 + j_0 + a_0$ and $s_0 \geq N$, so $2^{\kappa+2-s_0} \leq 2^{3+a_0}N2^{-3N/4}$; with $\theta_k \leq 1/2$, (a) gives $\beta_a \leq 4\bar{V}_m^{(a)} + 2^{3+a_0}N2^{-3N/4}$ for every $s_0 > m$. Under Conjecture S this tends to 0 for each $a \leq a_0$, and $(*)$ holds for all large m. Choose m_0 such that for $m \geq m_0$ the size condition holds and $\sum_{a\leq a_0}\beta_a \leq (1-\gamma)/4$ for every $s_0 > m$; then (b) bounds the failure probability by $1/2$. Formally, (a) is `MergeDepth.notGoodProb_le`, with the mean of the profile computed in `MergeDepth.profileMean_eq` and Cantelli's inequality in `MergeDepth.cantelli_interval`; (b) is `MergeDepth.prob_some_bad_le`, which holds without the size condition; and (c) is `MergeDepth.good_shift_prob`, with Conjecture S as a hypothesis. □

7.3 Half-blindness and witnesses

Definition 7.6 (Families and the fine zone)**.** Lean: `MergeDepth.InFamily`, `MergeDepth.familyPerPeriod`, `MergeDepth.HalfBlindPair` [`MergeDepth/DeepNestingBP.lean`]

Let $1 \leq k \leq m-2$ and let h be a head. The four *families* of the class (h,k) are the *apex family*, the apex hops $(\alpha - d, \alpha + d)$ with $1 \leq d < R_k$ of the level-k V's of head h, and for $j \in \{0,1,2\}$ the *band-j family*, their valley hops at the cells $v + \sigma x$ with $R_{k+j+1} < x < R_{k+j}$ [3, Lemma 5.1(b)]. A family is *half-blind* at ξ if at least half of its hops with later endpoint in $[0, P)$ are blind. For $0 < \varepsilon \leq 1/8$ the *fine zone* is $\mathcal{F}_\varepsilon = \{k \in \mathbb{Z} : m - \lfloor \varepsilon m \rfloor \leq k \leq m-2\}$.

The pair is P-periodic, so a family has the same fraction of blind hops in every period and on every cover. In Lean, a family consists of the shifts by multiples of P of hops that `MergeDepth.family` lists in one period, in the time of their head, and the fine zone is `MergeDepth.fineZone`.

Lemma 7.7 (Families of fine classes)**.** Lean: `MergeDepth.familyPerPeriod_subset_class`, `MergeDepth.class_card_le_family`, `MergeDepth.hopError_of_inFamily_blind`, `MergeDepth.familyBlindErr_le_of_thetaServes` [`MergeDepth/DeepNestingBP.lean`]

Let $s_0 > m$, $1 \leq k \leq m-2$, let h be a head, and let G be a block-pair label modulo q on the r-fold cover.

(a) Each family of the class (h,k) contains at least a fraction $1/32$ of the hops of the class.

(b) All blind hops of the apex family have error $a^{(h)}(k)$, and all blind hops of the band-j family have error $b_j^{(h)}(k)$.

(c) If G θ-serves level k for head h and a family of the class (h,k) is half-blind at ξ, then the common error of its blind hops is at most $64\theta q$.

Outline of the formal proof. (a) Per level-k V of head h the class has $2R_k$ hops [3, Lemma 5.1(b)], the apex family $R_k - 1$, and the band-j family ($l = k + j$) $R_{l+1} - 1$ if $l < m$ and $R_m - 1$ if $l = m$. As $s_0 > m$, $R_k \geq 2^{m-k+1} \geq 8$, and $R_k \geq 16$ if $k+2 < m$. So the fractions are at least $\frac{1}{2} - \frac{1}{16}$ (apex), $2^{-j-2} - \frac{1}{2R_k} \geq \frac{1}{32}$ (band j with $l < m$; if $j = 2$ then $k + 2 < m$) and $2^{-j-1} - \frac{1}{2R_k} \geq \frac{1}{16}$ (band j with $l = m$).

(b) This is the band decomposition with $F = 0$, as $A_V = A_{\mathrm{lev}V}$ and $E_V = E_{\mathrm{lev}V}$ for a block label [3, Definition 5.11].

(c) The class has $4r\Lambda$ hops and total error at most $4r\Lambda\theta q$. By (a) and periodicity the family has at least $4r\Lambda/32$ hops, at least half of them blind, each with the common error e of (b); so $e \cdot 4r\Lambda/64 \leq 4r\Lambda\theta q$. Formally, (a)–(c) are first proved for the families listed in one period (`MergeDepth.family_card_ge`, `MergeDepth.pairHopErr_of_blind`, `MergeDepth.familyBlindErr_le_of_served`); in (b) the potential of the other head is constant on a blind window, and the tenth paper's stroke recursion gives the increment of the own head. They transfer to the walk pair as described in Section 7.1. □

Lemma 7.8 (Half-blindness at a random odd shift)**.** Lean: `MergeDepth.sum_nonHalfBlindPair_le`, `MergeDepth.prob_many_nonHalfBlindPair_le` [`MergeDepth/DeepNestingBP.lean`]

Let $0 < \varepsilon \leq 1/8$ and $m < s_0$ with $\varepsilon m \geq 4$, and let ξ be uniform over the odd residues modulo P. The expected number of triples (h,k,f), with h a head, $k \in \mathcal{F}_\varepsilon$ and f a family of the class (h,k), that are not half-blind at ξ is at most $64\varepsilon^2 m$. Hence, for $a_0 \geq 1$ and $\delta = 1/(48(a_0+1))$, more than $\delta|\mathcal{F}_\varepsilon|$ such triples exist with probability at most $6144(a_0+1)\varepsilon$.

Outline of the formal proof. First, a window $(p,u]$ of length ℓ of either head is blind with probability at least

$$N^{-1}\sum_{i=1}^{m}(1-\ell/R_i)^+ - 2/(NR_m)$$

over odd shifts; this is [3, Proposition 5.17] without its level-0 term. Indeed, in the time of the other head, where it is U, each of the 2^{i+2} turns of level $i \geq 1$ per period is followed by R_i steps without a turn, a window that starts in the first $R_i - \ell$ of them is blind, these starts are disjoint, and at least $((R_i-\ell)^+ - 1)/2$ of them have the parity that an odd shift requires; as $2^{i+2}R_i = P/N$ and $\sum_{i=1}^{m} 2^{i+2}/P < 2/(NR_m)$, the bound follows. If $\ell \leq R_L$ with $L \leq m$, the levels $1,\dots,L$ contribute at least $\sum_{i=1}^{L}(1-2^{i-L}) \geq L-2$ to the sum, and $2/(NR_m) \leq 1/N$ as $R_m \geq 2$.

Let $k \in \mathcal{F}_\varepsilon$ and $g = m - k$, so $2 \leq g \leq \varepsilon m$. An apex hop of (h,k) has length $2d < R_{k-1}$, so the sum is at least $k-3$. A valley hop of band j, at level $l = k + j$, has length $(m-l)R_l + 2x < (m-l+2)R_l \leq R_{l-\lambda}$ with $\lambda = \lceil \log_2(m-l+2) \rceil \leq \lceil \log_2(g+2) \rceil$ [3, Lemma 5.1(b)], so the sum is at least $l - \lambda - 2$ if $l \geq \lambda$. So every hop of every family of (h,k) fails to be blind with probability at most $\varepsilon_g = (g + 4 + \lceil \log_2(g+2) \rceil)/N$ (trivially if $l < \lambda$, as then $\varepsilon_g > 1$). Hence the expected non-blind fraction of a family is at most ε_g, and by Markov's inequality it fails to be half-blind with probability at most $2\varepsilon_g$. With $G = \lfloor \varepsilon m \rfloor$ we have $\varepsilon_g \leq 4G/N$, as $\lceil \log_2(g+2) \rceil \leq g$; summing over the two heads, the four families and the $G-1$ levels of $\mathcal{F}_\varepsilon$ gives at most $64(G-1)G/N \leq 64\varepsilon^2 m$. As $|\mathcal{F}_\varepsilon| = G - 1 \geq \varepsilon m/2$, Markov's inequality bounds the probability of more than $\delta|\mathcal{F}_\varepsilon|$ such triples by

$128\varepsilon/\delta = 6144(a_0+1)\varepsilon$. Formally, the blindness bound is `MergeDepth.blindnessBound_holds`, and the lemma is first proved for the families listed in one period (`MergeDepth.sum_nonHalfBlind_le'`, `MergeDepth.prob_many_nonHalfBlind_le'`); for the walk pair, half-blindness is the same condition (`MergeDepth.halfBlindPair_iff`). □

Lemma 7.9 (Witnesses give unserved levels)**.** Lean: `MergeDepth.witnesses_unserved_pair` [`MergeDepth/DeepNestingBP.lean`]

Let $a_0 \geq 1$, $0 \leq \gamma \leq 1$, $0 < \varepsilon \leq 1/8$, $\theta_0 = 2^{-(a_0+11)}$ and $\delta = 1/(48(a_0+1))$, and let $m < s_0$ satisfy $\varepsilon m \geq 4$ and $(*)$. Let ξ be an odd shift such that

(i) $\Gamma_{a,\pm}(\xi) \geq \gamma|\mathcal{K}|$ for all $1 \leq a \leq a_0$ and both signs, and

(ii) at most $\delta|\mathcal{F}_\varepsilon|$ triples (h,k,f) with $k \in \mathcal{F}_\varepsilon$ are not half-blind at ξ.

Then for all $1 \leq a_1, a_2 \leq a_0$, all $K_1, K_2 \in \mathbb{Z}$, every $q \geq 3$ and every multiple r of q, at least $c(m+1)-2$ levels are not doubly θ_0-served by the block-pair label of $(B(a_1,K_1), B(a_2,K_2))$ modulo q on the r-fold cover, where $c = \min(\gamma/8,\ \varepsilon/(26(a_0+1)))$.

Outline of the formal proof. Call a triple *bad* if it is not half-blind. As $\varepsilon m \geq 4$ and $\varepsilon \leq 1/8$, $|\mathcal{F}_\varepsilon| = \lfloor \varepsilon m \rfloor - 1 \geq \varepsilon m/2$ and $\mathcal{F}_\varepsilon \subseteq [1, m-2]$. We find levels that are not θ_0-served for some head, in three cases covering all $q \geq 3$.

Case 1: $q < 32$. Then $2\lfloor \log_2 q \rfloor + 2 \leq 10$. Split $\mathcal{F}_\varepsilon$ into $\lfloor |\mathcal{F}_\varepsilon|/11 \rfloor$ disjoint windows of 11 consecutive levels and a remainder. For a window $\{k_0, \dots, k_0+10\}$, the run bound with $s = 11$, applied to $B(a_1, K_1)$, fails: there are $0 \leq i \leq 10$ and a family f of the class $(1, k_0+i)$, the apex family or the band-j family with $i + j \leq 10$, whose blind error exceeds $q/8$; these quantities involve only the levels $k_0, \dots, k_0+10 \leq m-2$, so they are blind errors of actual hops (Lemma 7.7(b)). Distinct windows give distinct levels, and at most $\delta|\mathcal{F}_\varepsilon| \leq |\mathcal{F}_\varepsilon|/96$ of these triples are bad. If such a triple $(1,k,f)$ is half-blind and level k were θ_0-served for head 1, Lemma 7.7(c) would bound its blind error by $64\theta_0 q < 1$, so the error would be 0, not more than $q/8$. So at least $\lfloor |\mathcal{F}_\varepsilon|/11 \rfloor - |\mathcal{F}_\varepsilon|/96 \geq |\mathcal{F}_\varepsilon|/13 - 1$ levels are not θ_0-served for head 1.

Case 2: $q \geq 32$ and $4\tilde{Q}_h + 4 > q$ for some head h. The overflow bound for $B(a_h, K_h)$ gives rem $< a_h$, and each of the at least $\lfloor |\mathcal{F}_\varepsilon|/(a_h+1) \rfloor \geq |\mathcal{F}_\varepsilon|/(a_0+1) - 1$ levels $k \in \mathcal{F}_\varepsilon$ with $(m-k) \bmod (a_h+1) =$ rem has a family f of the class (h,k) (apex, band 0 or band 1) with blind error at least $q/16$. If (h,k,f) is half-blind and level k were θ_0-served for head h, Lemma 7.7(c) would bound this error by $64\theta_0 q = 2^{-(a_0+5)} q \leq q/64$. Discarding the levels of the at most $|\mathcal{F}_\varepsilon|/(48(a_0+1))$ bad triples leaves at least $|\mathcal{F}_\varepsilon|/(2(a_0+1)) - 1$ levels that are not θ_0-served for head h.

Case 3: $q \geq 32$ and $4\tilde{Q}_h + 4 \leq q$ for both heads. Suppose that $\tilde{Q}_1 \leq \tilde{Q}_2$. By (i), at least $\gamma|\mathcal{K}|$ levels $k \in \mathcal{K}$ are a_2-good in direction $+$. Fix one and let $\kappa = \kappa_{a_2}(k)$, a canceller of $B(a_2, \cdot)$ with $\kappa \leq m$ and $\Pi_\kappa \leq R_k$. Let Φ be the potential of U carrying $B(a_2, K_2)$, so that head 2 has potential $\Phi(t+\xi)$ up to a constant, and let $\alpha \in \mathcal{A}_\kappa$ be the apex of a level-κ V V. By the path sums [3, Lemma 5.9], for $0 \leq o < \Pi_\kappa$ the values $\Phi(\alpha+o)$ and $\Phi(\alpha+\Pi_\kappa+o)$ are a constant C_V (from the start of the level-0 stroke and the V's containing V), plus $c_{\mathrm{b}}^{(2)}(\kappa)$ or $c_{\mathrm{f}}^{(2)}(\kappa)$ (from V), plus the contribution $Y(o)$ of the V's inside the stroke of V at offset o, which is the same for both strokes because the increments of a block label depend only on the level [3, Lemma 5.12]. This is the decomposition in the proof of [3, Lemma 5.20], and it is the hypothesis of Theorem V*.

Let $\alpha_1 \in \mathcal{A}_k$, $c_0 = (\alpha_1 + \xi) \bmod P$ and $S = \mathrm{Strad}(c_0, \alpha, \Pi_\kappa, R_k)$. For $d \in S$, the apex hop $(\alpha_1 - d, \alpha_1 + d)$ of head 1 lies in the class $(1,k)$ with $d < R_k$, its other-head increment is $\Phi(c_0+d) - \Phi(c_0-d)$ because Φ has P-periodic increments, and so its error is $e(d) = (A_k^{(1)} + \Phi(c_0+d) - \Phi(c_0-d)) \bmod q$ [3, Lemma 5.10(a)]. Theorem V* with $A' = A_k^{(1)}$ and the residue bound give $\sum_{d \in S} e(d) \geq |S|\, q/4$.

For fixed α_1 at most one α has $S \neq \emptyset$, since $S \neq \emptyset$ puts c_0 in the span $[\alpha, \alpha + 2\Pi_\kappa)$ and the spans of distinct V's of one level are disjoint (`DirectedReciprocity.periodVApices_spans_disjoint` [9]); and distinct pairs (α_1, d) give distinct hops. Summing over α_1 and α, the hops of the class $(1, k)$ in one period have total error at least $\frac{q}{4} X^+_{k,\kappa}(\xi) \geq \frac{q}{4} \cdot 2^{k+2} R_k / 2^{a_2+7} = q\Lambda / 2^{a_2+7}$, as $2^k R_k = \Lambda$. Errors are P-periodic and the class has 4Λ hops per period, so its mean error is at least $q/2^{a_2+9} > \theta_0 q$: level k is not θ_0-served for head 1. This gives at least $\gamma|\mathcal{K}| \geq \gamma N/4 - 1$ levels.

If $\tilde{Q}_2 < \tilde{Q}_1$, exchange the heads: use the apex hops of head 2, centred at the times $(\alpha_1 - \xi) \bmod P$ of head 1, the level-$\kappa_{a_1}(k)$ V's of head 1, X^-, condition (i) with $a = a_1$, and the residue bound with the indices 1 and 2 exchanged.

In every case, since $|\mathcal{F}_\varepsilon| \geq \varepsilon m/2$, at least $\min(\varepsilon m/(26(a_0+1)),\ \gamma N/4) - 1 \geq c(m+1) - 2$ levels are not θ_0-served for one of the heads, hence not doubly θ_0-served. Formally, the three cases are `MergeDepth.case1_unserved`, `MergeDepth.case2_unserved` and `MergeDepth.case3_unserved`, through the tenth paper's run bound of Theorem LU, its overflow band witness and Theorem V*, and `MergeDepth.witnesses_unserved` combines them for service measured on the families listed in one period; a level that is doubly θ_0-served on a cover is served in this sense (`MergeDepth.ThetaServes.servedOnWitness`). □

7.4 Block pairs at one odd shift

Theorem 7.10 (Theorem BP: block pairs at one odd shift, assuming Conjecture S)**.** Lean: `MergeDepth.theoremBP_pair` [`MergeDepth/DeepNestingBP.lean`]

Let $a_0 \geq 1$, and assume that Conjecture S holds for $a = 1, \ldots, a_0$. Put $\theta_0 = 2^{-(a_0+11)}$ and $c = 2^{-16}/(26(a_0+1)^2)$. There is a constant C', depending only on a_0 (through the rate of convergence in Conjecture S), such that for all $m \geq 1$ and $s_0 > m$ some odd shift ξ has the following property: for all integers $1 \leq a_1, a_2 \leq a_0$ and K_1, K_2, every $q \geq 3$ and every multiple r of q, the block-pair label of $(B(a_1, K_1), B(a_2, K_2))$ modulo q on the r-fold cover of the pair $x_1 = U(s_0, m, 1)$, $x_2 = x_1(\cdot + \xi)$ doubly θ_0-serves at most $(1-c)(m+1) + C'$ levels.

Outline of the formal proof. Put $\gamma = 1/2$ and $\varepsilon = 2^{-16}/(a_0+1)$, so that $c = \min(\gamma/8, \varepsilon/(26(a_0+1)))$. By Proposition 7.5(c) there is m_0 such that for $m \geq m_0$ and $s_0 > m$ the size condition holds and condition (i) of Lemma 7.9 holds at a uniformly random odd shift with probability at least $1/2$. Let $m_1 = \max(m_0, 2^{18}(a_0+1))$ and $C' = \max(2, c(m_1+1))$. For $m \geq m_1$ and $s_0 > m$ we have $\varepsilon m \geq 4$, and by Lemma 7.8 condition (ii) fails with probability at most $6144(a_0+1)\varepsilon = 3/32 < 1/2$. So both conditions hold at some odd shift ξ, which depends only on a_0, m and s_0, and Lemma 7.9 leaves at most $(1-c)(m+1) + 2 \leq (1-c)(m+1) + C'$ doubly θ_0-served levels, for every block pair with $1 \leq a_1, a_2 \leq a_0$, every $q \geq 3$ and every multiple r of q. For $m < m_1$ every odd shift has the property, since $m + 1 < (1-c)(m+1) + c(m_1+1) \leq (1-c)(m+1) + C'$. Formally, the theorem is first proved for service measured on the families listed in one period (`MergeDepth.theoremBP'`), and `MergeDepth.theoremBP_pair` follows, since a level that is doubly θ_0-served on a cover is served in this sense; the hypothesis $q \mid r$ is not used. □

Theorem BP matters because block labels are how one head alone serves most of its levels (the fraction $a/(a+1)$ in the single-head regime [3, Lemma 5.12]). Assuming Conjecture S, it shows that at one fixed odd shift no pair of them with block lengths at most a_0, for any modulus, doubly θ_0-serves more than a fraction $1 - c$ of the levels, up to the additive constant C', as Conjecture A″ predicts for every label of cost at most $n_h/8$.

Remark 7.11 (Scope of Theorem BP)**.** In the notation of Conjecture A″, Theorem BP gives, for pairs of block labels of length at most a_0 on covers with $q \mid r$, the constants θ_0, $\eta = 1 - c$, $C = 0$

and C', without the cost hypothesis; labels with $q \le 2$ values cost at least $n_{\mathrm{h}}/2$ by Lemma Zq [3, Lemma 2.7], so $q \ge 3$ loses nothing. It leaves open Conjecture S; block lengths growing with q, such as $a \approx \frac{1}{2}\log_2 q$, where the straddle density 2^{-a} in Case 3 of the proof of Lemma 7.9 falls below every fixed θ_0; and general labels (step (d)). Even for a level-uniform label without cancellers the straddle residue $(2A_k + 2E_\kappa + 2) \bmod q$ can be small, so Case 3 does not apply. Theorem BP therefore does not decide TCP(2); it is evidence for depth-robustness from the labels whose cancellations are understood.

7.5 Partial results on Conjecture S

These results are not used in the proof of Theorem BP. Throughout, $\operatorname{sinc} y = \sin(\pi y)/(\pi y)$ with $\operatorname{sinc} 0 = 1$, and for $1 \le k < \kappa \le m$ and $r \in \mathbb{Z}$

$$\varphi_k(r) = \operatorname{sinc}^4\Big(\frac{rn_\kappa}{N2^{\kappa+1}}\Big)\prod_{i=1}^{k-1}\cos^4\Big(\frac{\pi r n_i}{N2^i}\Big)\prod_{i=k}^{\kappa-1}\cos^2\Big(\frac{\pi r n_i}{N2^i}\Big).$$

Proposition 7.12 (Exact form of V_k)**.** Lean: `MergeDepth.exact_form_V`, `MergeDepth.density_W`, `MergeDepth.sideband_eq_densityW` [`MergeDepth/StraddleDensity.lean`]
Let $1 \le k < \kappa \le m$.

(a) $V_k = \sum_{r\neq 0}\varphi_k(r)$.

(b) Let $\zeta_1, \dots, \zeta_{\kappa-1}, \omega_1, \dots, \omega_4$ be independent, with $\zeta_i + 2 \sim \mathrm{Bin}(4, \frac{1}{2})$ for $i < k$, $\zeta_i + 1 \sim \mathrm{Bin}(2, \frac{1}{2})$ for $k \le i < \kappa$, and ω_j uniform on $[-\frac{1}{4}, \frac{1}{4}]$, and let $W = \sum_{i<\kappa}\zeta_i\hat{\Pi}_i + Z$ with $Z = \hat{\Pi}_\kappa(\omega_1 + \cdots + \omega_4)$. Then W has a continuous density f_W, and $1 + V_k = N\sum_{l\in\mathbb{Z}} f_W(lN)$.

(c) $V_k \ge F_k := \sum_{t\neq 0}\varphi_k(N2^{k-1}t)$, and $F_k = F(n_\kappa, \kappa - k)$, where

$$F(n, D) = \sum_{t\neq 0}\operatorname{sinc}^4(tn2^{-D-2})\prod_{e=1}^{D}\cos^2(\pi t(n + D + 1 - e)2^{-e}).$$

Outline of the formal proof. (a) With $\Delta_N(y) = \sum_l \Delta(y - lN)$, $x_k(t) = (2^{k-1}\hat{\Pi}_\kappa)^{-1}\sum_{S,S'}\Delta_N(t + \hat{a}_S - \hat{v}_{S'})$, so the Fourier coefficients $\hat{x}(r) = N^{-1}\int_0^N x_k(t)e^{-2\pi i r t/N}dt$ are

$$\hat{x}(r) = \frac{\hat{\Delta}(r/N)}{2^{k-1}\hat{\Pi}_\kappa N}\sum_S e^{2\pi i r\hat{a}_S/N}\sum_{S'} e^{-2\pi i r\hat{v}_{S'}/N},$$

with $\hat{\Delta}(\eta) = \int_{\mathbb{R}}\Delta(y)e^{-2\pi i\eta y}dy = \frac{1}{2}\hat{\Pi}_\kappa^2\operatorname{sinc}^2(\hat{\Pi}_\kappa\eta/2)$ by direct integration. As $\hat{a}_S$ is a constant plus $\sum_{i\in S}\hat{\Pi}_i$, $|\sum_S e^{2\pi i r\hat{a}_S/N}| = \prod_{i<k}|1 + e^{2\pi i r\hat{\Pi}_i/N}| = 2^{k-1}\prod_{i<k}|\cos(\pi r n_i/(N2^i))|$, and likewise for the valleys with $\kappa - 1$ factors. Hence $|\hat{x}(r)|^2/\hat{x}(0)^2 = \varphi_k(r)$, the factors with $i < k$ occurring in both sums; $\hat{x}(0) = \mu_k$, and Parseval's identity $\operatorname{Var} x_k = \sum_{r\neq 0}|\hat{x}(r)|^2$ gives (a).

(b) For $\zeta + 2 \sim \mathrm{Bin}(4, \frac{1}{2})$, $\mathbb{E}e^{2\pi i y\zeta} = e^{-4\pi i y}((1 + e^{2\pi i y})/2)^4 = \cos^4(\pi y)$; for $\zeta + 1 \sim \mathrm{Bin}(2, \frac{1}{2})$ it is $\cos^2(\pi y)$; for ω uniform on $[-\frac{1}{4}, \frac{1}{4}]$ it is $\operatorname{sinc}(y/2)$. By independence, $\psi(y) = \mathbb{E}e^{2\pi i y W}$ is the product of these factors at $y\hat{\Pi}_i$ and $y\hat{\Pi}_\kappa$, and $\psi(r/N) = \varphi_k(r)$ for $r \in \mathbb{Z}$. Let f_W be the mixture, with the probabilities of the values of $(\zeta_1, \dots, \zeta_{\kappa-1})$ as weights, of the translates by $\sum_i \zeta_i\hat{\Pi}_i$ of the density of Z (a constant times the convolution square of Δ). It is continuous, nonnegative and compactly supported, and its Fourier transform is ψ, so it is the density of W. Poisson summation

for $y \mapsto f_W(Ny)$, whose Fourier transform at $r \in \mathbb{Z}$ is $\psi(r/N)/N = \varphi_k(r)/N$ (φ_k is even) and is summable as $\operatorname{sinc}^4 y \le \min(1, (\pi y)^{-4})$, gives $N \sum_l f_W(lN) = \sum_r \varphi_k(r) = 1 + V_k$ by (a).

(c) All terms in (a) are nonnegative. At $r = N2^{k-1}t$ the factors with $i < k$ equal 1, the factor at $k \le i < \kappa$ is $\cos^2(\pi t n_i 2^{k-1-i})$, and the sinc factor is $\operatorname{sinc}^4(t n_\kappa 2^{k-\kappa-2})$; with $D = \kappa - k$, $e = i - k + 1$ and $n_i = n_\kappa + D + 1 - e$, this is the term t of $F(n_\kappa, D)$. Formally, φ_k is `MergeDepth.straddlePhi`; (a) is `MergeDepth.hasSum_straddlePhi_ne_zero`, through Mathlib's Parseval identity on the circle of length N, and holds for $1 \le k \le \kappa$ (for every $\kappa \ge 1$ in the general form `MergeDepth.hasSum_phiGen`); (c) is `MergeDepth.fineFloorK_le_relVariance` and `MergeDepth.fineFloorK_eq`, and only the identity $F_k = F(n_\kappa, \kappa - k)$ uses $\kappa \le m$; `MergeDepth.exact_form_V` collects (a) and (c). Part (b) is `MergeDepth.density_W`, for independent random variables with these laws on any probability space (they exist, `MergeDepth.density_W_model_exists`); it needs only $1 \le \kappa$ and $k \le \kappa$, the law of W is identified through characteristic functions, and the identity $\mathbb{E}\exp(2\pi i r W/N) = \varphi_k(r)$ is `MergeDepth.charFun_lawW_eq_straddlePhi`. □

So $1 + V_k$ is N times the density at 0 of a random sum of the digits $\hat{\Pi}_i$ modulo N, smoothed at the scale $\hat{\Pi}_\kappa$. For $k \in \mathcal{K}$ and $\kappa = \kappa_a(k)$, splitting the frequencies $r \ne 0$ into the multiples of $N2^{k-1}$, the *semi-local* ones $r = N2^{k-h}s$ (s odd, $2 \le h \le k$) and the *sidebands* $r \notin N\mathbb{Z}$ gives $V_k = F_k + \sum_{h=2}^{k} SL_k^{[h]} + SB_k$ with $SL_k^{[h]} = \sum_{s \text{ odd}} \varphi_k(N2^{k-h}s)$ and $SB_k = \sum_{r \notin N\mathbb{Z}} \varphi_k(r)$, all nonnegative (formally `MergeDepth.relVariance_decomp`, for $1 \le k \le \kappa$). The argument of (b) with period 1 gives $\sum_w \varphi_k(Nw) = \sum_l f_W(l)$, so $SB_k = N \sum_l f_W(lN) - \sum_l f_W(l)$ (`MergeDepth.sideband_eq_densityW`) measures how far the law of W modulo N is from invariance under integer translations.

Proposition 7.13 (Classes with a large fine floor)**.** `Lean`: `MergeDepth.resonance_class` [`MergeDepth/StraddleResonance.lean`]

Let $a = 1$, let $q \ge 6$ be even, let $k \ge 1$ with $3k \le 2^q + 4$, and let $m = 2^q + k + 3$. Then $k \in \mathcal{K}$, $\kappa_1(k) = k + q + 2$, $n_{\kappa_1(k)} = 2^q - q + 2$, and $V_k^{(1)} \ge F_k > 0.585$.

Outline of the formal proof. Here $N = 2^q + k + 4$ and $k + 4 \le (2^q + 16)/3 < 2^q$, so $j_0 = q + 1$, and $4k \le N$, so $k \in \mathcal{K}$. As $m - k - j_0 = 2^q - q + 2$ is even, the least $\kappa \ge k + q + 1$ with $m - \kappa$ odd is $k + q + 2$, so $n_\kappa = 2^q - q + 2$ and $D = q + 2$. By Proposition 7.12(c), $V_k^{(1)} \ge F_k = F(n_\kappa, q + 2)$, whose cosine factors are $\cos^2(\pi t(2^q + 5 - e)2^{-e})$ for $1 \le e \le q + 2$. Keep only the two equal terms $t = \pm 4$. The sinc factor is $\operatorname{sinc}^4(y)$ with $y = (2^q - q + 2)2^{-q-2} \in (0, \frac{1}{4}]$, so it is at least $\operatorname{sinc}^4(\frac{1}{4}) = (2\sqrt{2}/\pi)^4 > 0.65702$. For $t = 4$ the factor of index e is $\cos^2(\pi(2^q + 5 - e)2^{2-e})$: it is 1 for $e \in \{1, 2, 3, 5\}$; $\frac{1}{2}$ for $e = 4$; and $\cos^2(\pi(e - 5)2^{2-e})$ for $6 \le e \le q + 2$, since 2^{q+2-e} is an integer. These factors lie in $[0, 1]$, so their product is at least $\prod_{e \ge 6} \cos^2(\pi(e-5)2^{2-e}) \ge \cos^4(\pi/16)\cos^2(3\pi/64)(1 - \pi^2 \sum_{j \ge 4} j^2 4^{-j-3}) > 0.89142$, using the factors $e = 6, 7, 8$ exactly, $\cos^2 y \ge 1 - y^2$ and $\prod(1 - y_e) \ge 1 - \sum y_e$ for $e \ge 9$, and $\sum_{j \ge 4} j^2 4^{-j} = \frac{20}{27} - \frac{41}{64}$. Hence $F_k \ge 2 \cdot 0.65702 \cdot \frac{1}{2} \cdot 0.89142 > 0.585$. Formally, the numerical bounds come from the exact value of $\cos(\pi/8)$, the bound $\cos y \ge 1 - y^2/2$ and Mathlib's bounds on π. The bound on F_k does not use that q is even; evenness gives $\kappa_1(k) = k + q + 2$. □

So $V_k^{(1)} \not\to 0$ along classes $k \to \infty$ (this family has at most one class per m), and variance bounds for single classes cannot show that every coarse class is good; Conjecture S averages over classes, as Proposition 7.5(b) allows.

Arithmetic of κ_a. Let $a \ge 1$ with $4(j_0 + a) \le N$, and for $k \in \mathcal{K}$ let $\rho(k) = (m - k - j_0) \bmod (a+1)$ and $\ell_k = \lfloor (m - k - j_0 + 1)/(a+1) \rfloor$. Then $\kappa_a(k) = k + D(k)$ with $D(k) = j_0 + ((\rho(k) + 1) \bmod (a+1)) \in [j_0, j_0 + a]$, which depends only on $\rho(k)$, and $n_{\kappa_a(k)} = (a + 1)\ell_k$. The map $k \mapsto (\rho(k), \ell_k)$ is injective on $\mathcal{K}$, and the ℓ_k lie in an interval I of integers with $(a + 1)|I| \le \lfloor N/4 \rfloor + 2a \le N$.

Moreover $2^{D(k)} \geq 2^{j_0} > N$, and $N \leq 2n_{\kappa_a(k)}$ and $2^{j_0} \leq 2N$ give $2^{D(k)} \leq 2^{a+2} n_{\kappa_a(k)}$, that is, $n_{\kappa_a(k)} 2^{-D(k)-2} \geq 2^{-a-4}$.

Lemma 7.14 (Dyadic products along a progression)**.** Lean: `MergeDepth.window_sum_prod_cos_sq`, `MergeDepth.progression_avg_le_strong` [`MergeDepth/StraddleProgression.lean`]
Let $j \geq 0$.

(a) If u is an odd integer and $\gamma_1, \dots, \gamma_j$ are real, then for every integer ℓ_0

$$\sum_{\ell=\ell_0}^{\ell_0+2^j-1} \prod_{s=1}^{j} \cos^2\bigl(\pi(u\ell 2^{-s} + \gamma_s)\bigr) = 1.$$

(b) Let $a+1 = 2^w o$ and $t = 2^v t'$ with o, t' odd and $w, v \geq 0$, let c_e be real numbers for $v+w < e \leq v+w+j$, and let I be a set of $L \geq 1$ consecutive integers. Then

$$\frac{1}{L} \sum_{\ell \in I} \prod_{e=v+w+1}^{v+w+j} \cos^2\bigl(\pi t((a+1)\ell + c_e) 2^{-e}\bigr) \;\leq\; 2^{-j} + \frac{1-2^{-j}}{L}.$$

Outline of the formal proof. (a) For $j = 0$ the sum has the single term 1. For $j \geq 1$, replacing ℓ by $\ell + 2^{j-1}$ shifts the arguments with $s < j$ by the integer multiples $u 2^{j-1-s}\pi$ of π and the argument with $s = j$ by $\pi u/2$, which turns $\cos^2$ into $\sin^2$ as u is odd. So pairing ℓ with $\ell + 2^{j-1}$ for $\ell_0 \leq \ell < \ell_0 + 2^{j-1}$ turns the last factor into $\cos^2 + \sin^2 = 1$ and reduces the sum to the same sum for $j-1$.

(b) Put $u = t'o$, which is odd, and $e = v + w + s$; then $t(a+1)2^{-e} = u2^{-s}$, and the product is $\prod_{s=1}^{j} \cos^2(\pi(u\ell 2^{-s} + \gamma_s))$ with $\gamma_s = t c_e 2^{-e}$. Split I into $\lfloor L/2^j \rfloor$ runs of 2^j consecutive integers and a rest of fewer than 2^j. By (a) each run contributes exactly 1, and the rest at most 1, as the terms are nonnegative; so the sum is at most $\lceil L/2^j \rceil \leq L2^{-j} + 1 - 2^{-j}$. Formally, (a) is `MergeDepth.window_sum_prod_cos_sq` and (b) is `MergeDepth.progression_avg_le_strong`. □

Proposition 7.15 (Fine floors and near semi-local frequencies)**.** Lean: `MergeDepth.sum_fineFloorK_le`, `MergeDepth.sum_semiLocal_le`, `MergeDepth.relVariance_decomp`, `MergeDepth.conjectureS_iff_meanNonLocalGen` [`MergeDepth/StraddleLocal.lean`]
Let $a \geq 1$, for $k \in \mathcal{K}$ let $\kappa = \kappa_a(k)$, and put $C_a = (a+1)\bigl(3(2^{a+4}/\pi)^2 + 8 \cdot 2^{a+4}/(3\pi)\bigr)$.

(a) If $4(j_0 + a) \leq N$, then $\sum_{k \in \mathcal{K}} F_k \leq C_a$.

(b) If $4(j_0 + a) \leq N$, then $\sum_{k \in \mathcal{K}} \sum_{h=2}^{\min(H,k)} SL_k^{[h]} \leq (a+1)2^{a+H+8}/(3\pi)$ for every integer $H \geq 0$.

(c) Let $H = H(m) \geq 0$ be integers with $2^H/N \to 0$ as $m \to \infty$, and put $NL_k = V_k - F_k - \sum_{h=2}^{\min(H,k)} SL_k^{[h]}$, which is $SB_k + \sum_{h=\max(2,H+1)}^{k} SL_k^{[h]}$. Then Conjecture S holds for a if and only if $|\mathcal{K}|^{-1} \sum_{k \in \mathcal{K}} NL_k \to 0$ as $m \to \infty$.

Outline of the formal proof. Write $a+1 = 2^w o$ with o odd, and let I be the interval of the arithmetic above, so that $2^w |I| \leq (a+1)|I| \leq N$.

(a) By the arithmetic above, $F_k = F((a+1)\ell_k, D(k))$ with $n_\kappa 2^{-D(k)-2} \geq 2^{-a-4}$. As $|\operatorname{sinc} y| \leq \min(1, 1/(\pi|y|))$, the sinc factor of the term t of F_k is at most $\operatorname{sb}(t) = \min(1, (\pi|t|2^{-a-4})^{-4})$, so $F_k \leq M(\ell_k, D(k))$ with $M(\ell, D) = \sum_{t \neq 0} \operatorname{sb}(t) \prod_{e=1}^{D} \cos^2(\pi t((a+1)\ell + D + 1 - e)2^{-e}) \geq 0$. As $k \mapsto (\rho(k), \ell_k)$ is injective, $\sum_{k \in \mathcal{K}} F_k \leq \sum_{\rho \leq a} \sum_{\ell \in I} M(\ell, D(\rho))$. Fix $D = D(\rho)$ and $t \neq 0$, and write $t = 2^v t'$ with t' odd. If $v + w < D$, drop the cosine factors with $e \leq v + w$, which lie in $[0, 1]$,

and apply Lemma 7.14(b) with $j = D - v - w$ and $c_e = D + 1 - e$: the sum over $\ell \in I$ of the cosine products is at most $|I|2^{v+w-D} + 1$. If $v + w \geq D$, it is at most $|I| \leq |I|2^{v+w-D}$. As $2^v \leq |t|$ and $2^w|I| \leq N < 2^D$, it is at most $|t| + 1$ in both cases. So $\sum_{\ell \in I} M(\ell, D) \leq \sum_t (|t| + 1)\,\mathrm{sb}(t)$, and telescoping bounds give $\sum_t |t|\,\mathrm{sb}(t) \leq 3(2^{a+4}/\pi)^2$ and $\sum_t \mathrm{sb}(t) \leq 8 \cdot 2^{a+4}/(3\pi)$.

(b) Fix $2 \leq h \leq H$, and for $k \in \mathcal{K}$ with $k \geq h$ put $E = D(k) + h - 1$. At $r = N2^{k-h}s$ with s odd, the factors of φ_k with $i \leq k - h$ equal 1. Bound the $\cos^4$ factors by $\cos^2$, keep only the factors with $i = k - h + l$, $w < l \leq E$, which are $\cos^2(\pi s(n_\kappa + E + 1 - l)2^{-l})$, and bound the sinc factor $\mathrm{sinc}^4(sn_\kappa 2^{-D(k)-h-1})$ by $\min(1, (\pi|s|2^{-a-3-h})^{-4})$. As in (a), pass to the pairs (ρ, ℓ): Lemma 7.14(b) with $t = s$, $v = 0$ and $j = E - w$ bounds the sum over $\ell \in I$ of the kept products by $|I|2^{w-E} + 1 \leq 2$, as $2^E \geq 2^{j_0} > N$, and the sinc bounds sum to at most $8 \cdot 2^{a+3+h}/(3\pi)$. So $\sum_{k \in \mathcal{K},\, k \geq h} SL_k^{[h]} \leq (a+1)2^{a+7+h}/(3\pi)$, and summing over $2 \leq h \leq H$ gives the bound.

(c) The second form of NL_k follows from the decomposition $V_k = SB_k + F_k + \sum_{h=2}^{k} SL_k^{[h]}$. For all large m the size condition holds, and then (a) and (b) give

$$0 \leq \bar{V}_m^{(a)} - \frac{1}{|\mathcal{K}|}\sum_{k \in \mathcal{K}} NL_k = \frac{1}{|\mathcal{K}|}\sum_{k \in \mathcal{K}} \Big(F_k + \sum_{h=2}^{\min(H,k)} SL_k^{[h]}\Big) \leq \frac{C_a + (a+1)2^{a+H+8}/(3\pi)}{|\mathcal{K}|},$$

which tends to 0 as $|\mathcal{K}| = \lfloor N/4 \rfloor$ and $2^H/N \to 0$; so the two averages tend to 0 together. Formally, (a), (b) and (c) are `MergeDepth.sum_fineFloorK_le`, `MergeDepth.sum_semiLocal_le` and `MergeDepth.conjectureS_iff_meanNonLocalGen`, and the decomposition is `MergeDepth.relVariance_decomp`; for $H = \lfloor \frac{1}{4}\log_2 N \rfloor$, (c) is also `MergeDepth.conjectureS_iff_meanNonLocal`. □

So the fine floors and the semi-local frequencies with $2^h = o(N)$, for instance $h \leq \lfloor \log_2 N \rfloor - \lceil \log_2 \log_2 N \rceil$, contribute $o(1)$ to $\bar{V}_m^{(a)}$, and Conjecture S is a statement about the sidebands and the semi-local frequencies with 2^h at least of order N. By (a), at most $2C_a$ coarse classes have $F_k \geq 1/2$, for every m with $4(j_0 + a) \leq N$; the classes of Proposition 7.13 are of this kind.

Remark 7.16 (The carry chain)**.** Proof sketch only: a representation for the non-local part. Let $b_1, \ldots, b_{\kappa-1}$ be the binary digits of $\sum_{i<\kappa} \zeta_i 2^{-i} \bmod 1$, and define carries by $\varrho_\kappa = 0$ and $\varrho_i = (\zeta_i + \varrho_{i+1} - b_i)/2$ for $i = \kappa - 1, \ldots, 1$. Each ζ_i is odd with probability $\frac{1}{2}$, so the b_i are independent fair bits, and given them the carries, which stay in $\{-2, \ldots, 2\}$, form a Markov chain run downwards from level $\kappa - 1$. Telescoping gives $\sum_i \zeta_i \hat{\Pi}_i \equiv \sum_i b_i \hat{\Pi}_i - 2\varrho \pmod N$ with $\varrho = \sum_i \varrho_i 2^{-i}$, so $1 + V_k = N\,\mathbb{E}\big[\sum_l f_Z\big(\sum_i b_i \hat{\Pi}_i - 2\varrho - lN\big)\big]$, where f_Z is the density of Z and $\sum_i b_i \hat{\Pi}_i$ is a uniformly random valley position of the comb. The difficulty is the correlation of the carries with the leading bits, the mixed-radix carry problem that also blocked the fixed-shift approach of [3, Section 5.6]: the carries forget distant bits quickly (Remark 7.17), but a proof needs near-invariance of the law of W modulo N under integer translations, which SB_k measures.

Remark 7.17 (Numerical: Conjecture S)**.** The values below were computed from Proposition 7.12(b) by propagating the exact law of the partial sums $\sum_{i \leq j} \zeta_i \hat{\Pi}_i$ level by level ($O(\kappa N)$ floating-point operations per class, no sampling); where both were run, this agrees with direct summation of Proposition 7.12(a) to four or five decimals.

(1) For $a = 1$, $\bar{V}_m^{(1)} = 0.268,\ 0.134,\ 0.061,\ 0.0345,\ 0.0189,\ 0.0114$ at $m = 127,\ 255,\ 511,\ 1023,\ 2047,\ 4095$; at the eleven values of m computed between 127 and 4095, $m\bar{V}_m^{(1)}$ lies between 30 and 47, consistent with $\bar{V}_m^{(1)} = O(\log m/m)$. For $a = 2$ and $a = 3$, $\bar{V}_m^{(a)} = 0.126,\ 0.058,\ 0.031,\ 0.021$ and $0.139,\ 0.064,\ 0.030,\ 0.018$ at $m = 255,\ 511,\ 1023,\ 2047$.

(2) The sidebands of the first classes carry most of $\bar{V}$: for $a = 1$ and $m \geq 511$ the averages of F_k and $\sum_h SL_k^{[h]}$ are at most 0.0017 and 0.0044, while $\sum_k SB_k \in [6.5, 10.5]$ for all eleven m; at $m = 2047$, $SB_k \approx 1.68$, 0.28, 0.024, 0.0017, 10^{-5} at $k = 1$, 8, 16, 24, 40.

(3) By concavity of $v \mapsto v/(v+s)$, Proposition 7.5(a) gives $\beta_a \leq \bar{V}_m^{(a)}/(\bar{V}_m^{(a)} + (1-\theta^*)^2) + 2^{3+a_0} N 2^{-3N/4}$ with $\theta^* = \max_k \theta_k \approx 0.23$, so $\beta_1 \leq 0.055$, 0.031, 0.019 at $m = 1023$, 2047, 4095; at $m = 1023$, 2047 the event of Proposition 7.5(c) has probability at least $\frac{1}{2}$ for all $s_0 > m$, for $(a_0, \gamma) = (2, \frac{1}{2})$ ($\beta_1 + \beta_2 \leq 0.105$, 0.065) and $(3, \frac{1}{4})$ ($\beta_1 + \beta_2 + \beta_3 \leq 0.154$, 0.094).

(4) At the classes of Proposition 7.13, $F_k = 0.641$, 0.604, 0.593, 0.590 for $q = 6$, 8, 10, 12; at $(m, k) = (85, 18)$ and $(89, 22)$ the class is not 1-good at about 24–29% of sampled shifts.

(5) By exact enumeration, two copies of the carry chain of Remark 7.16 driven by the same bits coalesce within two levels $i < k$ with probability at least 5/16, whatever the bits and carries; so if $p + 2j < k$, the bits b_i with $i > p + 2j$ move the law of the carries at the levels $i \leq p$ by at most $(11/16)^j$ in total variation.

8 What a linear-time transposer would need

This section collects the necessary conditions that the series now places on a linear-time transposer and states the constructive target that remains. It proves nothing new except Lemma 8.2; its purpose is to say where a construction would have to live.

Unconditional constraints on the motion. A linear-time transposer needs at least two work tapes, since with one work tape transposition takes superlinear time [4, Corollary 3.18(c)]; much stronger bounds hold in the classical one-tape models [13, 12]. Its heads must make $\Omega(\log d)$ passes, by the first paper's pass bound, Lean-verified there [6, Theorems 5.9 and 5.12], and conservative machines with $o(\log d)$ reversals cannot transpose at all [15]. Among oblivious geometries with bounded span overlap, the ninth paper excludes those with boundedly many large return families, and those whose families form a Q_2 core and at most one further family [11, Corollaries 4.3 and 4.5]; with $q \geq 2$ exceptional families, the sorted hop fractions h_i of every near-full sequence satisfy $\sum_i i^{q-1} h_i = \infty$ [11, Theorem 4.2], and the frozen gap-area certificate cannot exclude the boundary case of two exceptional families with hop counts of order L/i^2 at moduli comparable to the protected rank [5]. By Theorem 6.1, a machine is excluded as soon as, for a small enough $\eta > 0$, one of its traces of probability at least $2^{-\eta L}$ falls under these exclusions. By the fourth paper's Theorem Cq, in every macroscopic limit the work heads travel at least half a block unit along motion that is not countably-piecewise affine [4, Theorem 3.12].

Unconditional constraints on the coding. By the first paper, every linear-time transposer carries, on the block network of its run, a causal code that solves complete exchange on a large set of inputs and beats multicommodity flow by a factor $\Omega(\log d)$ [6, Theorems 8.8 and 9.2], Lean-verified there. By Theorem 6.4, every linear-time transposer, adaptive or not, yields an external-memory transposition with linearly many I/Os that is oblivious on a set of $2^{L-O(d)}$ inputs, and that algorithm must combine records (Remark 6.6). An oblivious transposer whose code has bounded merge depth carries, by Theorem 3.11(b), an index code whose fibres are storage codes of rate close to one on relations inside balls of bounded radius, and these relations are paired-distance separated after a paid trimming (Lemma 3.15). They must therefore lie outside every class on which the half bound is proved: the degree-three, circular-chromatic and directed classes of the seventh and

tenth papers [7, 9], and the hyperfinite and two-input relations of the fifth [10, Corollary 4.4 and Theorem 5.1].

Constraints given [Q9.1]. Some tape is revisited unboundedly often (Corollary 4.17). If the work tapes have bounded entry, input cells are revisited unboundedly often and the input head carries about half of the input in its coarse motion at every fixed scale (Corollary 4.16); padding meets the second condition at no cost (Remark 4.18), so it shapes a construction only through the revisits. If the machine is oblivious, it revisits a work tape unboundedly often, and for all fixed V, s and ε and every $\eta < \frac{1}{2} - \varepsilon$, more than ηL of its outputs have, whatever the pruning, cold genie at (V, ε) and lineage assignment, a genie-free path with more than s deep merges (Corollary 3.16 and Theorem 5.1). Proposition 3.21 excludes in the same way every code whose merge DAG has a cheap cut, and Conjecture 3.22 would exclude all remaining codes (Remark 3.23). Question 9.1 has a negative answer [8, Theorem A], so these constraints, and the target below, rest on the native hypotheses that the proofs use (Section 2), which remain open.

The target. Two forms of a construction remain open.

(i) *Merge depth.* An oblivious machine that revisits some work-tape cells unboundedly often and whose code merges deep lineages unboundedly often, along merge DAGs without cheap cuts. After the cold genie, the genie-free paths from almost every output to its demanded input bit use many deep hops, so routing through the deep hops delivers little (Proposition 3.17); heuristically, the code must beat routing on them by a factor that grows with the radius (Remark 3.23(d)).

(ii) *Adaptive revisits.* An adaptive machine that revisits a work tape unboundedly often in a data-dependent way, or one whose work tapes have bounded entry and which revisits input cells unboundedly often; in the second case Corollary IS puts about $L/2$ bits of the data into the coarse motion of the input head, which padding provides at no cost.

In both forms the block-scale image is an external-memory transposition with linearly many I/Os that is oblivious on a large set of inputs (Theorem 6.4), so both need, at block scale, a code that beats every indivisible algorithm by a logarithmic factor. No mechanism of either kind is known. Coding gains over multicommodity flow for multiple unicast that grow with the size of an undirected network exist [2], but the known growing gains are far below logarithmic, and none is known on networks of the shape that tapes produce.

A ruled-out building block. Without the native hypotheses, an oblivious transposer whose work heads enter every cell boundedly often, or whose code has bounded merge depth, is not excluded by the results above; it carries storage codes of rate close to one on its cone relations or on relations inside balls, as above. Question 9.1 no longer stands in the way: the thirteenth paper answers it negatively [8, Theorem A], but its codes are not shown to have rate near one, their guaranteed rates approaching $4 - 2\sqrt{3} \approx 0.536$ [8, Theorem A*], its relations have astronomically large degree and are not known to arise from tape machines, and it constructs no transposer. The rest of this section rules out one natural way to assemble such codes: Tanner codes built from projective planes, with every coordinate recovered from one line through it, violate the paired-distance condition.

Codes assembled from projective planes. For an oblivious transposer whose work heads enter every cell boundedly often, Theorem F of the eighth paper, with its conversion of cuts into

genies, gives for every $\varepsilon > 0$ a genie of cost at most εL and bounded full depth, and some fibre of the revealed word is a storage code of rate at least $1 - \varepsilon$ on the cone relation [3, proofs of Corollaries 6.1 and 6.3]. After a paid trimming, the cone relations of linear-time schedules, for genies of bounded full depth, have bounded incidence degree and satisfy the paired-distance condition [10, Definition 2.3, Theorem 3.3 and Corollary 3.8]. The same holds for an oblivious transposer whose code has bounded merge depth, with the relations inside balls of Theorem 3.11(b) and Lemma 3.15 in place of the cone relations. A construction of either kind therefore needs storage codes of rate close to one on relations of bounded incidence degree with the paired-distance condition. Projective planes are natural building blocks. For $q = 2^s$, the binary words on the points of $\mathrm{PG}(2, q)$ that have even weight on every line form a code of rate $1 - (3^s + 1)/(q^2 + q + 1)$, which tends to one, because the point–line incidence matrix of $\mathrm{PG}(2, 2^s)$ has rank $3^s + 1$ over $\mathbb{F}_2$, a classical fact; and each point is recovered from the other q points of any line through it. If every coordinate lies in one such local code and is recovered inside it, the incidence graph has components of at most $2(q^2 + q + 1)$ vertices, and the paired-distance condition limits every code to rate $\frac{1}{2} + o(1)$ [10, Theorem 4.2]. The Tanner form of the assembly avoids this. Its coordinates are the edges of a $(q^2 + q + 1)$-regular graph G, the edges at each vertex are identified with the points of a copy of $\mathrm{PG}(2, q)$, a word is a codeword when its restriction to every vertex star is a codeword of the plane, which gives rate at least $1 - 2(3^s + 1)/(q^2 + q + 1)$, and each coordinate is recovered at one of its two endpoints u from the other points of one line of the plane at u.

Lemma 8.2 below rules this assembly out for every $q \geq 7$, whatever the graph and the choices of endpoints and lines. The coordinates recovered at a vertex u, each with its line, form a centred set S_u in the sense of Definition 8.1, and its incidence graph $E(S_u)$ is a subgraph of the paired incidence graph of the whole recovery relation. If S_u is not separated, some coordinate recovered at u therefore has its two ports at distance at most $2(q^2 + q + 1)$. Every coordinate is recovered at exactly one endpoint, so the sizes $|S_u|$ average $(q^2 + q + 1)/2$ over the vertices, while Lemma 8.2 bounds every separated centred set by $\sigma_q = \max(4q, q^{3/2} + 1) < (q^2 + q + 1)/2$. Hence at least a fraction $\kappa_q = \big((q^2 + q + 1)/2 - \sigma_q\big)/(q^2 + q + 1 - \sigma_q) > 0$ of the vertices carry a centred set that is not separated, at least a fraction $2\kappa_q/(q^2 + q + 1)$ of all coordinates have their two ports at bounded distance, and the paired-distance condition fails. The lemma concerns recovery along single lines; recovery through sums of several lines, and assemblies whose local pieces have unequal sizes, are not covered.

Definition 8.1 (Centred sets in a projective plane)**.** Lean: `MergeDepth.Centred`, `MergeDepth.Centred.graph`, `MergeDepth.Centred.Separated`, `MergeDepth.Centred.BipCompat` [`MergeDepth/ProjectivePlane.lean`]

Let $q \geq 2$, let $\mathcal{P}$ and $\mathcal{L}$ be the sets of points and lines of a finite projective plane of order q, such as $\mathrm{PG}(2, q)$ for a prime power q, and put $v = q^2 + q + 1 = |\mathcal{P}| = |\mathcal{L}|$. A *centred set* is a set $S \subseteq \mathcal{P}$ together with a line ℓ_s through s for every $s \in S$; the *helpers* of s are the q points of $N(s) = \ell_s \setminus \{s\}$. Its *incidence graph* $E(S)$ is the bipartite graph with vertices L_p and R_p for $p \in \mathcal{P}$ and an edge $L_h R_s$ whenever $s \in S$ and $h \in N(s)$; it is the paired incidence graph of [10, Definition 2.3] for the relation that recovers each $s \in S$ from $N(s)$. The centred set is *separated* if L_s and R_s lie in different components of $E(S)$ for every $s \in S$, and *bipartite-compatible* if some colouring $\chi \colon \mathcal{P} \to \{0, 1\}$ has $\chi(h) \neq \chi(s)$ whenever $s \in S$ and $h \in N(s)$.

Lemma 8.2 (Lemma PG: separated centred sets are small)**.** Lean: `MergeDepth.Centred.separated_of_bipCompat`, `MergeDepth.Centred.card_le`, `MergeDepth.Centred.two_mul_card_lt` [`MergeDepth/ProjectivePlane.lean`]

Let S be a centred set in a finite projective plane of order q. If S is bipartite-compatible, then it is

separated. If S is separated, then

$$|S| \le \max(4q,\ q^{3/2}+1), \tag{4}$$

and consequently $|S| < (q^2+q+1)/2$ for every $q \ge 7$.

Outline of the formal proof. We use only these properties of a projective plane of order q: two distinct lines meet in exactly one point, every line has $q+1$ points, every point lies on $q+1$ lines, and there are $v = q^2+q+1$ points. Formally, the first assertion is `MergeDepth.Centred.separated_of_bipCompat`, which uses no property of the plane; the bound (4) is stated as $|S| \le 4q$ or $(|S|-1)^2 \le q^3$ (`MergeDepth.Centred.card_le`); the components below are the classes of points of S whose vertices R_s are connected in $E(S)$, which are the components of the adjacency relation; Steps 1 to 3 are `MergeDepth.Centred.line_ne`, `MergeDepth.Centred.mem_line_of_not_reach` (Step 2 for points in different components) and `MergeDepth.Centred.not_mem_line`; Steps 4 and 5 are `MergeDepth.Centred.card_le_of_one` and `MergeDepth.Centred.card_le_of_two`; and the consequence for $q \ge 7$ is `MergeDepth.Centred.two_mul_card_lt`.

Bipartite-compatible sets are separated. Given χ, give L_p the colour $\chi(p)$ and R_p the colour $1-\chi(p)$. An edge L_hR_s has $\chi(h) = 1-\chi(s)$, so its two ends have the same colour, and colours are constant on the components of $E(S)$. Since L_s and R_s have different colours, they lie in different components.

From now on let S be separated and nonempty; for $S = \emptyset$ there is nothing to prove. Call distinct points $s, s' \in S$ *adjacent* if $N(s) \cap N(s') \ne \emptyset$, and call the connected components of this adjacency graph on S the *components* of S.

Step 1: the lines ℓ_s are distinct. If $\ell_s = \ell_{s'} = \ell$ with $s \ne s'$, pick $h \in \ell \setminus \{s, s'\}$, which exists since $|\ell| = q+1 \ge 3$. Then $s' \in N(s)$ and $h \in N(s) \cap N(s')$, so $L_{s'}R_sL_hR_{s'}$ is a path in $E(S)$, contradicting separation.

Step 2: two lines. For distinct $s, s' \in S$ at least one of the following holds: (a) $s \in \ell_{s'}$; (b) $s' \in \ell_s$; (c) s and s' are adjacent. Indeed, by Step 1 the lines ℓ_s and $\ell_{s'}$ meet in exactly one point z. If $z = s$, then (a) holds, and if $z = s'$, then (b) holds; otherwise $z \in N(s) \cap N(s')$, and (c) holds.

Step 3: components. If distinct s, s' lie in the same component, then neither (a) nor (b) holds. Indeed, a chain $s = s_0, s_1, \dots, s_m = s'$ of adjacent points with common helpers $h_i \in N(s_{i-1}) \cap N(s_i)$ gives the path $R_{s_0}L_{h_1}R_{s_1}\cdots L_{h_m}R_{s_m}$ in $E(S)$. If (a) held, then $s \in N(s')$, and the edge $L_sR_{s'}$ would join L_s to $R_{s'}$ and hence to R_s; (b) is excluded in the same way, with s and s' exchanged. So two distinct points of one component never lie on each other's lines, while two points of different components, which are not adjacent, satisfy (a) or (b) by Step 2.

Step 4: one component. Suppose that S has a single component, and put $t = |S| \ge 1$. By Step 3, a line ℓ_s contains no point of S other than s. Two lines ℓ_s and $\ell_{s'}$ with $s \ne s'$ are distinct by Step 1, so they meet in exactly one point, and that point lies off S, since a point of S on both lines would equal both s and s'. For a point $p \notin S$ let d_p be the number of $s \in S$ with $p \in \ell_s$. Every line ℓ_s has q points off S, and every ordered pair of distinct points of S has its two lines meeting in one point off S, so

$$\sum_{p \notin S} d_p = tq, \qquad \sum_{p \notin S} d_p(d_p-1) = t(t-1).$$

There are $v-t$ points off S, so the Cauchy–Schwarz inequality gives $(tq)^2 \le (v-t)(tq + t(t-1))$. Dividing by t and expanding with $v = q^2+q+1$ turns this into $(t-1)^2 \le q^3$, that is, $t \le q^{3/2}+1$.

Step 5: several components. Suppose that S has at least two components, let K be one of them, and pick $s' \in S \setminus K$. By Step 3 every $s \in K$ satisfies $s \in N(s')$ or $s' \in \ell_s$. At most $|N(s')| = q$ points satisfy the first condition. The lines ℓ_s of the points that satisfy the second are distinct by

Step 1, pass through s', and differ from $\ell_{s'}$ by Step 1; since only q lines through s' differ from $\ell_{s'}$, at most q points satisfy the second condition. Hence every component has at most $2q$ points. Now let Π be the set of unordered pairs of points of S that lie in different components. By Step 3, each pair in Π has one of its points among the helpers of the other, and every point has q helpers, so $|\Pi| \le q|S|$. If $K_1, \dots, K_n$ are the components, then

$$|\Pi| = \frac{1}{2}\Big(|S|^2 - \sum_{i=1}^{n} |K_i|^2\Big) \ge \frac{1}{2}(|S|^2 - 2q|S|),$$

because $\sum_i |K_i|^2 \le \max_i |K_i| \cdot |S| \le 2q|S|$. Hence $|S|^2 - 2q|S| \le 2q|S|$, that is, $|S| \le 4q$.

Step 6: the consequence. $4q < v/2$ is equivalent to $q^2 - 7q + 1 > 0$, which holds for $q \ge 7$, and $q^{3/2} + 1 < v/2$ is equivalent to $(q - \sqrt{q})^2 > 1$, which holds for $q \ge 3$. □

Remark 8.3 (Sharpness of the one-component bound)**.** When $q = r^2$ is a square, the bound of Step 4 is attained. The Hermitian curve $x^{r+1} + y^{r+1} + z^{r+1} = 0$ in $\mathrm{PG}(2, q)$ has $r^3 + 1 = q^{3/2} + 1$ points and meets every line in 1 or $r + 1$ points, a classical fact. The $q + 1$ lines through one of its points s partition the other r^3 points of the curve, and each of these lines contains 0 or r of them; so exactly $r^2 = q$ of the lines contain r of them, and one line meets the curve only in s. Taking that tangent line as ℓ_s for every point s of the curve gives a centred set whose helpers all lie off the curve; it is bipartite-compatible, with $\chi = 0$ on the curve and $\chi = 1$ elsewhere, hence separated. Two of its tangent lines meet in a point off the curve, a common helper, so it has a single component.

Remark 8.4 (Numerical)**.** This remark is numerical and is used in no proof. An exact constraint solver gives the largest bipartite-compatible centred sets in $\mathrm{PG}(2, q)$ for $q = 2, 3, 4, 5$: they have 6, 10, 10 and 12 points. For separated centred sets the same solver, with the components of Step 3 as variables and every optimum rechecked against the definition, gives the same maxima 6, 10 and 10 for $q = 2, 3, 4$; it did not settle $q = 5$ within its time limit. Since $(q^2 + q + 1)/2 = 3.5$, 6.5, 10.5 and 15.5, a single plane can recover half of its points for $q = 2$ and 3, but not for $q = 4$, nor, with a bipartite-compatible pattern, for $q = 5$. At $q = 2$ the binary code of the plane has rate $3/7$, below one half, and $q = 3$ is not a power of two.

9 Open problems

The following problems remain. A positive answer to Problem 9.3 would refute (T). Problems 9.1 and 9.2 concern the three kinds of machines that Theorem L leaves given [Q9.1] (Section 5), or rather, since [Q9.1] is false [8], given the native hypotheses that its proof uses; Problems 9.4 and 9.5 concern the hypotheses of the conditional results.

Problem 9.1 (Code-level depth reduction)**.** *Prove or refute Conjecture 3.22. A proof, together with the all-code crux for ball systems or with the native bound of Lemma 3.15(b), excludes every oblivious linear-time transposer (Remark 3.23(c)). Both hypotheses are open; [Q9.1], which implies the second, is false [8]. A counterexample would consist of oblivious linear-time transposers whose ball systems have no index codes of the conjectured length at any fixed radius; by Remark 3.23, their codes merge deep lineages unboundedly often, along merge DAGs without cheap cuts, in the deep part of their schedules: the mechanism that an oblivious construction needs.*

Problem 9.2 (Adaptive deep nesting)**.** *Given the native hypotheses of Section 2 in place of [Q9.1], which is false [8], Theorem 5.1 leaves adaptive machines that revisit some tape unboundedly often. Is there an analogue of hop charging (Theorem 3.5) for those that revisit a work tape, with the*

coarse skeleton of Section 4 in place of the fixed schedule? For those whose work tapes have bounded entry and which revisit input cells, the entropy condition of Corollary 4.16 can be met by padding (Remark 4.18); what else must their input-head revisits accomplish?

Problem 9.3 (An indivisibility-free external-memory bound)**.** *Is it true that for every finite alphabet and all constants $m \geq 1$, $\gamma \geq 0$ and $A \geq 0$, at all large d, every external-memory algorithm with block size $d\,b(d)$, m slots and a register of at most $\gamma \log_2(2L)$ bits that is correct and oblivious on a set of at least 2^{L-Ad} inputs performs more than Ad I/Os on that set? By Corollary 6.5, a positive answer refutes (T) for every number of work tapes; by its formal form, a lower bound at a single dimension for each alphabet and choice of constants already suffices. Bounds of this kind are known only when records are moved as indivisible atoms [1], or conditionally on a network-coding conjecture [14] that is false in general [2].*

Problem 9.4 (The crux at large degree)**.** *Answer Question 9.1 of [10] in the forms that remain open. The half bound is proved for all codes on the degree-three and directed classes of the seventh and tenth papers [7, 9], and it fails on a finite four-regular triangle-free graph [7], so the remaining question is asymptotic: degree four and growing odd girth, and the large degrees at which the cones of real machines live. At large degree the thirteenth paper has answered the question negatively, over the alphabet $\mathbb{F}_{127^2}$ and for binary codes, on relations that are not known to arise from machines [8, Theorem A and Corollary A′]. Open are the native form, for the relations of machines on which the conditional results rest (Section 2); the deficit form, rate at most $1 - \rho(\Delta) + o(1)$ for some $\rho(\Delta) > 0$ (Remark 4.19); the form for $\mathbb{F}_2$-linear codes; degree four with growing odd girth; and the least degree at which the answer is negative.*

Problem 9.5 (Concentration of straddle counts)**.** *Prove Conjecture 7.3; by Proposition 7.15 it is equivalent to showing that the non-local part of the normalized variance averages to zero. With Theorem 7.10 it gives the eighth paper's Conjecture A″ for pairs of block labels of bounded length at one odd shift [3, Conjecture 5.24], which is evidence that the deep-nesting pairs are depth-robust; the general labels that would refute* TCP(2) *[3, Corollary 5.26] remain.*

A Index of Lean declarations

The status lines of the paper cite the following declarations, listed by Lean module with their fully qualified names. Module files are relative to the paper's `lean/` directory. The axiom checks in `lean/PaperAxioms.lean` are generated from the same sources.

`MergeDepth.BlockMachine` `MergeDepth.BlockVal`, `MergeDepth.InClass`, `MergeDepth.Skeleton`, `MergeDepth.blockMachine`, `MergeDepth.blockMachine_card_cone_le`, `MergeDepth.blockMachine_cone`, `MergeDepth.blockMachine_isStorageCode_fibre`, `MergeDepth.blockMachine_output_eq`, `MergeDepth.blockVal`, `MergeDepth.blockVal_eq_of_args`, `MergeDepth.blockWidth`, `MergeDepth.block_oblivious`, `MergeDepth.card_outTime_eq_le`, `MergeDepth.outTime_spec`, `MergeDepth.runSkeleton`, `MergeDepth.skelClass`, `MergeDepth.skeleton_entries_ownerLoad`.

`MergeDepth.BlockPairsProfile` `MergeDepth.apexHat`, `MergeDepth.coarseZone`, `MergeDepth.goodCount`, `MergeDepth.goodLevels`, `MergeDepth.jZero`, `MergeDepth.meanRelVariance`, `MergeDepth.perTent`, `MergeDepth.piHat`, `MergeDepth.profile`, `MergeDepth.profileMean`, `MergeDepth.profileVariance`, `MergeDepth.relVariance`, `MergeDepth.relVarianceA`, `MergeDepth.tent`, `MergeDepth.valleyHat`.

MergeDepth.BlockPairsShift MergeDepth.good_shift_prob, MergeDepth.notGoodProb_le, MergeDepth.prob_some_bad_le.

MergeDepth.BlockPairsTransfer MergeDepth.prob_straddle_vs_profile.

MergeDepth.Capacity MergeDepth.card_deepHops_le, MergeDepth.card_deep_le_of_lt, MergeDepth.card_deep_mul_le, MergeDepth.two_mul_card_deepHops_le.

MergeDepth.CoarseCount MergeDepth.card_coarse_le, MergeDepth.card_monotone_le.

MergeDepth.Covering MergeDepth.covering.

MergeDepth.DeepNestingBP MergeDepth.HalfBlindPair, MergeDepth.InFamily, MergeDepth.class_card_le_family, MergeDepth.familyBlindErr_le_of_thetaServes, MergeDepth.familyPerPeriod, MergeDepth.familyPerPeriod_subset_class, MergeDepth.hopError_of_inFamily_blind, MergeDepth.prob_many_nonHalfBlindPair_le, MergeDepth.sum_nonHalfBlindPair_le, MergeDepth.theoremBP_pair, MergeDepth.witnesses_unserved_pair.

MergeDepth.EMModel MergeDepth.EM.Alg, MergeDepth.EM.CorrectAt, MergeDepth.EM.HaltsWith, MergeDepth.EM.ObliviousOn, MergeDepth.EM.Runs, MergeDepth.EM.inputDisk.

MergeDepth.ExpansionCone MergeDepth.C_eq_zero_of_not_mem_coneI, MergeDepth.coneI, MergeDepth.expansion_cone.

MergeDepth.ExternalMemory MergeDepth.EM.EMLowerBound, MergeDepth.EM.corollaryEM, MergeDepth.EM.corollaryEM_some, MergeDepth.EM.em_core, MergeDepth.EM.em_oblivious, MergeDepth.EM.theoremEM, MergeDepth.EM.theoremEM_sharp.

MergeDepth.FieldMachine MergeDepth.hop_charging_pruning, MergeDepth.hop_charging_pruning_bits, MergeDepth.short_paths_pruning.

MergeDepth.Formation MergeDepth.DeepFormation, MergeDepth.IsCopy, MergeDepth.copyForest, MergeDepth.dm_le_countP_deepFormation, MergeDepth.root.

MergeDepth.HCStarQ91 MergeDepth.ball_paired_packing_trace, MergeDepth.ball_rate_trace, MergeDepth.ball_self_demands_trace, MergeDepth.cor_HCstar_uniform, MergeDepth.cor_HCstar_uniform_empty, MergeDepth.cor_HCstar_uniform_frozen.

MergeDepth.HopFields MergeDepth.short_paths.

MergeDepth.HopMachine MergeDepth.cold_genie_exists, MergeDepth.hop_charging_HC.

MergeDepth.HopRoutingTrace MergeDepth.hop_routing_trace.

MergeDepth.InputSteering MergeDepth.cor_IS, MergeDepth.cor_RV.

MergeDepth.LinearCrux MergeDepth.FormationShallow, MergeDepth.LinCrux, MergeDepth.LinDataPath, MergeDepth.MergeShallow, MergeDepth.SBall, MergeDepth.cor_HC, MergeDepth.cor_HC_formation, MergeDepth.cor_HC_formation_uniform, MergeDepth.cor_HC_uniform.

MergeDepth.Localization MergeDepth.thm_L_a, MergeDepth.thm_L_b, MergeDepth.thm_L_c.

MergeDepth.LowAdaptivity MergeDepth.low_adaptivity_a, MergeDepth.low_adaptivity_a_exists, MergeDepth.low_adaptivity_b, MergeDepth.low_adaptivity_b_exists.

MergeDepth.MergeDag MergeDepth.MCut, MergeDepth.MEdge, MergeDepth.MInit.

MergeDepth.MergeSteps MergeDepth.merge_reduction_machine.

MergeDepth.ProjectivePlane MergeDepth.Centred, MergeDepth.Centred.BipCompat, MergeDepth.Centred.Separated, MergeDepth.Centred.card_le, MergeDepth.Centred.graph, MergeDepth.Centred.separated_of_bipCompat, MergeDepth.Centred.two_mul_card_lt.

MergeDepth.SkeletonEntropy MergeDepth.coarse_entropy_le, MergeDepth.theta_tendsto.

MergeDepth.SkeletonGenie MergeDepth.skel_genie_cost.

MergeDepth.SkeletonPaired MergeDepth.realizes_cone, MergeDepth.skel_disjoint_paths, MergeDepth.skel_disjoint_paths_transport, MergeDepth.skel_paired_packing, MergeDepth.skel_self_demands.

MergeDepth.SkeletonSE MergeDepth.skel_uniform_class, MergeDepth.thm_SE, MergeDepth.thm_SE_class.

MergeDepth.SkeletonWalk MergeDepth.card_block_changes_le, MergeDepth.card_entries_skel_le, MergeDepth.entries_skel_subset.

MergeDepth.StraddleDensity MergeDepth.density_W, MergeDepth.exact_form_V, MergeDepth.sideband_eq_densityW.

MergeDepth.StraddleLocal MergeDepth.conjectureS_iff_meanNonLocalGen, MergeDepth.relVariance_decomp, MergeDepth.sum_fineFloorK_le, MergeDepth.sum_semiLocal_le.

MergeDepth.StraddleProgression MergeDepth.progression_avg_le_strong, MergeDepth.window_sum_prod_cos_sq.

MergeDepth.StraddleResonance MergeDepth.resonance_class.

MergeDepth.Transport MergeDepth.transport, MergeDepth.transport_sharp, MergeDepth.transport_tpos.

Also named in the text From this paper's library: `MergeDepth.BlockFunDag`, `MergeDepth.BlockFunDag.card_coneJ_le`, `MergeDepth.BlockFunDag.cone`, `MergeDepth.BlockFunDag.coneJ`, `MergeDepth.BlockFunDag.isStorageCode_fibre`, `MergeDepth.BlockFunDag.output_eq`, `MergeDepth.BlockStep`, `MergeDepth.Centred.card_le_of_one`, `MergeDepth.Centred.card_le_of_two`, `MergeDepth.Centred.line_ne`, `MergeDepth.Centred.mem_line_of_not_reach`, `MergeDepth.Centred.not_mem_line`, `MergeDepth.ConjectureS`, `MergeDepth.DeepStep`, `MergeDepth.DepStep`, `MergeDepth.EM.emAlg`, `MergeDepth.EM.theoremEM_nat`, `MergeDepth.Forest`, `MergeDepth.FunDag.cone`, `MergeDepth.FunDag.isIndexCode`, `MergeDepth.IsBlind`, `MergeDepth.IsBlockLabel`, `MergeDepth.IsBlockLabel.potentialOf_eq`, `MergeDepth.IsColdGenie`, `MergeDepth.IsOneExceptionConstant`, `MergeDepth.IsRun`, `MergeDepth.IsSkeletonConstant`, `MergeDepth.LinDataPath.minrank_ballSystem_le`, `MergeDepth.LinDataPath.minrank_ballSystem_le_formation`, `MergeDepth.LinDataPath.net`, `MergeDepth.LinDataPath.net_depStep`, `MergeDepth.LinDataPath.rigidity_machine`, `MergeDepth.LinDataPath.trans_of_transposer`, `MergeDepth.LinDataPath.val_card_le`, `MergeDepth.LinDataPath.val_eq_enc`, `MergeDepth.ObliviousTransposer`, `MergeDepth.Pruning`, `MergeDepth.Pruning.field_cone`, `MergeDepth.Pruning.field_isIndexCode`, `MergeDepth.Pruning.full`, `MergeDepth.Pruning.toFunDag`, `MergeDepth.Q91`, `MergeDepth.Q91.binary`, `MergeDepth.Q91Binary`, `MergeDepth.ThetaServes`, `MergeDepth.ThetaServes.servedOnWitness`, `MergeDepth.TransposesAt`, `MergeDepth.ball_paired_packing`, `MergeDepth.ball_rate_binary`, `MergeDepth.ball_self_demands`, `MergeDepth.ball_trimming`, `MergeDepth.blindnessBound_holds`, `MergeDepth.blockPairLabel`, `MergeDepth.blockPairLabel_add_mul`, `MergeDepth.blockStep_iff`, `MergeDepth.blockVal_eq_of_coarseAgree`, `MergeDepth.blockVal_mem_admissible`, `MergeDepth.cantelli_interval`, `MergeDepth.card_admissible_le`, `MergeDepth.card_allHops_le`, `MergeDepth.card_blockIn_le`, `MergeDepth.card_blockOut_le`, `MergeDepth.card_coneV_le`, `MergeDepth.card_correctSet_le_a`, `MergeDepth.card_correctSet_le_b`, `MergeDepth.card_correctSet_le_of_class`, `MergeDepth.card_coverClass`, `MergeDepth.card_exceptional_eq`, `MergeDepth.card_revealed_le`, `MergeDepth.card_straddleSet_eq`, `MergeDepth.card_traces_ge_a`, `MergeDepth.card_turns_period`, `MergeDepth.card_unit_le`, `MergeDepth.case1_unserved`, `MergeDepth.case2_unserved`, `MergeDepth.case3_unserved`, `MergeDepth.charFun_lawW_eq_straddlePhi`, `MergeDepth.classRel_subset_coneRel`, `MergeDepth.classStraddleCount_eq_profile`, `MergeDepth.classStraddleCount_false_eq`, `MergeDepth.cold_genie`, `MergeDepth.coneRel`, `MergeDepth.coneV`, `MergeDepth.conjectureS_iff_meanNonLocal`, `MergeDepth.coordNet`, `MergeDepth.coord_paths`, `MergeDepth.coord_reach_subset`, `MergeDepth.coord_val`, `MergeDepth.cor_HCstar_frozen`, `MergeDepth.cor_HCstar_pruning`, `MergeDepth.cor_HCstar_pruning_empty`, `MergeDepth.cor_HCstar_pruning_sym`, `MergeDepth.cor_HCstar_step`, `MergeDepth.deepHops`, `MergeDepth.density_W_model_exists`, `MergeDepth.disjoint_paths_bound`, `MergeDepth.disjoint_paths_transport`, `MergeDepth.dm`, `MergeDepth.dm_le_of_cut`, `MergeDepth.dnpWalk`, `MergeDepth.dnpWalk_V`, `MergeDepth.entropy`, `MergeDepth.entropy_comp_le`, `MergeDepth.entropy_le_of_fibres`, `MergeDepth.exists_large_fibre`, `MergeDepth.exists_large_fibre_card`, `MergeDepth.family`, `MergeDepth.familyBlindErr_le_of_served`, `MergeDepth.family_card_ge`, `MergeDepth.family_inHopClass`, `MergeDepth.fibreGeometry`, `MergeDepth.fibreGeometry_deficit_le`, `MergeDepth.fineFloorK_eq`, `MergeDepth.fineFloorK_le_relVariance`, `MergeDepth.fineZone`, `MergeDepth.finrank_revealed_coord_le`, `MergeDepth.finrank_revealed_le`, `MergeDepth.finrank_revealed_le_of_values`, `MergeDepth.finrank_revealed_union_le`, `MergeDepth.frozenGeometry`, `MergeDepth.frozen_coarse_entropy_le`, `MergeDepth.frozen_coarse_entropy_le_of_halts`, `MergeDepth.halfBlindPair_iff`, `MergeDepth.hasSpanColoring_iff_activeFamilies_card_le`, `MergeDepth.hasSum_phiGen`, `MergeDepth.hasSum_straddlePhi_ne_zero`, `MergeDepth.hballs_of_obliviousTransposer`, `MergeDepth.hopError`, `MergeDepth.hopError_blockPairLabel`, `MergeDepth.hop_charging`, `MergeDepth.hop_charging_fields`, `MergeDepth.hop_charging_fields_heads`, `MergeDepth.hop_charging_machine`, `MergeDepth.hop_charging_machine_real`,

MergeDepth.hop_charging_pruning_HC, MergeDepth.hop_charging_pruning_sym, MergeDepth.hop_charging_steps, MergeDepth.hop_routing_a, MergeDepth.hop_routing_b, MergeDepth.hop_routing_count, MergeDepth.inClass_iff_runSkeleton, MergeDepth.inSkel, MergeDepth.inSkel_entropy_le, MergeDepth.isStorageCode_fibre_of_indexCode, MergeDepth.isTurnZ_dnpWalk_iff, MergeDepth.length_le_of_coldDep, MergeDepth.length_le_of_cov, MergeDepth.length_le_of_cut, MergeDepth.length_le_of_mem_paths, MergeDepth.levelCount_eq, MergeDepth.levelHops_eq_listed, MergeDepth.logb_card_le_of_conditioning, MergeDepth.merge_reduction, MergeDepth.merge_reduction_fields, MergeDepth.merge_reduction_pruning, MergeDepth.merge_reduction_steps, MergeDepth.outTime, MergeDepth.paid_trimming, MergeDepth.pairHopErr_of_blind, MergeDepth.paths_short, MergeDepth.pot_run, MergeDepth.prevS, MergeDepth.prob_many_nonHalfBlind_le', MergeDepth.profileMean_eq, MergeDepth.rCount_deep_coarse_eq_zero, MergeDepth.rCount_le_card_edgesOf, MergeDepth.realize_paths, MergeDepth.realizes_ball, MergeDepth.rigidity_core_rev, MergeDepth.rigidity_finrank, MergeDepth.routing_count, MergeDepth.rpow_mono_of_le, MergeDepth.skel, MergeDepth.skelData, MergeDepth.skel_genie, MergeDepth.skel_paired_separation, MergeDepth.skel_trimming, MergeDepth.skel_uniform_binary, MergeDepth.skeleton_genie, MergeDepth.stepBall, MergeDepth.stepValue, MergeDepth.stepValue_eq_of_parents, MergeDepth.stepValue_eq_of_preds, MergeDepth.straddlePhi, MergeDepth.subrel_paired_packing, MergeDepth.subrel_self_demands, MergeDepth.sum_choose_le, MergeDepth.sum_nonHalfBlind_le', MergeDepth.theoremBP', MergeDepth.trace, MergeDepth.traceProb, MergeDepth.traceProb_le_a, MergeDepth.traceProb_le_b, MergeDepth.trace_ball_data, MergeDepth.uniform_rate, MergeDepth.uniform_rate_binary, MergeDepth.valueWidth, MergeDepth.witnesses_unserved.
From the first paper's library: Transpose.LinearTimeTransposable.
From the eighth paper's library: BoundedNesting.LinNet.vlin_expand, BoundedNesting.card_heavyPairs_mul_le, BoundedNesting.entries, BoundedNesting.heavyPairs, BoundedNesting.lightHops, BoundedNesting.theoremF_owner.
From the fourth paper's library: MotionDepthCodes.IsStorageCode.
From the fifth paper's library: DirectedRecovery.self_demands_count_of_disjoint_transport, DirectedRecovery.short_paired_count_log_bound.
From the ninth paper's library: SourceCovers.finite_skeleton_uniform_deficit, SourceCovers.one_exception_uniform_deficit.
From the tenth paper's library: DirectedReciprocity.blindApexErr, DirectedReciprocity.blindBandErr, DirectedReciprocity.block_overflow_band_witness, DirectedReciprocity.block_straddle_residue_ge_half_q, DirectedReciprocity.cancellerBelow, DirectedReciprocity.cancellerBelow_props, DirectedReciprocity.classStraddleCount, DirectedReciprocity.periodVApices_spans_disjoint.

References

[1] Alok Aggarwal and Jeffrey Scott Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988. doi:10.1145/48529.48535.

[2] Mark Braverman and Zhongtian He. The multiple unicast conjecture is false. arXiv:2610.10108, 2026. URL: https://arxiv.org/abs/2610.10108.

[3] Hans Brende and Claude Opus 5.5 (Anthropic). Bounded nesting and depth reduction for matrix transposition on tapes. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision fd5db85a2b5baba976046138ecf2bb3ec8f2cb13. URL: https://github.com/HansBrende/papers/tree/fd5db85a2b5baba976046138ecf2bb3ec8f2cb13/Bounded-Nesting-and-Depth-Reduction-for-Matrix-Transposition-on-Tapes.

[4] Hans Brende and Claude Opus 5.5 (Anthropic). Curved motion, depth reduction, and storage codes for matrix transposition on tapes. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision fd5db85a2b5baba976046138ecf2bb3ec8f2cb13. URL: https://github.com/HansBrende/papers/tree/fd5db85a2b5baba976046138ecf2bb3ec8f2cb13/Curved-Motion-Depth-Reduction-and-Storage-Codes-for-Matrix-Transposition-on-Tapes.

[5] Hans Brende and Claude Opus 5.5 (Anthropic). Frozen gap areas and the sharpness of source-cover certificates. Memo and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision fd5db85a2b5baba976046138ecf2bb3ec8f2cb13. URL: https://github.com/HansBrende/papers/tree/fd5db85a2b5baba976046138ecf2bb3ec8f2cb13/Source-Covers-and-Barriers-to-Tape-Transposition/memo-gap-area.

[6] Hans Brende and Claude Opus 5.5 (Anthropic). Matrix transposition and the tape coding conjecture. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision fd5db85a2b5baba976046138ecf2bb3ec8f2cb13. URL: https://github.com/HansBrende/papers/tree/fd5db85a2b5baba976046138ecf2bb3ec8f2cb13/Matrix-Transposition-and-the-Tape-Coding-Conjecture.

[7] Hans Brende and Claude Opus 5.5 (Anthropic). Nested elimination and half-rate bounds for storage codes. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision fd5db85a2b5baba976046138ecf2bb3ec8f2cb13. URL: https://github.com/HansBrende/papers/tree/fd5db85a2b5baba976046138ecf2bb3ec8f2cb13/Nested-Elimination-and-Half-Rate-Bounds-for-Storage-Codes.

[8] Hans Brende and Claude Opus 5.5 (Anthropic). Storage codes above half rate under paired-distance separation. Manuscript and Lean formalization, repository HansBrende/papers, November 2026. Pinned revision 13b224bd021094b39a57074101c824d15432ed53. URL: https://github.com/HansBrende/papers/tree/13b224bd021094b39a57074101c824d15432ed53/Storage-Codes-above-Half-Rate-under-Paired-Distance-Separation.

[9] Hans Brende and Gemini 4 Argon (Google DeepMind). Directed reciprocity and deep-nesting bounds for matrix transposition on tapes. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision fd5db85a2b5baba976046138ecf2bb3ec8f2cb13. URL: https://github.com/HansBrende/papers/tree/fd5db85a2b5baba976046138ecf2bb3ec8f2cb13/Directed-Reciprocity-and-Deep-Nesting-Bounds-for-Matrix-Transposition-on-Tapes.

[10] Hans Brende and GPT-6 Astra (OpenAI). Directed recovery structures in tape computations. Manuscript and partial Lean formalization, repository HansBrende/papers, October 2026. Pinned revision fd5db85a2b5baba976046138ecf2bb3ec8f2cb13. URL: https://github.com/HansBrende/papers/tree/fd5db85a2b5baba976046138ecf2bb3ec8f2cb13/Directed-Recovery-Structures-in-Tape-Computations.

[11] Hans Brende and GPT-6 Astra (OpenAI). Source covers and barriers to tape transposition. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision fd5db85a2b5baba976046138ecf2bb3ec8f2cb13. URL: https://github.com/HansBrende/papers/tree/fd5db85a2b5baba976046138ecf2bb3ec8f2cb13/Source-Covers-and-Barriers-to-Tape-Transposition.

[12] Martin Dietzfelbinger and Wolfgang Maass. The complexity of matrix transposition on one-tape off-line Turing machines with output tape. *Theoretical Computer Science*, 108(2):271–290, 1993.

[13] Martin Dietzfelbinger, Wolfgang Maass, and Georg Schnitger. The complexity of matrix transposition on one-tape off-line Turing machines. *Theoretical Computer Science*, 82(1):113–129, 1991. doi:10.1016/0304-3975(91)90175-2.

[14] Alireza Farhadi, MohammadTaghi Hajiaghayi, Kasper Green Larsen, and Elaine Shi. Lower bounds for external memory integer sorting via network coding. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC 2019)*. ACM, 2019. Also arXiv:1811.01313.

[15] Walter W. Kirchherr. Transposition of an $l \times l$ matrix requires $\Omega(\log l)$ reversals on conservative Turing machines. *Information Processing Letters*, 28(2):55–59, 1988. doi:10.1016/0020-0190(88)90163-9.